

# WaveForms™ SDK Reference Manual

Revised February 15, 2024

---

## Table of Contents

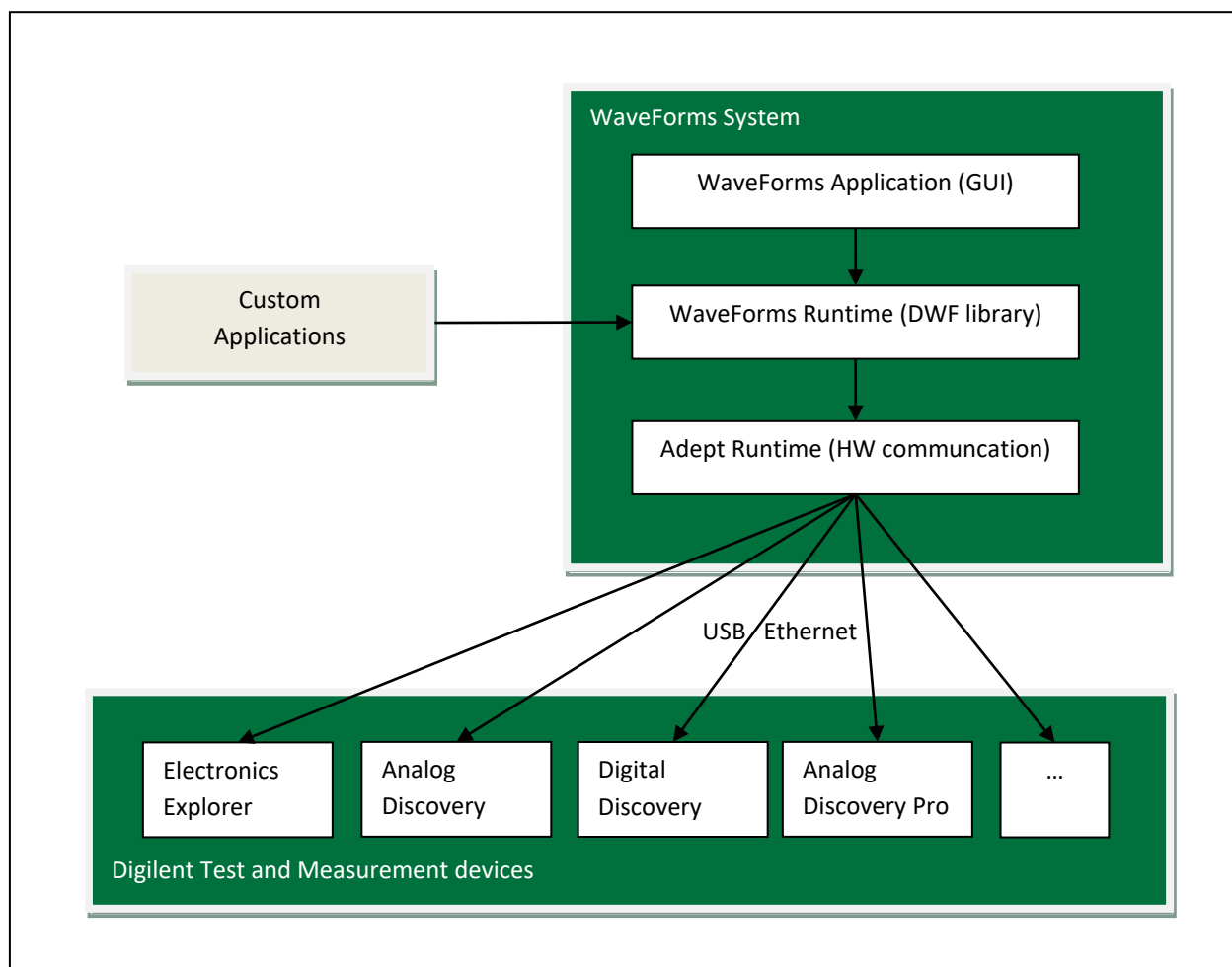
|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Table of Contents .....</b>                           | <b>1</b>  |
| <b>Overview .....</b>                                    | <b>3</b>  |
| <b>1 The System .....</b>                                | <b>4</b>  |
| 1.1 The API.....                                         | 5         |
| 1.2 Calling API Functions.....                           | 7         |
| <b>2 System .....</b>                                    | <b>8</b>  |
| <b>3 Device Enumeration.....</b>                         | <b>10</b> |
| <b>4 Device Control .....</b>                            | <b>13</b> |
| <b>5 Analog In (Oscilloscope).....</b>                   | <b>18</b> |
| 5.1 Control.....                                         | 19        |
| 5.2 Configuration.....                                   | 24        |
| 5.3 Channels .....                                       | 29        |
| 5.4 Trigger .....                                        | 37        |
| 5.5 Trigger Detector .....                               | 42        |
| <b>6 Analog Out (Arbitrary Waveform Generator) .....</b> | <b>48</b> |
| 6.1 Control.....                                         | 49        |
| 6.2 Configuration.....                                   | 52        |
| 6.3 States.....                                          | 62        |
| <b>7 Analog I/O .....</b>                                | <b>67</b> |
| <b>8 Digital I/O .....</b>                               | <b>72</b> |

|           |                                              |            |
|-----------|----------------------------------------------|------------|
| <b>9</b>  | <b>Digital In (Logic Analyzer)</b> .....     | <b>78</b>  |
| 9.1       | Control.....                                 | 79         |
| 9.2       | Configuration.....                           | 82         |
| 9.3       | Trigger .....                                | 88         |
| 9.4       | Trigger Detector .....                       | 92         |
| <b>10</b> | <b>Digital Out (Pattern Generator)</b> ..... | <b>95</b>  |
| 10.1      | Control.....                                 | 98         |
| 10.2      | Configuration.....                           | 98         |
| <b>11</b> | <b>Miscellaneous</b> .....                   | <b>110</b> |
| <b>12</b> | <b>Analog Impedance</b> .....                | <b>112</b> |
| <b>13</b> | <b>Digital Protocols</b> .....               | <b>119</b> |
| 13.1      | UART .....                                   | 119        |
| 13.2      | SPI .....                                    | 121        |
| 13.3      | I2C .....                                    | 131        |
| 13.4      | CAN .....                                    | 134        |
| 13.5      | SWD.....                                     | 136        |
| <b>14</b> | <b>Devices</b> .....                         | <b>139</b> |
| 14.1      | Electronics Explorer.....                    | 139        |
| 14.2      | Analog Discovery .....                       | 140        |
| 14.3      | Analog Discovery 2 .....                     | 141        |
| 14.4      | Digital Discovery.....                       | 142        |
| 14.5      | Analog Discovery Pro 3000 Series .....       | 143        |
| 14.6      | Analog Discovery Pro 5250.....               | 144        |
| 14.7      | Eclipse Z7 Zmods .....                       | 145        |
| 14.8      | Discovery Power Supply 3340 .....            | 146        |
| 14.9      | Analog Discovery 3 .....                     | 147        |
| <b>15</b> | <b>Deprecated functions</b> .....            | <b>148</b> |

## Overview

WaveForms™ provides an interface that allows users to interact with Diligent Analog Design hardware, such as the Analog Discovery™, Analog Discovery 2™, Analog Discovery Pro™, Digital Discovery™, Discovery Power Supply™ and Electronics Explorer™. While the WaveForms application offers a refined graphical interface, the WaveForms SDK provides access to a public application programming interface (API) that gives users the ability to create custom PC applications.

This WaveForms SDK manual describes the main components and architecture of the WaveForms system and details each function contained in the WaveForms API. The SDK package also offers examples demonstrating how to identify, connect to, and control analog hardware devices.



# 1 The System

The WaveForms System is comprised of multiple components. The most visible component is the WaveForms Application; a suite of graphical instrument panels that give full access to the analog and digital instruments in the connected hardware. The WaveForms application uses WaveForms Runtime to communicate with the device. For a custom application it is sufficient to access only the WaveForms Runtime, but both of these Runtimes are required on the target machine in order to run the application. The WaveForms Runtime is comprised of the *DWF* Dynamic Library and several configuration files. This library is located in:

- Windows in System Directory: C:\Windows\System32\dwf.dll
- Linux: /usr/lib/libdwf.so.x.x.x

The static library is located in Windows through the install path:

- Windows 32-bit: C:\Program Files (x86)\Diligent\WaveFormsSDK\lib\x86
- Windows 64-bit: C:\Program Files (x86)\Diligent\WaveFormsSDK\lib\x64
- Windows ARM64: C:\Program Files (x86)\Diligent\WaveFormsSDK\lib\arm64
- Windows ARM64EC: C:\Program Files (x86)\Diligent\WaveFormsSDK\lib\arm64ec

The C header file is located in:

- Windows 32-bit: C:\Program Files\Diligent\WaveFormsSDK\inc
- Windows 64-bit: C:\Program Files (x86)\Diligent\WaveFormsSDK\inc
- Linux: /usr/include/diligent/waveforms

Working code examples are provided with the SDK to demonstrate basic use of each API function set. You can find samples in the installation directory, which are located here:

- Windows 32-bit: C:\Program Files\Diligent\WaveFormsSDK\samples
- Windows 64-bit: C:\Program Files (x86)\Diligent\WaveFormsSDK\samples
- Linux: /usr/share/diligent/waveforms/samples
- OS X: /Applications/WaveForms.app/Contents/Resources/SDK/

The DWF Library uses Diligent Adept Runtime, which provides basic communication with the targeted hardware instruments (i.e., Analog Discovery and Electronics Explorer). Although the Adept Runtime is an integral part of the WaveForms System, knowledge of its structure *is not required* to write custom applications.

On Mac OS X the WaveForms Runtime (including Adept Runtime) is installed to:

- /Library/Frameworks/dwf.framework

The sample path contains the following directories:

- py: several Python script examples
- C: some C code examples
- cs: C# wrapper, dwf class for the API functions
- vb: VisualBasic wrapper, dwf module for the API function
- dwfcmd: a complex C example application

## 1.1 The API

Everything needed to write custom applications is included in the WaveForms SDK, which provides the header/library files and documentation to access the API for the DWF Library. A custom application must properly link to these files to make the appropriate API function calls. Every function in the WaveForms public API is declared in the `dwf.h` header file.

Basic usage of the WaveForms API can be broken down into the following steps:

1. Call enumeration functions to discover connected hardware devices.
2. Call *FDwfDeviceOpen* function to establish a connection to specific hardware device.
3. Call function to enable instrument within hardware device.
4. Call functions to configure instrument and acquire/generate signals.
5. Call function to disable instrument.
6. Call *FDwfDeviceClose* function to disconnect from device.

There are nine main groups of API functions, each named with different prefixes:

| Main Groups of API Functions                | Instrument Function                                                        | Prefix               |
|---------------------------------------------|----------------------------------------------------------------------------|----------------------|
| Device Enumeration                          | Controls the enumeration of connected and supported devices.               | <i>DwfEnum</i>       |
| Device Control                              | Controls opening and closing specific devices.                             | <i>DwfDevice</i>     |
| AnalogIn<br>(Oscilloscope)                  | Acquires samples from each enabled channel synchronously.                  | <i>DwfAnalogIn</i>   |
| AnalogOut<br>(Arbitrary Waveform Generator) | Drives signals from each channel independently.                            | <i>DwfAnalogOut</i>  |
| AnalogIO                                    | Acquires and drives various analog signals.                                | <i>DwfAnalogIO</i>   |
| DigitalIn (Logic Analyzer)                  | Acquires samples from digital I/O pins.                                    | <i>DwfDigitalIn</i>  |
| DigitalOut (Pattern Generator)              | Drives digital I/O signals.                                                | <i>DwfDigitalOut</i> |
| DigitalIO                                   | Acquires and drives digital I/O signals.                                   | <i>DwfDigitalIO</i>  |
| System                                      | Obtain basic system information that is instrument and device independent. | <i>DwfGet</i>        |

Each instrument is directly controlled using three types of functions in the API:

| API Functions             | Instrument Function                                                      | Example                       |
|---------------------------|--------------------------------------------------------------------------|-------------------------------|
| <b>Reset function</b>     | This function resets all of the instrument parameters to default values. | <i>FDwfAnalogInReset</i>      |
|                           |                                                                          | <i>FDwfAnalogOutReset</i>     |
|                           |                                                                          | <i>FDwfDigitalIOReset</i>     |
| <b>Configure function</b> | This function configures and/or starts the instrument.                   | <i>FDwfAnalogInConfigure</i>  |
|                           |                                                                          | <i>FDwfAnalogOutConfigure</i> |
|                           |                                                                          | <i>FDwfDigitalIOConfigure</i> |
| <b>Status function</b>    | This function polls and reads all information from the instrument.       | <i>FDwfAnalogInStatus</i>     |
|                           |                                                                          | <i>FDwfAnalogOutStatus</i>    |
|                           |                                                                          | <i>FDwfDigitalIOStatus</i>    |

Note: Although there are multiple “Status” functions for each instrument, these functions are the only ones that actually read data from the device.

There are several type definitions and corresponding constants in the dwf.h include file. The majority of them are used as parameters. When a hardware device is opened, a handle is returned (HDWF), which is used to access and finally close in all instrument API functions.

The following examples are provided in Python and C++ language.

| File                                   | Description                                                            |
|----------------------------------------|------------------------------------------------------------------------|
| Device_Enumeration                     | List the supported and connected devices.                              |
| AnalogIO_AnalogDiscovery_SystemMonitor | Reading the system monitor information                                 |
| AnalogIO_AnalogDiscovery_Power         | Enable power supplies.                                                 |
| AnalogOut_Sine                         | Generate sine waveform on analog out channel.                          |
| AnalogOut_Sweep                        | Generate frequency sweep.                                              |
| AnalogOut_Custom                       | Arbitrary waveform generation.                                         |
| AnalogOut_Sync                         | How to synchronize the analog output channels                          |
| AnalogOutIn                            | Generate analog output signal and perform analog in acquisition.       |
| AnalogIn_Sample                        | Open the first device, configure analog in and read single sample.     |
| AnalogIn_Acquisition                   | Perform acquisition and plot data for first channel.                   |
| AnalogIn_Trigger                       | Perform triggered acquisition.                                         |
| AnalogIn_Record                        | Performs recording of large number of samples.                         |
| DigitalIO                              | Drive and read digital IO pins                                         |
| DigitalOut_Pins                        | Generate pulse, random and custom signal on digital out pins.          |
| DigitalOut_BinaryCounter               | Generate binary counter                                                |
| DigitalIn_Acquisition                  | Generate signals on digital out and perform acquisition on digital in. |
| DigitalIn_Record                       | Perform recording of large number of digital in samples.               |

To compile and link the c/cpp applications with the dwf library use:

Window: `gcc device_enumeration.cpp -L../lib/x86 -ldwf`

Linux: `gcc device_enumeration.cpp -ldwf`

MacOS: `gcc device_enumeration.cpp -F/Library/Frameworks -framework dwf`  
`install_name_tool -add_rpath /Library/Frameworks a.out`  
 or add `dwf.framework` to your app bundle/Frameworks

## 1.2 Calling API Functions

The API functions are C style and return a integer value: 1 if the call is successful, 0 if unsuccessful. Further information about the failure can be obtained using the `FDwfGetLastError` and `FDwfGetLastErrorMsg` functions. In general, the API functions contain variations of the following parameters:

| Parameters     | Parameter Function                                                                                                                                                               |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>*Info</b>   | Returns detailed information about the parameter support for the instrument (i.e., minimum/maximum values, supported modes, etc.)                                                |
| <b>*Set</b>    | Sets an instrument parameter. When the AutoConfigure is enabled (by default), the instrument is reconfigured and stopped.                                                        |
| <b>*Get</b>    | Gets the actual instrument parameter. Use this function to get the actual set value. For instance, an arbitrary voltage offset is set and Get returns the real DAC output value. |
| <b>*Status</b> | Returns the parameter value from the device.                                                                                                                                     |

The API functions won't fail when a parameter pointer is NULL or when a setting (\*Set) parameter value is out of limits. To verify the actual setting value, use \*Get API to return the actual value.

The indices used in function parameters are zero based.

The supported discrete parameters are retrieved in bit field value. To decode the capabilities of the device, use the `IsBitSet` macro.

```
int fsfilter;
FDwfAnalogInChannelFilterInfo(h, &fsfilter)
if(IsBitSet(fsfilter, filterAverage)){
    FDwfAnalogInChannelFilterSet(hdwf, 0, filterAverage)
}
```

For better performance it is recommended to disable AutoConfigure after opening a device. This will prevent each `FDwf*Set` call to communicate with the device which can take considerable time.

```
FDwfDeviceAutoConfigureSet(hdwf, 0)
```

Details about eventual errors can be displayed like this:

```
bool myFunction(){
    if(!FDwf...(...)) return false
    if(!FDwf...(...)) return false
    ...
    return true
}
if(!myFunction()){
    char szError[512]
    FDwfGetLastErrorMsg(szError)
    print(szError)
}
```

## 2 System

**FDwfGetLastError** (DWFERC \*pdwferc)

**Description:** Retrieves the last error code in the calling process. The error code is cleared when other API functions are called and is only set when an API function fails during execution. Error codes are declared in dwf.h:

| DWFERC                  | int  | Error Code Definition                           |
|-------------------------|------|-------------------------------------------------|
| dwfercNoErc             | 0    | No error occurred.                              |
| dwfercUnknownError      | 1    | Unknown error.                                  |
| dwfercApiLockTimeout    | 2    | Call waiting on pending API time out.           |
| dwfercAlreadyOpened     | 3    | Device already opened.                          |
| dwfercNotSupported      | 4    | Device or function not supported by the device. |
| dwfercInvalidParameter0 | 0x10 | Parameter 0 was invalid in last API call.       |
| dwfercInvalidParameter1 | 0x11 | Parameter 1 was invalid in last API call.       |
| dwfercInvalidParameter2 | 0x12 | Parameter 2 was invalid in last API call.       |
| dwfercInvalidParameter3 | 0x13 | Parameter 3 was invalid in last API call.       |

**Parameters:**

- pdwferc – Variable to receive error code.

**FDwfGetLastErrorMsg** (char szError[512])

**Description:** Retrieves the last error message. This may consist of a chain of messages, separated by a new line character, that describe the events leading to the failure.

**Parameters:**

- szError – Pointer to buffer to receive error string.

**FDwfGetVersion** (char szVersion[32])

**Description:** Retrieves the version string. The version string is composed of major, minor, and build numbers (i.e., "2.0.19").

**Parameters:**

- szVersion – Pointer to buffer to receive version string.



```
FDwfParamSet(DwfParam param, int value)
```

**Description:** Configures various global parameters that will be applied on newly opened devices.

| DwfParam               | int |                                                                                                                                                                                                                                                |
|------------------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DwfParamLedBrightness  | 3   | 0..100 % LED brightness<br>Supported by Digital Discovery                                                                                                                                                                                      |
| DwfParamOnClose        | 4   | Device close behavior.<br>0 = Continue, keep running<br>1 = Stop the device outputs but keep the device operational to prevent temperature drifts<br>2 = Shutdown device to minimize power consumption                                         |
| DwfParamAudioOut       | 5   | 1 = Enable audio output (default)<br>0 = Disable audio output<br>Supported by Analog Discovery 1 and 2                                                                                                                                         |
| DwfParamUsbLimit       | 6   | USB current limitation in mA, recommended value 600-1000.<br>Supported by Analog Discovery 1 and 2                                                                                                                                             |
| DwfParamAnalogOut      | 7   | 1 = Enable Wavegen outputs (default)<br>0 = Disable<br>Supported by ADP3X50                                                                                                                                                                    |
| DwfParamFrequency      | 8   | Adjust system frequency in Hz (default 100MHz)<br>Supported by Digital Discovery, ADP3X50, Analog Discovery 3                                                                                                                                  |
| DwfParamExtFreq        | 9   | Specify for input or set reference output frequency in Hz (default 10MHz)<br>Supported by ADP3X50, Analog Discovery 3                                                                                                                          |
| DwfParamClockMode      | 10  | 0 = use internal oscillator (default)<br>1 = enable reference output on Trigger IO 1<br>2 = use reference input from Trigger IO 1<br>3 = use Trigger IO 1 as reference input-output (only ADP3X50)<br>Supported by ADP3X50, Analog Discovery 3 |
| DwfParamTempLimit      | 11  | Specifies the over temperature threshold in degree Celsius on devices which support such option.                                                                                                                                               |
| DwfParamFreqPhase      | 12  | Specifies the system clock phase which is useful for device synchronization when reference input clock is used.                                                                                                                                |
| DwfParamDigitalVoltage | 13  | Adjusts the digital IO voltage in mV.<br>Supported by Digital Discovery, ADP3X50                                                                                                                                                               |

**Parameters:**

- param – Pointer to buffer to receive version string.
- value – Value to set.

```
FDwfParamGet(DwfParam param, int *pValue)
```

**Description:** Retrieves the parameter value.

**Parameters:**

- param – Parameter.
- pValue – Pointer to value.

### 3 Device Enumeration

The *FDwfEnum* functions are used to discover all connected, compatible devices.

See examples: Device\_Enumeration.py, Device\_Info.py, Device\_InfoEx.py.

**FDwfEnum**(ENUMFILTER enumfilter, int \*pnDevice)

**Description:** Builds an internal list of detected devices filtered by the *enumfilter* parameter. It must be called before using other *FDwfEnum* functions because they obtain information about enumerated devices from this list identified by the device index.

| ENUMFILTER        | int       |                                                |
|-------------------|-----------|------------------------------------------------|
| enumfilterAll     | 0         | Enumerate all supported devices                |
| DEVID             | devid     | Use devid to filter specific devices           |
| enumfilterType    | 0x8000000 | Use in conjunction with the following filters: |
| enumfilterUSB     | 0x0000001 | USB devices                                    |
| enumfilterNetwork | 0x0000002 | Network devices                                |
| enumfilterAXI     | 0x0000004 | Embedded devices                               |
| enumfilterRemote  | 0x1000000 | Remote table devices                           |
| enumfilterAudio   | 0x2000000 | Sound card device                              |
| enumfilterDemo    | 0x4000000 | Demo devices                                   |

**Parameters:**

- enumfilter – Filter value to be used for device enumeration. Use the *enumfilterAll* constant to discover all compatible devices or “enumfilterType| enumfilterUSB”
- pnDevice – Integer pointer to return count of found devices by reference.

Example:

FDwfEnum(enumfilterAll, &nDev) # list all devices

FDwfEnum(devidDiscovery3, &nDev) # list Analog Discovery 3 devices

FDwfEnum(enumfilterType|enumfilterUSB|enumfilterAXI, &nDev) # list USB and AXI devices

**FDwfEnumStart**(ENUMFILTER enumfilter)

**Description:** Starts enumerating the devices. Nonblocking alternative for *FDwfEnum*.

**Parameters:**

- enumfilter – Filter value to be used for device enumeration. Use the *enumfilterAll* constant to discover all compatible devices.

**FDwfEnumStop**(int \*pnDevice)

**Description:** Stops enumeration and returns the number of detected devices. Call this function after *FDwfEnumStart*.

**Parameters:**

- pnDevice – Integer pointer to return count of found devices by reference.

---

**FDwfEnumDeviceType**(int idxDevice, DEVID \*pDeviceId, DEVVER \*pDeviceRevision)
 

---

**Description:** Returns the device ID and version ID.

| DEVID           | int |                                  |
|-----------------|-----|----------------------------------|
| devidEEexplorer | 1   | Electronics Explorer             |
| devidDiscovery  | 2   | Analog Discovery (1)             |
| devidDiscovery2 | 3   | Analog Discovery 2               |
| devidDDiscovery | 4   | Digital Discovery                |
| devidADP3X50    | 6   | Analog Discovery Pro 3000 series |
| devidADP5250    | 8   | Analog Discovery Pro 5250        |
| devidDPS3340    | 9   | Discovery Power Supply 3340      |

**Parameters:**

- idxDevice – Zero based index of the enumerated device for which to return the type and revision.
- pDeviceId – Variable to return the device id.
- pDeviceRevision – Pointer to DEVVER instance to return the device revision by reference.

---

**FDwfEnumDeviceIsOpened**(int idxDevice, int \*pfIsUsed)
 

---

**Description:** Retrieves a Boolean specifying if a device is already opened by this, or any other process.

**Parameters:**

- idxDevice – Index of the enumerated device.
- pfIsUsed – Pointer to variable to receive Boolean indicating if the device is in use.

---

**FDwfEnumUserName**(int idxDevice, char szUserName[32])
 

---

**Description:** Retrieves the user name of the enumerated device.

**Parameters:**

- idxDevice – Index of the enumerated device.
- szUserName – Pointer to character array to return the user name string by reference.

---

**FDwfEnumDeviceName**(int idxDevice, char szDeviceName[32])
 

---

**Description:** Retrieves the device name of the enumerated device.

**Parameters:**

- idxDevice – Index of the enumerated device.
- szDeviceName – Pointer to character array to return the device name by reference.

```
FDwfEnumSN(int idxDevice, char szSN[32])
```

**Description:** Retrieves the 12-digit, unique serial number of the enumerated device.

**Parameters:**

- idxDevice – Index of the enumerated device.
- szSN – Pointer to character array to return the serial number by reference.

```
FDwfEnumConfig(int idxDevice, int *pcConfig)
```

**Description:** Builds an internal list of detected configurations for the selected device. The function above must be called before using other *FDwfEnumConfigInfo* function because this obtains information about configurations from this list identified by the configuration index.

**Parameters:**

- idxDevice – Index of the enumerated device.
- pcConfig – Integer pointer to return count of found configurations by reference.

```
FDwfEnumConfigInfo(int idxConfig, DwfEnumConfigInfo info, int *pValue)
```

**Description:** Returns information about the configuration. These are intended for preliminary information before opening a device. Further information are available with various the FDwf#Info functions.

| DwfEnumConfigInfo          | int |
|----------------------------|-----|
| DECIAnalogInChannelCount   | 1   |
| DECIAnalogOutChannelCount  | 2   |
| DECIAnalogIOChannelCount   | 3   |
| DECIDigitalInChannelCount  | 4   |
| DECIDigitalOutChannelCount | 5   |
| DECIDigitalIOChannelCount  | 6   |
| DECIAnalogInBufferSize     | 7   |
| DECIAnalogOutBufferSize    | 8   |
| DECIDigitalInBufferSize    | 9   |
| DECIDigitalOutBufferSize   | 10  |

**Parameters:**

- idxConfig – Index of the configuration for which to return the information.
- info – Information type.
- pValue – Integer pointer to return selected information type by reference.

## 4 Device Control

See Device\_Synchronization.py example.

```
FDwfDeviceOpen(int idxDevice, HDWF *phdwf)
```

**Description:** Opens a device identified by the enumeration index and retrieves a handle. To automatically enumerate all connected devices and open the first discovered device, use index -1.

**Parameters:**

- idxDevice – Zero based index of the enumerated device.
- phdwf – Pointer to *HDWF* variable to receive opened interface handle by reference.

```
FDwfDeviceConfigOpen(int idxDevice, int idxCfg, HDWF *phdwf)
```

**Description:** Opens a device identified by the enumeration index with the selected configuration and retrieves a handle.

**Parameters:**

- idxDevice – Index of the enumerated device.
- idxCfg – Index of the device configuration.
- phdwf – Pointer to *HDWF* variable to receive opened interface handle by reference.

```
FDwfDeviceOpenEx(const char *szOpt, HDWF *phdwf)
```

**Description:** Opens a device and retrieves a handle.

**Parameters:**

- szOpt – Options delimited by new line, like “index:0\nconfig:1”.
- phdwf – Pointer to *HDWF* variable to receive opened interface handle by reference.

| options                   |                                                        |
|---------------------------|--------------------------------------------------------|
| index:#                   | Connect to device by enumeration 0 based index         |
| sn:#####                  | Open by serial number*                                 |
| name:device-name          | Open by device name*                                   |
| config:#                  | User configuration 0 based index                       |
| ip:###.###/host           | Connect to network device identified by IP or hostname |
| ip:user:pass@###.###/host | Connect to network device                              |
| user: username            | Provide username for IP/Host                           |
| pass:password             | Provide password for IP/Host                           |
| secure:#                  | Enable TLS communication encryption 0/1                |
| *                         | The devices will be enumerated before connection       |

---

**FDwfDeviceClose**(HDWF hdwf)

---

**Description:** Closes an interface handle when access to the device is no longer needed. Once the function above has returned, the specified interface handle can no longer be used to access the device.

**Parameters:**

- hdwf – Interface handle to be closed.

---

**FDwfDeviceCloseAll**()

---

**Description:** Closes all opened devices by the calling process. It does not close all devices across all processes.

**Parameters:** None.

---

**FDwfDeviceEnableSet**(HDWF hdwf, int fEnable)

---

**Description:** Enables (default) or disables the device outputs.

**Parameters:**

- hdwf – Interface handle to be closed.
- fEnable – Set TRUE to enable, FALSE to disable.

---

**FDwfDeviceAutoConfigureSet**(HDWF hdwf, int fAutoConfigure)

---

**Description:** Enables or disables the AutoConfig setting for a specific device. When this setting is enabled, the device is automatically configured every time an instrument parameter is set. For example, when AutoConfigure is enabled, FDwfAnalogOutConfigure does *not* need to be called after FDwfAnalogOutRunSet. This adds latency to every Set function; just as much latency as calling the corresponding Configure function directly afterward. With value 3 the analog-out configuration will be applied dynamically, without stopping the instrument.

**Parameters:**

- hdwf – Interface handle.
- fAutoConfigure – Value for this option: 0 disable, 1 enable, 3 dynamic

---

**FDwfDeviceAutoConfigureGet**(HDWF hdwf, int \*pfAutoConfigure)

---

**Description:** Returns the AutoConfig setting in the device. See the function description for *FDwfDeviceAutoConfigureSet* for details on this setting.

**Parameters:**

- hdwf – Interface handle.
- pfAutoConfigure – Pointer to variable to receive the current value of this option.

**FDwfDeviceReset**(HDWF hdwf)

**Description:** Resets and configures (by default, having auto configure enabled) all device and instrument parameters to default values.

**Parameters:**

- hdwf – Interface handle.

**FDwfDeviceParamSet**(HDWF hdwf, [DwfParam](#) param, [int](#) value)

**Description:** Configures various parameters for the respective device.

**Parameters:**

- param – Pointer to buffer to receive version string.
- value – Value to set

**FDwfDeviceParamGet**(HDWF hdwf, [DwfParam](#) param, [int](#) \*pValue)

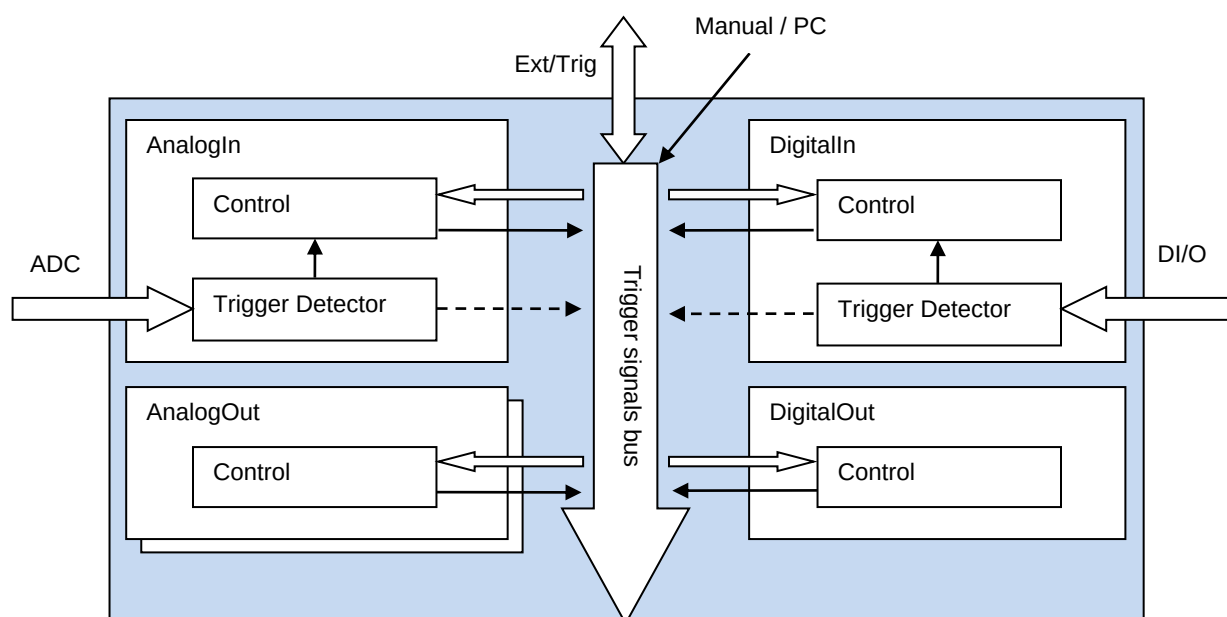
**Description:** Retrieves the parameter value.

**Parameters:**

- param – Parameter.
- pValue – Pointer to value.

The global trigger bus allows multiple instruments to trigger each other. These trigger source options are:

| TRIGSRC                  | BYTE | Trigger Source Function                                                                                                                   |
|--------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------|
| trigsrcNone              | 0    | For trigger IO it configures to be input, in high impedance.<br>This is the default setting.<br>For instruments it configures no trigger. |
| trigsrcPC                | 1    | Trigger from PC, this can be used to synchronously start multiple instruments.                                                            |
| trigsrcDetectorAnalogIn  | 2    | Trigger detector on analog in channels.                                                                                                   |
| trigsrcDetectorDigitalIn | 3    | Trigger on digital input channels.                                                                                                        |
| trigsrcAnalogIn          | 4    | Trigger on device instruments, these output high when running.                                                                            |
| trigsrcDigitalIn         | 5    | Trigger on device instruments, these output high when running.                                                                            |
| trigsrcDigitalOut        | 6    | Trigger on device instruments, these output high when running.                                                                            |
| trigsrcAnalogOut1        | 7    | Trigger on device instruments, these output high when running.                                                                            |
| trigsrcAnalogOut2        | 8    | Trigger on device instruments, these output high when running.                                                                            |
| trigsrcAnalogOut3        | 9    | Trigger on device instruments, these output high when running.                                                                            |
| trigsrcAnalogOut4        | 10   | Trigger on device instruments, these output high when running.                                                                            |
| trigsrcExternal1         | 11   | External trigger signal.                                                                                                                  |
| trigsrcExternal2         | 12   | External trigger signal.                                                                                                                  |
| trigsrcExternal3         | 13   | External trigger signal.                                                                                                                  |
| trigsrcExternal4         | 14   | External trigger signal.                                                                                                                  |
| trigsrcHigh              | 15   | High level output for trigger IO.                                                                                                         |
| trigsrcLow               | 16   | Low level output for trigger IO.                                                                                                          |
| trigsrcClock             | 17   | Reference clock output for trigger IO.                                                                                                    |





---

**FDwfDeviceTriggerInfo**(HDWF hdwf, int \*pfstrigsrc)

**Description:** Returns the supported trigger source options for the global trigger bus. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the TRIGSRC constants in dwf.h.

**Parameters:**

- hdwf – Interface handle.
- pfstrigsrc – Variable to receive the supported trigger sources.

---

**FDwfDeviceTriggerSet**(HDWF hdwf, int idxPin, TRIGSRC trigsrc)

**Description:** Configures the trigger I/O pin with a specific TRIGSRC option.

**Parameters:**

- hdwf – Interface handle.
- idxPin – External trigger, I/O pin index.
- trigsrc – Trigger source to set.

---

**FDwfDeviceTriggerGet**(HDWF hdwf, int idxPin, TRIGSRC \*ptrigsrc)

**Description:** Returns the configured trigger setting for a trigger I/O pin. The trigger source can be “none”, an internal instrument, or an external trigger.

**Parameters:**

- hdwf – Interface handle.
- idxPin – External trigger, I/O pin index.
- ptrigsrc – Variable to receive the current trigger source.

---

**FDwfDeviceTriggerPC**(HDWF hdwf)

**Description:** Generates one pulse on the PC trigger line.

**Parameters:**

- hdwf – Interface handle.

---

**FDwfDeviceTriggerSlopeInfo**(HDWF hdwf, int \*pfsslope)

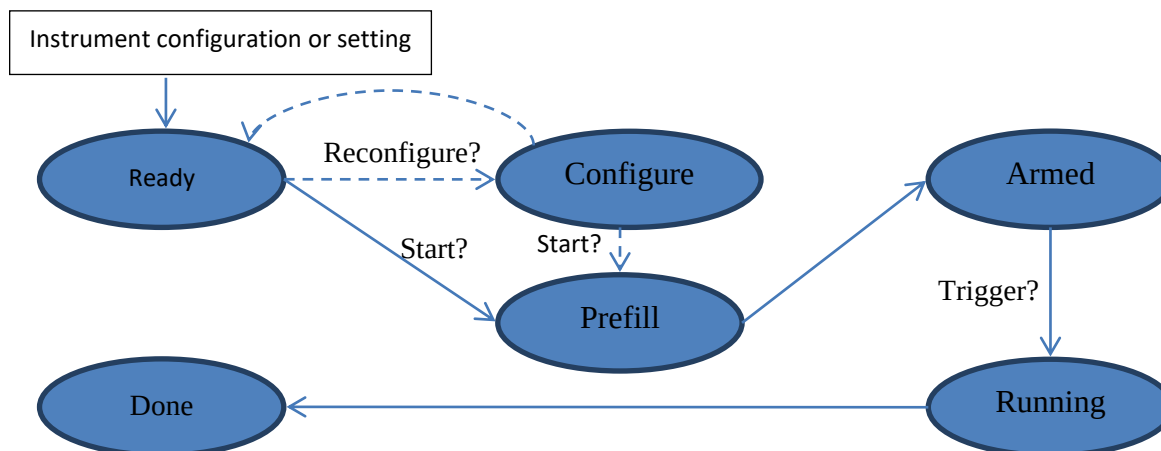
**Description:** Returns the supported trigger slopes: rising, falling and either edge.

**Parameters:**

- hdwf – Interface handle.
- pfsslope – Variable to receive the supported trigger slopes.

## 5 Analog In (Oscilloscope)

The Analog In instrument states:



The states are defined in dwf.h DwfState type.

- **Ready:** Initial state. After *FDwfAnalogInConfigure* or any *FDwfAnalogIn\*Set* function call goes to this state. With *FDwfAnalogInConfigure*, reconfigure goes to Configure state.
- **Configure:** The needed configurations are performed, and auto trigger is reset.
- **Prefill:** Prefills the buffer with samples needed before trigger.
- **Armed:** Waits for the trigger.
- **Running (Trig'd):**
  - o Single acquisition mode: remains in this state to acquire samples after trigger according trigger position parameter.
  - o Scan screen and shift modes: remains in this state until configure or any set function of this instrument.
  - o Record mode: the time period according to the record length parameter.
- **Done:** Final state.

See the following examples: `AnalogIn_Sample/Acquisition/Trigger/Record.py` `AnalogOutIn.py`

## 5.1 Control

### 5.1 Control

**FDwfAnalogInReset**(HDWF hdwf)

**Description:** Resets all AnalogIn instrument parameters to default values. If auto configure is enabled (through **FDwfDeviceAutoConfigureSet**), the instrument is also configured.

**Parameters:**

- hdwf – Interface handle.

**FDwfAnalogInConfigure**(HDWF hdwf, int fReconfigure, int fStart)

**Description:** Configures the instrument and start or stop the acquisition. To reset the Auto trigger timeout, set fReconfigure to TRUE.

**Parameters:**

- hdwf – Interface handle.
- fReconfigure – Configure the device.
- fStart – Start the acquisition.

**FDwfAnalogInStatus**(HDWF hdwf, int fReadData, DwfState \*psts)

**Description:** Checks the state of the acquisition. To read the data from the device, set fReadData to TRUE. For single acquisition mode, the data will be read only when the acquisition is finished.

**Parameters:**

- hdwf – Interface handle.
- fReadData – TRUE if data should be read.
- psts – Variable to receive the acquisition state.

**Note:** To ensure consistency between device status and measured data, the following AnalogInStatus\* functions do not communicate with the device. These functions only return information and data from the last **FDwfAnalogInStatus** call.

**FDwfAnalogInStatusSamplesLeft**(HDWF hdwf, int \*pcSamplesLeft)

**Description:** Retrieves the number of samples left in the acquisition.

**Parameters:**

- hdwf – Interface handle.
- pcSamplesLeft – Variable to receive the remaining samples to acquire.

```
FDwfAnalogInStatusSamplesValid(HDWF hdwf, int *pcSamplesValid)
```

**Description:** Retrieves the number of valid/acquired data samples.

**Parameters:**

- hdwf – Interface handle.
- pcSamplesValid – Variable to receive the number of valid samples.

```
FDwfAnalogInStatusIndexWrite(HDWF hdwf, int *pidxWrite)
```

**Description:** Retrieves the buffer write pointer which is needed in ScanScreen acquisition mode to display the scan bar.

**Parameters:**

- hdwf – Interface handle.
- pidxWrite – Variable to receive the position of the acquisition.

```
FDwfAnalogInStatusAutoTriggered(HDWF hdwf, int *pfAuto)
```

**Description:** Verifies if the acquisition is auto triggered.

**Parameters:**

- hdwf – Interface handle.
- pfAuto – Returns TRUE if the acquisition was auto triggered.

```
FDwfAnalogInStatusData(  
HDWF hdwf, int idxChannel, double *rgdVoltData, int cdData)
```

**Description:** Retrieves the acquired data samples from the specified idxChannel on the AnalogIn instrument. It copies the data samples to the provided buffer.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- rgdVoltData – Pointer to allocated buffer to copy the acquisition data.
- cdData – Number of samples to copy.

**FDwfAnalogInStatusData2 (**

HDWF hdwf, **int** idxChannel, **double** \*rgdVoltData, **int** idxData, **int** cdData)

**Description:** Retrieves the acquired data samples from the specified idxChannel on the AnalogIn instrument. It copies the data samples to the provided buffer.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- rgdVoltData – Pointer to allocated buffer to copy the acquisition data.
- idxData – First sample index to copy.
- cdData – Number of samples to copy.

**FDwfAnalogInStatusData16 (**

HDWF hdwf, **int** idxChannel, **short**\*rgs16Data, **int** idxData, **int** cdData)

**Description:** Retrieves the acquired raw data samples from the specified idxChannel on the AnalogIn instrument. It copies the data samples to the provided buffer.

**Example:** To convert raw data voltage value:

rgs16Data[i] \* FDwfAnalogInChannelRangeGet / 65536+FDwfAnalogInChannelOffsetGet

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- rgs16VoltData – Pointer to allocated buffer to copy the acquisition data.
- idxData – Source sample index to copy.
- cdData – Number of samples to copy.

**FDwfAnalogInStatusNoise (**

HDWF hdwf, **int** idxChannel, **double** \*rgdMin, **double** \*rgdMax, **int** cdData)

**Description:** Retrieves the acquired noise samples from the specified idxChannel on the AnalogIn instrument. It copies the data samples to the provided buffer.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- rgdMin – Pointer to allocated buffer to copy the minimum noise data.
- rgdMax – Pointer to allocated buffer to copy the maximum noise data.
- cdData – Number of min/max samples to copy.

```
FDwfAnalogInStatusNoise2(
HDWF hdwf, int idxChannel, double *rgdMin, double *rgdMax,
int idxData, int cdData)
```

**Description:** Retrieves the acquired noise samples from the specified idxChannel on the AnalogIn instrument. It copies the data samples to the provided buffer.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- rgdMin – Pointer to allocated buffer to copy the minimum noise data.
- rgdMax – Pointer to allocated buffer to copy the maximum noise data.
- idxData – First sample index to copy.
- cdData – Number of min/max samples to copy.

```
FDwfAnalogInStatusSample(HDWF hdwf, int idxChannel, double *pdVoltSample)
```

**Description:** Gets the last ADC conversion sample from the specified idxChannel on the AnalogIn instrument.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pdVoltSample – Variable to receive the sample value.

```
FDwfAnalogInStatusRecord(
HDWF hdwf, int *pcdDataAvailable, int *pcdDataLost, int *pcdDataCorrupt)
```

**Description:** Retrieves information about the recording process. The data loss occurs when the device acquisition is faster than the read process to PC. In this case, the device recording buffer is filled and data samples are overwritten. Corrupt samples indicate that the samples have been overwritten by the acquisition process during the previous read. In this case, try optimizing the loop process for faster execution or reduce the acquisition frequency or record length to be less than or equal to the device buffer size (record length <= buffer size/frequency).

**Parameters:**

- hdwf – Interface handle.
- pcdDataAvailable – Pointer to variable to receive the available number of samples.
- pcdDataLost – Pointer to variable to receive the lost samples after the last check.
- pcdDataCorrupt – Pointer to variable to receive the number of samples that could be corrupt.

```
FDwfAnalogInStatusTime(HDWF hdwf,  
unsigned int *psecUtc, unsigned int *ptick, unsigned int *pticksPerSecond)
```

**Description:** Retrieves instrument trigger time information.

With Digital Discovery, ADP3000, EclipseZ7, AD3.. returns a high precision device count value. This is not accurately synchronized between devices but it can be used to measure trigger distances at system frequency like 100 or 125MHz, at 10 or 8ns resolution.

With other devices returns the host time.

**Parameters:**

- hdwf – Interface handle.
- psecUtc –Seconds elapsed since Epoch 1970-01-01T00:00:00Z in local time zone.
- pTick – Additional ticks since the second count, like 0 to 999,999,999 or 124,999,999
- pticksPerSecond – Number of ticks in second like 100M or 125M.

```
FDwfAnalogInRecordLengthSet(HDWF hdwf, double sLegth)
```

**Description:** Sets the Record length in seconds. With length of zero, the record will run indefinitely.

**Parameters:**

- hdwf – Interface handle.
- sLegth – Record length to set expressed in seconds.

```
FDwfAnalogInRecordLengthGet(HDWF hdwf, double *psLegth)
```

**Description:** Gets the current Record length in seconds.

**Parameters:**

- hdwf – Interface handle.
- sLegth – Pointer to variable to receive the record length.

## 5.2 Configuration

**FDwfAnalogInFrequencyInfo** (HDWF hdwf, double \*phzMin, double \*phzMax)

**Description:** Retrieves the minimum and maximum (ADC frequency) settable sample frequency.

**Parameters:**

- hdwf – Interface handle.
- phzMin – Pointer to return the minimum allowed frequency.
- phzMax – Pointer to return the maximum allowed frequency.

**FDwfAnalogInFrequencySet** (HDWF hdwf, double hzFrequency)

**Description:** Sets the sample frequency for the instrument.

**Parameters:**

- hdwf – Interface handle.
- hzFrequency – Acquisition frequency to set.

**FDwfAnalogInFrequencyGet** (HDWF hdwf, double \*phzFrequency)

**Description:** Reads the configured sample frequency. The AnalogIn ADC always runs at maximum frequency, but the method in which the samples are stored in the buffer can be individually configured for each channel with **FDwfAnalogInChannelFilterSet** function.

**Parameters:**

- hdwf – Interface handle.
- phzFrequency – Variable to receive the acquisition frequency.

**FDwfAnalogInBitsInfo** (HDWF hdwf, int \*pnBits)

**Description:** Retrieves the number bits used by the AnalogIn ADC.

**Parameters:**

- hdwf – Interface handle.
- pnBits – Variable to receive the number of ADC bits.



```
FDwfAnalogInBufferSizeInfo(HDWF hdwf, int *pnSizeMin, int *pnSizeMax)
```

**Description:** Returns the minimum and maximum allowable buffer sizes for the instrument.

**Parameters:**

- hdwf – Interface handle.
- pnMin – Pointer to return the minimum buffer size.
- pnMax – Pointer to return the maximum buffer size.

```
FDwfAnalogInBufferSizeSet(HDWF hdwf, int nSize)
```

**Description:** Adjusts the AnalogIn instrument buffer size.

**Parameters:**

- hdwf – Interface handle.
- nSize – Buffer size to set.

```
FDwfAnalogInBufferSizeGet(HDWF hdwf, int *pnSize)
```

**Description:** Returns the used AnalogIn instrument buffer size.

**Parameters:**

- hdwf – Interface handle.
- pnSize – Variable to receive the current buffer size.

```
FDwfAnalogInNoiseSizeInfo(HDWF hdwf, int *pnSizeMax)
```

**Description:** Returns the maximum buffer size for the instrument.

**Parameters:**

- hdwf – Interface handle.
- pnMax – Pointer to return the maximum noise buffer size.

```
FDwfAnalogInNoiseSizeSet(HDWF hdwf, int nSize)
```

**Description:** Enables or disables (default) the capture on noise samples.

**Parameters:**

- hdwf – Interface handle.
- nSize – Zero disables and other values enable the noise buffer.

**FDwfAnalogInNoiseSizeGet**(HDWF hdwf, int \*pnSize)

**Description:** Returns the used AnalogIn instrument noise buffer size. This is automatically adjusted according to the sample buffer size. For instance, having maximum buffer size of 8192 and noise buffer size of 512, setting the sample buffer size to 4096 the noise buffer size will be 256.

**Parameters:**

- hdwf – Interface handle.
- pnSize – Variable to receive the current noise buffer size.

**FDwfAnalogInAcquisitionModeInfo**(HDWF hdwf, int \*pfsacqmode)

**Description:** Returns the supported AnalogIn acquisition modes. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the ACQMODE constants in dwf.h. The acquisition mode selects one of the following modes, ACQMODE:

| ACQMODE           | int | Constant Capabilities                                                                                                                                                                                                                                                                                                                                     |
|-------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| acqmodeSingle     | 0   | Perform a single buffer acquisition and rearm the instrument for next capture after the data is fetched to host using <code>FDwfAnalogInStatus</code> or <code>FDwfDigitalInStatus</code> function. This is the default setting.                                                                                                                          |
| acqmodeScanShift  | 1   | Perform a continuous acquisition in FIFO style. The trigger setting is ignored. The last sample is at the end of buffer. The <code>FDwfAnalog DigitalInStatusSamplesValid</code> function is used to show the number of the acquired samples, which will grow until reaching the BufferSize. Then the waveform “picture” is shifted for every new sample. |
| acqmodeScanScreen | 2   | Perform continuous acquisition circularly writing samples into the buffer. The trigger setting is ignored. The IndexWrite shows the buffer write position. This is similar to a heart monitor display.                                                                                                                                                    |
| acqmodeRecord     | 3   | Perform record acquisition by streaming data to host.                                                                                                                                                                                                                                                                                                     |
| acqmodeOvers      | 4   | With ADP3000 device series the consecutive captures are shifted by in four phases relative to the other instruments (AnalogOut, DigitalIn/Out) Otherwise it is identical to acqmodeSingle.                                                                                                                                                                |
| acqmodeSingle1    | 5   | Perform a single buffer acquisition without rearming the instrument.                                                                                                                                                                                                                                                                                      |

**Parameters:**

- hdwf – Interface handle.
- pfsacqmode – Pointer to return the supported acquisition modes.

**FDwfAnalogInAcquisitionModeSet**(HDWF hdwf, ACQMODE acqmode)

**Description:** Sets the acquisition mode.

**Parameters:**

- hdwf – Interface handle.
- acqmode – Acquisition mode to set.

**FDwfAnalogInAcquisitionModeGet**(HDWF hdwf, [ACQMODE](#) \*pacqmode)

---

**Description:** Retrieves the acquisition mode.

**Parameters:**

- hdwf – Interface handle.
- pacqmode – Variable to receive the current acquisition mode.

**FDwfAnalogInSamplingSourceSet**(HDWF hdwf, TRIGSRC trigsrc)

---

**Description:** Configures the AnalogIn acquisition data sampling source.

**Parameters:**

- hdwf – Interface handle.
- trigsrc – Trigger source to be used as data sampling clock.

**FDwfAnalogInSamplingSourceGet**(HDWF hdwf, TRIGSRC \*ptrigsrc)

---

**Description:** Returns the configured sampling source.

**Parameters:**

- hdwf – Interface handle.
- ptrigsrc – Variable to receive the current sampling source.

**FDwfAnalogInSamplingSlopeSet**(HDWF hdwf, DwfTriggerSlope slope)

---

**Description:** Sets the sampling slope for the instrument.

**Parameters:**

- hdwf – Interface handle.
- slope – Sampling clock slope to set.

**FDwfAnalogInSamplingSlopeGet**(HDWF hdwf, DwfTriggerSlope \*pslope)

---

**Description:** Returns the sampling for the instrument.

**Parameters:**

- hdwf – Interface handle.
- pslope – Variable to receive the current sampling slope.

---

**FDwfAnalogInSamplingDelaySet** (HDWF hdwf, double sec)

---

**Description:** Sets the sampling delay for the instrument.**Parameters:**

- hdwf – Interface handle.
- hzFrequency – Sampling delay to set.

---

**FDwfAnalogInSamplingDelayGet** (HDWF hdwf, double \*psec)

---

**Description:** Returns the configured sampling delay.**Parameters:**

- hdwf – Interface handle.
- phzFrequency – Variable to receive the sampling delay.

---

**FDwfAnalogInBuffersInfo** (HDWF hdwf, int \*pMax)

---

**Description:** Returns the maximum supported device buffers (memory segmentation) for acqmodeSingle, Oversample for low latency sequential captures.**Parameters:**

- hdwf – Interface handle.
- pnMax – Pointer to return the maximum buffers.

---

**FDwfAnalogInBuffersSet** (HDWF hdwf, int n)

---

**Description:** Sets the desired number of buffers.**Parameters:**

- hdwf – Interface handle.
- n – number of device buffers, 0 and 1 disables the device buffering, -1 sets maximum

---

**FDwfAnalogInBuffersGet** (HDWF hdwf, int \*pn)

---

**Description:** Returns the number of device buffers.**Parameters:**

- hdwf – Interface handle.
- pn – Variable to receive the current number of device buffers.

---

**FDwfAnalogInBuffersStatus** (HDWF hdwf, int \*pn)

---

**Description:** Returns the number of filled device buffers.**Parameters:**

- hdwf – Interface handle.
- pn – Variable to receive the current number of filled device buffers.

## 5.3 Channels

The oscilloscope channel settings are identical across all channels.

```
FDwfAnalogInChannelCount(HDWF hdwf, int *pcChannel)
```

**Description:** Reads the number of AnalogIn channels of the device.

**Parameters:**

- hdwf – Interface handle.
- pcChannel – Variable to receive the number of channels.

```
FDwfAnalogInChannelEnableSet(HDWF hdwf, int idxChannel, int fEnable)
```

**Description:** Enables or disables the specified AnalogIn channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Zero based index of channel to enable/disable.
- fEnable – Set TRUE to enable, FALSE to disable.

```
FDwfAnalogInChannelEnableGet(HDWF hdwf, int idxChannel, int *pfEnable)
```

**Description:** Gets the current enable/disable status of the specified AnalogIn channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Index of channel.
- pfEnable – Variable to return enable/disable status of channel.

```
FDwfAnalogInChannelFilterInfo(HDWF hdwf, int *pfsfilter)
```

**Description:** Returns the supported sampling modes. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the FILTER constants in dwf.h. When the acquisition frequency (FDwfAnalogInFrequencySet) is less than the ADC frequency (maximum acquisition frequency).

**Parameters:**

- hdwf – Interface handle.
- pfsfilter – Pointer to return the supported acquisition modes.

| FILTER                  | int | Constant Capabilities                                                                                                                            |
|-------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>filterDecimate</b>   | 0   | Store every Nth ADC conversion, where N = ADC frequency / acquisition frequency.                                                                 |
| <b>filterAverage</b>    | 1   | Store the average of N ADC conversions.                                                                                                          |
| <b>filterMinMax</b>     | 2   | Store interleaved, the minimum and maximum values, of 2xN conversions.                                                                           |
| <b>filterAverageFit</b> | 3   | The stored samples match the specified range instead of the device input range options. This can improve the vertical resolution of the samples. |

```
FDwfAnalogInChannelFilterSet(HDWF hdwf, int idxChannel, FILTER filter)
```

**Description:** Sets the acquisition filter for each AnalogIn channel. With channel index -1, each enabled AnalogIn channel filter will be configured to use the same, new option.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- filter – Acquisition sample filter to set.

```
FDwfAnalogInChannelFilterGet(HDWF hdwf, int idxChannel, FILTER *pfilter)
```

**Description:** Returns the configured acquisition filter.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pfilter – Variable to receive the current sample filter.

```
FDwfAnalogInChannelRangeInfo (
  HDWF hdwf, double *pvoltsMin, double *pvoltsMax, double *pnSteps)
```

**Description:** Returns the minimum and maximum range, peak to peak values, and the number of adjustable steps.

**Parameters:**

- hdwf – Interface handle.
- pvoltsMin – Variable to receive the minimum voltage range.
- pvoltsMax – Variable to receive the maximum voltage range.
- pnSteps – Variable to receive number of steps.

```
FDwfAnalogInChannelRangeSteps (
  HDWF hdwf, double rgVoltsStep[32], int *pnSteps)
```

**Description:** Reads the range of steps supported by the device. For instance: 1, 2, 5, 10, etc.

**Parameters:**

- hdwf – Interface handle.
- rgVoltsStep – Pointer to buffer to receive the range steps.
- pnSteps – Variable to receive number range steps.

```
FDwfAnalogInChannelRangeSet (HDWF hdwf, int idxChannel, double voltsRange)
```

**Description:** Configures the range for each channel. With channel index -1, each enabled Analog In channel range will be configured to the same, new value.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- voltsRange – Voltage range to set.

The actual range of the device may be different depending on the hardware design and calibration (tolerance of the components). For example, when setting 5V range the actual input range may be 5.456789V, which is returned by the `FDwfAnalogInChannelRangeGet` function.

With the `filterAverageFit` option the range is respected down to 1/100 of the hardware range, like a device with approximately 5V input range can be as low as 50mV.

```
FDwfAnalogInChannelRangeGet (HDWF hdwf, int idxChannel, double *pvoltsRange)
```

**Description:** Returns the real range value for the given channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pvoltsRange – Variable to receive the current voltage range.

```
FDwfAnalogInChannelOffsetInfo(  
HDF hdf, double *pvoltsMin, double *pvoltsMax, double *pnSteps)
```

**Description:** Returns the minimum and maximum offset levels supported, and the number of adjustable steps.

**Parameters:**

- hdf – Interface handle.
- pvoltsMin – Variable to receive the minimum offset voltage.
- pvoltsMax – Variable to receive the maximum offset voltage.
- pnSteps – Variable to receive the number offset steps.

```
FDwfAnalogInChannelOffsetSet(HDF hdf, int idxChannel, double voltOffset)
```

**Description:** Configures the offset for each channel. When channel index is specified as -1, each enabled AnalogIn channel offset will be configured to the same level.

**Parameters:**

- hdf – Interface handle.
- idxChannel – Channel index.
- voltsRange – Channel offset voltage to set.

```
FDwfAnalogInChannelOffsetGet(HDF hdf, int idxChannel, double *pvoltOffset)
```

**Description:** Returns for each AnalogIn channel the real offset level.

**Parameters:**

- hdf – Interface handle.
- idxChannel – Channel index.
- pvoltsRange – Variable to receive the offset voltage obtained.

```
FDwfAnalogInChannelAttenuationSet(  
HDF hdf, int idxChannel, double xAttenuation)
```

**Description:** Configures the attenuation for each channel. When channel index is specified as -1, each enabled AnalogIn channel attenuation will be configured to the same level. The attenuation does not change the attenuation on the device, just informs the library about the externally applied attenuation.

**Parameters:**

- hdf – Interface handle.
- idxChannel – Channel index.
- voltsRange – Channel attenuation to set.



```
FDwfAnalogInChannelAttenuationGet(  
HDWF hdwf, int idxChannel, double *pxAttenuation)
```

**Description:** Returns for each AnalogIn channel the configured attenuation.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pxAttenuation – Variable to receive the attenuation value.

```
FDwfAnalogInChannelBandwidthSet(  
HDWF hdwf, int idxChannel, double hz)
```

**Description:** Configures the bandwidth limitation for each channel.

Supported with ADP3000 and ADP5000 series devices, default maximum.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- hz – Channel bandwidth to set.

```
FDwfAnalogInChannelBandwidthGet(  
HDWF hdwf, int idxChannel, double *pHz)
```

**Description:** Returns for each AnalogIn channel the configured bandwidth.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pHz – Variable to receive the bandwidth value.

```
FDwfAnalogInChannelImpedanceSet(  
HDWF hdwf, int idxChannel, double ohms)
```

**Description:** Configures the bandwidth limitation for each channel.

Supported with ADP5470 and ADP5490 devices, default maximum.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- hz – Channel impedance to set.

```
FDwfAnalogInChannelImpedanceGet (  
HDWF hdwf, int idxChannel, double *pOhms)
```

**Description:** Returns for each AnalogIn channel the configured bandwidth.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pHz – Variable to receive the impedance value.

```
FDwfAnalogInChannelCouplingInfo (  
HDWF hdwf, int *pfscoupling)
```

**Description:** Returns the supported coupling options.

**Parameters:**

- hdwf – Interface handle.
- pfscoupling – Variable to receive the supported coupling modes.

```
FDwfAnalogInChannelCouplingSet (  
HDWF hdwf, int idxChannel, DwfAnalogCoupling coupling)
```

**Description:** Configures the coupling for each channel. When channel index is specified as -1, each enabled AnalogIn channel coupling will be configured to the same level.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- coupling – Channel coupling to set, DwfAnalogCoupling:
  - 0 DwfAnalogCouplingDC
  - 1 DwfAnalogCouplingAC

```
FDwfAnalogInChannelCouplingGet (  
HDWF hdwf, int idxChannel, DwfAnalogCoupling *pcoupling)
```

**Description:** Returns for each AnalogIn channel the real offset level.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pcoupling – Variable to receive the coupling.

**FDwfAnalogInChannelFfirInfo**(HDWF hdwf, int idxChannel, int \*cFIR, int \*cIIR)

**Description:** Returns the supported FIR and IIR options.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- cFIR – Variable to receive the maximum support FIR tap count.
- cIIR – Variable to receive the maximum support IIR count.

**FDwfAnalogInChannelFfirSet** (HDWF hdwf, int idxChannel, DwfFfirInput input, DwfFfirMode mode, DwfFfirType type, int ord, double hz1, double hz2, double ep)

**Description:** Configures the filter channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- input – Set filter input.
- mode – Set filter mode.
- type – Set filter type.
- ord – IIR order or FIR tap count
- hz1/2 – filter cutoff / system frequency or sample rate ratio, 0.01 - 0.5
- ep – epsilon for IIR Chebyshev filter

| DwfFfirInput    | int |                                                                                           |
|-----------------|-----|-------------------------------------------------------------------------------------------|
| DwfFfirRaw      | 0   | Filter is performed on raw ADC conversions at system frequency.                           |
| DwfFfirDecimate | 1   | Filter is performed on every Nth conversion set by the FDwfAnalogInFrequencySet function. |
| DwfFfirAverage  | 2   | Filter is perform on every Nth average sample.                                            |

| DwfFfirMode           | int |                                            |
|-----------------------|-----|--------------------------------------------|
| DwfFfirWindow         | 0   | Specified window or custom window is used. |
| DwfFfirFir            | 1   | FIR filtering                              |
| DwfFfirIirButterworth | 2   | IIR Butterworth filter                     |
| DwfFfirIirChebyshev   | 3   | IIR Chebyshev filter                       |

| DwfFfirType     | int |  |
|-----------------|-----|--|
| DwfFfirLowPass  | 0   |  |
| DwfFfirHighPass | 1   |  |
| DwfFfirBandPass | 2   |  |
| DwfFfirBandStop | 3   |  |

```
FDwfAnalogInChannelWindowSet(  
HDWF hdwf, int idxChannel, DwfWindow win, int size, int beta)
```

**Description:** Configures the filter window.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- win – Set window type.
- beta – for Keiser window.

```
FDwfAnalogInChannelCustomWindowSet(  
HDWF hdwf, int idxChannel, const double *rg, int size, int normalize)
```

**Description:** Configures the custom filter window.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- rg – Custom window array.
- size – Custom window size.
- normalize – Use 1 to normalize the window.

## 5.4 Trigger

The trigger is used for Single and Record acquisitions. For ScanScreen and ScanShift, the trigger is ignored.

To achieve the classical trigger types:

- **None:** Set *FDwfAnalogInTriggerSourceSet* to *trigsrcNone*.
- **Auto:** Set *FDwfAnalogInTriggerSourceSet* to something other than *trigsrcNone*, such as *trigsrcDetectorAnalogIn* and *FDwfAnalogInTriggerAutoTimeoutSet* to other than zero.
- **Normal:** Set *FDwfAnalogInTriggerSourceSet* to something other than *trigsrcNone*, such as *trigsrcDetectorAnalogIn* or *FDwfAnalogInTriggerAutoTimeoutSet* to zero.

See the description of **FDwfDeviceTriggerInfo**.

---

**FDwfAnalogInTriggerSourceSet** (HDWF hdwf, TRIGSRC trigsrc)

---

**Description:** Configures the AnalogIn acquisition trigger source.

**Parameters:**

- hdwf – Interface handle.
- trigsrc – Trigger source to set.

---

**FDwfAnalogInTriggerSourceGet** (HDWF hdwf, TRIGSRC \*ptrigsrc)

---

**Description:** Returns the configured trigger source. The trigger source can be “none” or an internal instrument or external trigger. To use the trigger on AnalogIn channels (edge, pulse, etc.), use *trigsrcDetectorAnalogIn*.

**Parameters:**

- hdwf – Interface handle.
- ptrigsrc – Variable to receive the current trigger source.

---

**FDwfAnalogInTriggerForce** (HDWF hdwf)

---

**Description:** Force trigger of AnalogIn instrument.

**Parameters:**

- hdwf – Interface handle.

**FDwfAnalogInTriggerPositionInfo (****HDWF hdwf, double \*psecMin, double \*psecMax, double \*pnSteps)****Description:** Returns the minimum and maximum values of the trigger position in seconds.

For Single/Repeated acquisition mode the horizontal trigger position is used is relative to the buffer middle point.

For Record mode the position is relative to the start of the capture.

**Parameters:**

- hdwf – Interface handle.
- psecMin – Variable to receive the minimum trigger position.
- psecMax – Variable to receive the maximum trigger position.
- pnSteps – Variable to return the number of steps.

**FDwfAnalogInTriggerPositionSet (HDWF hdwf, double secPosition)****Description:** Configures the horizontal trigger position in seconds.**Parameters:**

- hdwf – Interface handle.
- secPosition – Trigger position to set.

**FDwfAnalogInTriggerPositionGet (HDWF hdwf, double \*psecPosition)****Description:** Returns the configured trigger position in seconds.**Parameters:**

- hdwf – Interface handle.
- psecPosition – Variable to receive the current trigger position.

**FDwfAnalogInTriggerPositionStatus (HDWF hdwf, double \*psecPosition)****Description:** Returns the trigger position in seconds.

For triggerer source on analog input channels the value is corrected by the interpolation of triggering samples.

**Parameters:**

- hdwf – Interface handle.
- psecPosition – Variable to receive the trigger position.

```
FDwfAnalogInTriggerAutoTimeoutInfo (  
HDWF hdwf, double *psecMin, double *psecMax, double *pnSteps)
```

**Description:** Returns the minimum and maximum auto trigger timeout values, and the number of adjustable steps. The acquisition is auto triggered when the specified time elapses. With zero value the timeout is disabled, performing “Normal” acquisitions.

**Parameters:**

- hdwf – Interface handle.
- psecMin – Variable to receive the minimum timeout.
- psecMax – Variable to receive the maximum timeout.
- pnSteps – Variable to return the number of steps.

```
FDwfAnalogInTriggerAutoTimeoutSet (HDWF hdwf, double secTimeout)
```

**Description:** Configures the auto trigger timeout value in seconds.

**Parameters:**

- hdwf – Interface handle.
- secTimeout – Timeout to set.

```
FDwfAnalogInTriggerAutoTimeoutGet (HDWF hdwf, double *psecTimeout)
```

**Description:** Returns the configured auto trigger timeout value in seconds.

**Parameters:**

- hdwf – Interface handle.
- psecTimeout – Variable to receive the current timeout.

```
FDwfAnalogInTriggerHoldOffInfo (  
HDWF hdwf, double *psecMin, double *psecMax, double *pnStep)
```

**Description:** Returns the supported range of the trigger Hold-Off time in Seconds. The trigger hold-off is an adjustable period of time during which the acquisition will not trigger. This feature is used when you are triggering on burst waveform shapes, so the oscilloscope triggers only on the first eligible trigger point.

**Parameters:**

- hdwf – Interface handle.
- psecMin – Variable to receive the minimum hold off value.
- psecMax – Variable to receive the maximum hold off value.

---

```
FDwfAnalogInTriggerHoldOffSet(HDWF hdwf, double secHoldOff)
```

---

**Description:** Sets the trigger hold-off for the AnalogIn instrument in Seconds.

**Parameters:**

- hdwf – Interface handle.
- secHoldOff – Holdoff to set.

---

```
FDwfAnalogInTriggerHoldOffGet(HDWF hdwf, double *psecHoldOff)
```

---

**Description:** Gets the current trigger hold-off for the AnalogIn instrument in Seconds.

**Parameters:**

- hdwf – Interface handle.
- psecHoldOff – Variable to receive the current holdoff value.



See AnalogIn\_Counter.py example.

---

**FDwfAnalogInCounterInfo**(HDWF hdwf, double \*pcntMax, double \*psecMax)

---

**Description:** Returns the supported maximum count and timeout values.

**Parameters:**

- hdwf – Interface handle.
- pcntMax – Variable to receive the maximum count value.
- psecMax – Variable to receive the maximum timeout value.

---

**FDwfAnalogInCounterSet**(HDWF hdwf, double sec)

---

**Description:** Sets the timeout value, frequency measurement refresh rate.

**Parameters:**

- hdwf – Interface handle.
- sec – Timeout value in seconds.

---

**FDwfAnalogInCounterGet**(HDWF hdwf, double \*psec)

---

**Description:** Gets the current timeout value.

**Parameters:**

- hdwf – Interface handle.
- psec – Variable to receive the current timeout value.

---

**FDwfAnalogInCounterStatus**(HDWF hdwf,  
double \*pcnt, double \*pfreq, double \*ptick)

---

**Description:** Returns the count, frequency and tick values.

**Parameters:**

- hdwf – Interface handle.
- pcnt – Variable to receive the count value.
- pfreq – Variable to receive the frequency value in hertz.
- ptick – Variable to receive the tick value. This changes on each new measurement.

## 5.5 Trigger Detector

The following functions configure the trigger detector on analog in channels. To use this, set trigger source with `FDwfAnalogInTriggerSourceSet` to `trigsrcDetectorAnalogIn`.

See the `AnalogIn_Trigger.py` example.

---

```
FDwfAnalogInTriggerTypeInfo(HDWF hdwf, int *pfstrigtype)
```

---

**Description:** Returns the supported trigger type options for the instrument. They are returned (by reference) as a bit field. This bit field can be parsed using the `IsBitSet` Macro. Individual bits are defined using the `TRIGTYPE` constants in `dwf.h`. These trigger type options are:

- `trigtypeEdge`: trigger on rising or falling edge. This is the default setting.
- `trigtypePulse`: trigger on positive or negative; less, timeout, or more pulse lengths.
- `trigtypeTransition`: trigger on rising or falling; less, timeout, or more transition times.
- `trigtypeWindow`: trigger on exiting or entering level +/-hysteresis window.

**Parameters:**

- `hdwf` – Interface handle.
- `pfstrigtype` – Variable to receive the supported trigger types.

---

```
FDwfAnalogInTriggerTypeSet(HDWF hdwf, TRIGTYPE trigtype)
```

---

**Description:** Sets the trigger type for the instrument.

**Parameters:**

- `hdwf` – Interface handle.
- `trigtype` – Trigger type to set.

---

```
FDwfAnalogInTriggerTypeGet(HDWF hdwf, TRIGTYPE *ptrigtype)
```

---

**Description:** Gets the current trigger type for the instrument.

**Parameters:**

- `hdwf` – Interface handle.
- `ptrigtype` – Variable to receive the current trigger type.

---

```
FDwfAnalogInTriggerChannelInfo(HDWF hdwf, int *pidxmin, int *pidxmax)
```

---

**Description:** Returns the range of channels that can be triggered on.

**Parameters:**

- `hdwf` – Interface handle.
- `pidxmin` – Variable to receive the minimum channel index.
- `pidxmax` – Variable to receive the maximum channel index.

```
FDwfAnalogInTriggerChannelSet(HDWF hdwf, int idxChannel)
```

**Description:** Sets the trigger channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Trigger channel index to set.

```
FDwfAnalogInTriggerChannelGet(HDWF hdwf, int *pidxChannel)
```

**Description:** Retrieves the current trigger channel index.

**Parameters:**

- hdwf – Interface handle.
- pidxChannel – Variable to receive the current trigger channel index.

```
FDwfAnalogInTriggerFilterInfo(HDWF hdwf, int *pfsfilter)
```

**Description:** Returns the supported trigger filters. They are returned (by reference) as a bit field which can be parsed using the IsBitSet Macro. Individual bits are defined using the FILTER constants in DWF.h. Select trigger detector sample source, FILTER:

- filterDecimate: Looks for trigger in each ADC conversion, can detect glitches.
- filterAverage: Looks for trigger only in average of N samples, given by FDwfAnalogInFrequencySet.

**Parameters:**

- hdwf – Interface handle.
- pfsFilter – Variable to receive the supported trigger filters.

```
FDwfAnalogInTriggerFilterSet(HDWF hdwf, FILTER filter)
```

**Description:** Sets the trigger filter.

**Parameters:**

- hdwf – Interface handle.
- filter – Trigger filter to set.

---

**FDwfAnalogInTriggerFilterGet**(HDWF hdwf, FILTER \*pfilter)

---

**Description:** Gets the trigger filter.**Parameters:**

- hdwf – Interface handle.
- pfilter – Variable to receive the current trigger filter.

---

**FDwfAnalogInTriggerConditionInfo**(HDWF hdwf, int \*pfstrigcond)

---

**Description:** Returns the supported trigger type options for the instrument. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the DwfTriggerSlope constants in dwf.h. These trigger condition options are:

- DwfTriggerSlopeRise (This is the default setting):
  - o For edge and transition trigger on rising edge.
  - o For pulse trigger on positive pulse; For window exiting.
- DwfTriggerSlopeFall:
  - o For edge and transition trigger on falling edge.
  - o For pulse trigger on negative pulse; For window entering.
- DwfTriggerSlopeEither:
  - o For edge and transition trigger on either edge.
  - o For pulse trigger on either positive or negative pulse.

**Parameters:**

- hdwf – Interface handle.
- pfstrigcond – Variable to receive the supported trigger conditions.

---

**FDwfAnalogInTriggerConditionSet**(HDWF hdwf, DwfTriggerSlope trigcond)

---

**Description:** Sets the trigger condition for the instrument.**Parameters:**

- hdwf – Interface handle.
- trigcond – Trigger condition to set.

---

**FDwfAnalogInTriggerConditionGet**(HDWF hdwf, DwfTriggerSlope \*ptrigcond)

---

**Description:** Sets the trigger condition for the instrument.**Parameters:**

- hdwf – Interface handle.
- ptrigcond – Variable to receive the current trigger condition.

```
FDwfAnalogInTriggerLevelInfo(  
HDWF hdwf, double *pvoltsMin, double *pvoltsMax, double *pnSteps)
```

**Description:** Retrieves the range of valid trigger voltage levels for the AnalogIn instrument in Volts.

**Parameters:**

- hdwf – Interface handle.
- pvoltsMin – Variable to receive the minimum voltage level.
- pvoltsMax – Variable to receive the maximum voltage level.
- pnSteps – Variable to receive the number of voltage level steps.

```
FDwfAnalogInTriggerLevelSet(HDWF hdwf, double voltsLevel)
```

**Description:** Sets the trigger voltage level in Volts.

**Parameters:**

- hdwf – Interface handle.
- voltsLevel – Trigger voltage level to set.

```
FDwfAnalogInTriggerLevelGet(HDWF hdwf, double *pvoltsLevel)
```

**Description:** Gets the current trigger voltage level in Volts.

**Parameters:**

- hdwf – Interface handle.
- pvoltsLevel – Variable to receive the current trigger voltage level.

```
FDwfAnalogInTriggerHysteresisInfo(  
HDWF hdwf, double *pvoltsMin, double *pvoltsMax, double *pnSteps)
```

**Description:** Retrieves the range of valid trigger hysteresis voltage levels for the AnalogIn instrument in Volts. The trigger detector uses two levels: low level (TriggerLevel - Hysteresis) and high level (TriggerLevel + Hysteresis). Trigger hysteresis can be used to filter noise for Edge or Pulse trigger. The low and high levels are used in transition time triggering.

**Parameters:**

- hdwf – Interface handle.
- pvoltsMin – Variable to receive the minimum hysteresis level.
- pvoltsMax – Variable to receive the maximum hysteresis level.
- pnSteps – Variable to receive the number of hysteresis level steps.

---

**FDwfAnalogInTriggerHysteresisSet**(HDWF hdwf, double voltsLevel)

---

**Description:** Sets the trigger hysteresis level in Volts.**Parameters:**

- hdwf – Interface handle.
- voltsLevel – Trigger hysteresis level to set.

---

**FDwfAnalogInTriggerHysteresisGet**(HDWF hdwf, double \*pvoltsHysteresis)

---

**Description:** Gets the current trigger hysteresis level in Volts.**Parameters:**

- hdwf – Interface handle.
- pvoltsLevel – Variable to receive the current trigger hysteresis level.

---

**FDwfAnalogInTriggerLengthConditionInfo**(HDWF hdwf, int \*pfstriglen)

---

**Description:** Returns the supported trigger length condition options for the AnalogIn instrument. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the TRIGLEN constants in DWF.h. These trigger length condition options are:

- triglenLess: Trigger immediately when a shorter pulse or transition time is detected.
- triglenTimeout: Trigger immediately as the pulse length or transition time is reached.
- triglenMore: Trigger when the length/time is reached, and pulse or transition has ended.

**Parameters:**

- hdwf – Interface handle.
- pfsstriglen – Variable to receive the supported trigger length conditions.

---

**FDwfAnalogInTriggerLengthConditionSet**(HDWF hdwf, TRIGLEN triglen)

---

**Description:** Sets the trigger length condition for the AnalogIn instrument.**Parameters:**

- hdwf – Interface handle.
- triglen – Trigger length condition to set.

---

**FDwfAnalogInTriggerLengthConditionGet**(HDWF hdwf, TRIGLEN \*ptriglen)

---

**Description:** Gets the current trigger length condition for the AnalogIn instrument.**Parameters:**

- hdwf – Interface handle.
- ptriglen – Variable to receive the current trigger length condition.

```
FDwfAnalogInTriggerLengthInfo(  
HDWF hdwf, double *psecMin, double *psecMax, double *pnStep)
```

**Description:** Returns the supported range of trigger length for the instrument in Seconds. The trigger length specifies the minimal or maximal pulse length or transition time.

**Parameters:**

- hdwf – Interface handle.
- psecMin – Variable to receive the minimum trigger length.
- psecMax – Variable to receive the maximum trigger length.

```
FDwfAnalogInTriggerLengthSet(HDWF hdwf, double secLength)
```

**Description:** Sets the trigger length in Seconds.

**Parameters:**

- hdwf – Interface handle.
- secLength – Trigger length to set.

```
FDwfAnalogInTriggerLengthGet(HDWF hdwf, double *psecLength)
```

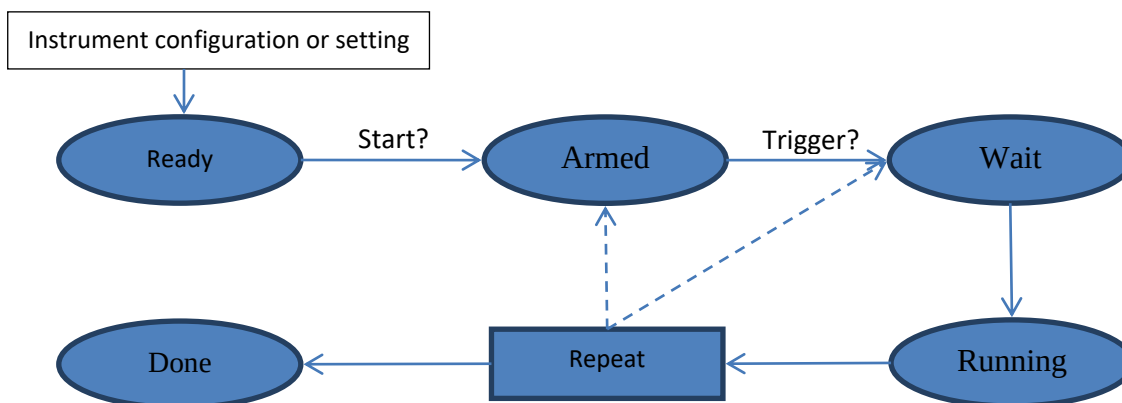
**Description:** Gets the current trigger length in Seconds.

**Parameters:**

- hdwf – Interface handle.
- secLength – Variable to receive the current trigger length.

## 6 Analog Out (Arbitrary Waveform Generator)

The Analog Out instrument states:

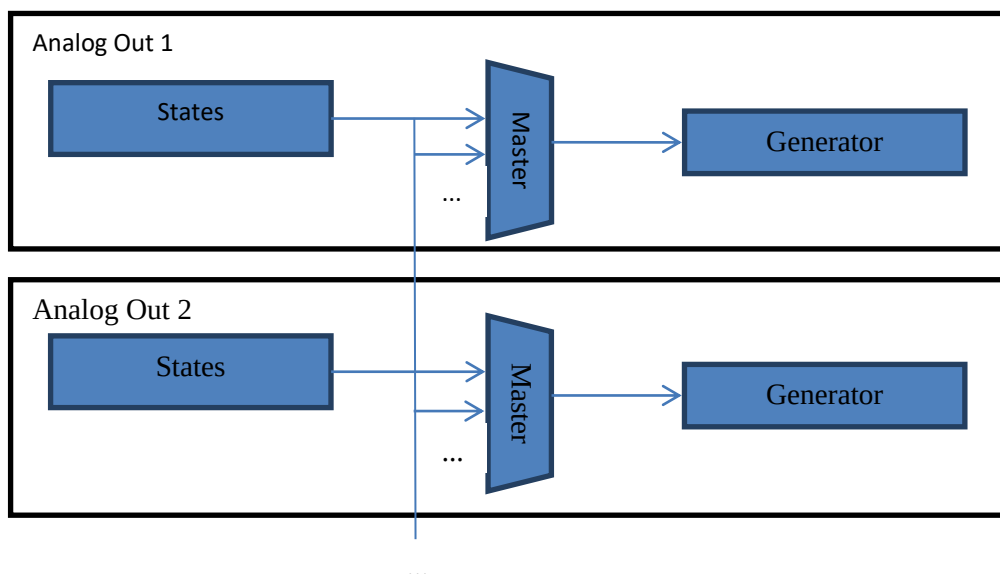


The states are defined in dwf.h DwfState type.

- **Ready:** Initial state. After *FDwfAnalogOutConfigure* or any *FDwfAnalogOut\*Set* function call goes to this state. With digital out, configure start command goes to Armed state.
- **Armed:** It waits for trigger.
- **Wait:** Remains in this state for the time period specified by *FDwfAnalogOutWaitSet* function.
- **Running:** Remains in this state for the time period specified by *FDwfAnalogOutRunSet* function.
- **Repeat:** Goes to Armed or Wait state according to the *FDwfAnalogOutRepeatTriggerSet* setting for the number of times specified by *FDwfAnalogOutRepeatSet*.
- **Done:** Final state.

The analog out channels can run independently or synchronized using the master parameter. The states are defined by trigger, wait, run, and repeat options. It is enough to start with *FDwfAnalogOutConfigure* (the master channel) the slave channels will also start.

See the following examples: `AnalogOut_Custom/Pattern/Play/Sine/Sweep/Sync.py` `AnalogOutIn.py`





## 6.1 Control

```
FDwfAnalogOutReset(HDWF hdwf, int idxChannel)
```

**Description:** Resets and configures (by default, having auto configure enabled) all AnalogOut instrument parameters to default values for the specified channel. To reset instrument parameters across all channels, set idxChannel to -1.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.

```
FDwfAnalogOutConfigure(HDWF hdwf, int idxChannel, int fStart)
```

**Description:** Starts or stops the instrument. Value 3 will apply the configuration dynamically without changing the state of the instrument. With channel index -1, each enabled Analog Out channel will be configured.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- fStart – Start the instrument: 0 stop, 1 start, 3 apply.

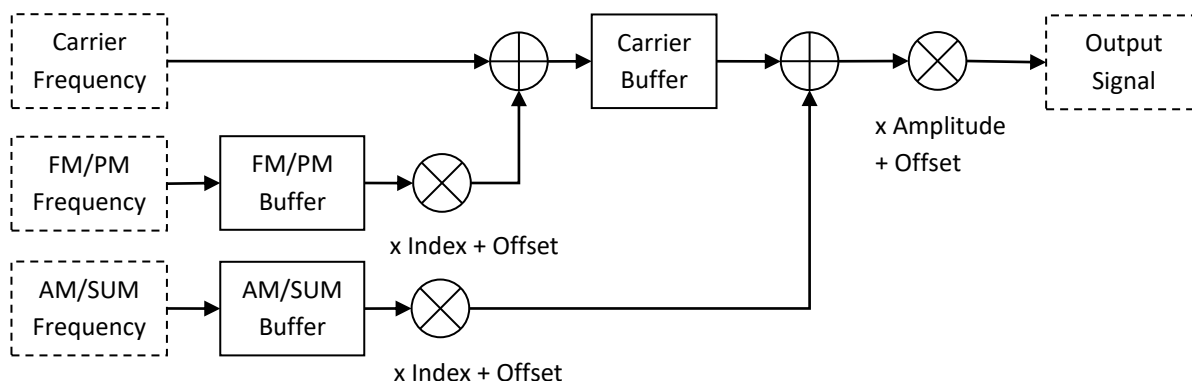
```
FDwfAnalogOutStatus(HDWF hdwf, int idxChannel, DwfState *psts)
```

**Description:** Checks the state of the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- psts – Pointer to variable to return the state.

The device output signal is output from carrier buffer but can also be frequency and/or amplitude modulated.



The Carrier/FM/AM nodes can be configured by selecting with the AnalogOutNode constants.

| AnalogOutNode        | int |                               |
|----------------------|-----|-------------------------------|
| AnalogOutNodeCarrier | 0   | Carrier signal                |
| AnalogOutNodeFM      | 1   | Frequency or Phase Modulation |
| AnalogOutNodeAM      | 2   | Amplitude Modulation or Sum   |

```
FDwfAnalogOutNodePlayStatus(HDWF hdwf, int idxChannel, AnalogOutNode node,
int *cdDataFree, int *cdDataLost, int *cdDataCorrupted)
```

**Description:** Retrieves information about the play process. The data lost occurs when the device generator is faster than the sample send process from the PC. In this case, the device buffer gets emptied and generated samples are repeated. Corrupt samples are a warning that the buffer might have been emptied while samples were sent to the device. In this case, try optimizing the loop for faster execution; or reduce the frequency or run time to be less or equal to the device buffer size (run time  $\leq$  buffer size/frequency).

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- cdDataFree – Pointer to variable to return the available free buffer space, the number of new samples that can be sent.
- cdDataLost – Pointer to variable to return the number of lost samples.
- cdDataCorrupted – Pointer to variable to return the number of samples that could be corrupted.

**FDwfAnalogOutNodePlayData (****HDWF hdwf, int idxChannel, AnalogOutNode node, double \*rgdData, int cdData)**

**Description:** Sends new data samples for play mode. Before starting the Analog Out instrument, prefill the device buffer with the first set of samples using the AnalogOutNodeDataSet function. In the loop of sending the following samples, first call AnalogOutStatus to read the information from the device, then AnalogOutPlayStatus to find out how many new samples can be sent, then send the samples with AnalogOutPlayData.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- rgdData – Pointer to samples array to be sent to the device.
- cdData – Number of samples to send.

## 6.2 Configuration

```
FDwfAnalogOutCount(HDWF hdwf, int *pcChannel)
```

**Description:** Returns the number of Analog Out channels by the device specified by hdwf.

**Parameters:**

- hdwf – Open interface handle on a device.
- pcChannel – Pointer to variable to receive the number of channels in the instrument.

```
FDwfAnalogOutNodeInfo(HDWF hdwf, int idxChannel, int *pfsnode)
```

**Description:** Returns the supported AnalogOut nodes of the AnalogOut channel. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pfsnode – Variable to receive the supported nodes.

```
FDwfAnalogOutNodeEnableSet(  
HDWF hdwf, int idxChannel, AnalogOutNode node, int fMode)
```

**Description:** Enables or disables the channel node specified by idxChannel and node. The Carrier node enables or disables the channel or selectects the modulation. With channel index -1, each Analog Out channel enable will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- fEnable – Carrier: 0 disable, 1 enable; FM: 1 FM, 2 PM, 3 PMD; AM: 1 AM, 2 SUM, 3 SUMV

The PM, PMD, SUM and SUMV options are available with ADP3000 series.

For PM (Phase Modulation) or PMD (expressed in degrees) the limits are  $\pm 100\%$  or  $\pm 180^\circ$ .

For SUM or SUMV (expressed in Volts) the limits are  $\pm 400\%$  or in Volts the Offset  $\pm$  Amplitude ca be up to four times the Carrier Amplitude.

```
FDwfAnalogOutNodeEnableGet (
HDWF hdwf, int idxChannel, AnalogOutNode node, int *pfEnable)
```

**Description:** Verifies if a specific channel and node is enabled or disabled.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- pfEnable – Pointer to variable to receive enabled state.

```
FDwfAnalogOutNodeFunctionInfo (
HDWF hdwf, int idxChannel, AnalogOutNode node, int *pfsfunc)
```

**Description:** Returns the supported generator function options. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the FUNC constants in dwf.h. These are:

| FUNC Constants    | Value | FUNC Constant Capabilities                                                                                                                       |
|-------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| funcDC            | 0     | Generate DC value set as offset.                                                                                                                 |
| funcSine          | 1     | Generate sine waveform.                                                                                                                          |
| funcSquare        | 2     | Generate square waveform, offset +/- amplitude.                                                                                                  |
| funcTriangle      | 3     | Generate triangle waveform.                                                                                                                      |
| funcRampUp        | 4     | Generate a waveform with a ramp-up voltage at the beginning.                                                                                     |
| funcRampDown      | 5     | Generate a waveform with a ramp-down voltage at the end.                                                                                         |
| funcNoise         | 6     | Generate noise waveform from random samples.                                                                                                     |
| funcPulse         | 7     | Generate pulse waveform, offset + amplitude.                                                                                                     |
| funcTrapezium     | 8     | Generate trapezium waveform.                                                                                                                     |
| funcSinePower     | 9     | Generate sine with symmetry used as power function.                                                                                              |
| funcCustomPattern | 28    | Generate waveform from custom samples. It provides constant sample rate, supporting integer divisions of the system frequency.                   |
| funcPlayPattern   | 29    | Generate waveform in stream play style. It provides constant sample rate.                                                                        |
| funcCustom        | 30    | Generate waveform from custom samples. Optimizes for average requested frequency, sample output lengths may vary by one system frequency period. |
| funcPlay          | 31    | Generate waveform in stream play style. Optimizes for average requested frequency.                                                               |

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- pfsfunc – Variable to receive the supported generator function options.

```
FDwfAnalogOutNodeFunctionSet (  
HDWF hdwf, int idxChannel, AnalogOutNode node, FUNC func)
```

**Description:** Sets the generator output function for the specified instrument channel. With channel index -1, each enabled Analog Out channel function will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- func – Generator function option to set.

```
FDwfAnalogOutNodeFunctionGet (  
HDWF hdwf, int idxChannel, AnalogOutNode node, FUNC *pfunc)
```

**Description:** Retrieves the current generator function option for the specified instrument channel.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- ptrgsrc – Pointer to variable to receive the generator function option.

```
FDwfAnalogOutNodeFrequencyInfo (HDWF hdwf, int idxChannel, AnalogOutNode node,  
double *phzMin, double *phzMax)
```

**Description:** Returns the supported frequency range for the instrument. The maximum value shows the DAC frequency. The frequency of the generated waveform: repetition frequency for standard types and custom data; DAC update for noise type; sample rate for play type.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Zero based channel index.
- node – Zero based node index.
- phzMin – Variable to receive the supported minimum frequency.
- phzMax – Variable to receive the supported maximum frequency.

```
FDwfAnalogOutNodeFrequencySet (  
HDWF hdwf, int idxChannel, AnalogOutNode node, double hzFrequency)
```

**Description:** Sets the frequency. With channel index -1, each enabled Analog Out channel frequency will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- hzFrequency – Frequency value to set expressed in Hz.

```
FDwfAnalogOutNodeFrequencyGet (
  HDWF hdwf, int idxChannel, AnalogOutNode node, double *phzFrequency)
```

**Description:** Gets the currently set frequency for the specified channel-node on the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- hzFrequency – Pointer to variable to receive frequency value in Hz.

```
FDwfAnalogOutNodeAmplitudeInfo (
  HDWF hdwf, int idxChannel, AnalogOutNode node, double *pvMin, double *pvMax)
```

**Description:** Retrieves the amplitude range for the specified channel-node on the instrument. The amplitude is expressed in Volt units for carrier and in percentage units (modulation index) for AM/FM.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- pvMin – Minimum amplitude level or modulation index.
- pvMax – Maximal amplitude level or modulation index.

```
FDwfAnalogOutNodeAmplitudeSet (
  HDWF hdwf, int idxChannel, AnalogOutNode node, double vAmplitude)
```

**Description:** Sets the amplitude or modulation index for the specified channel-node on the instrument. With channel index -1, each enabled Analog Out channel amplitude (or modulation index) will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- vAmplitude – Amplitude of channel in Volts or modulation index in percentage.

```
FDwfAnalogOutNodeAmplitudeGet (
  HDWF hdwf, int idxChannel, AnalogOutNode node, double *pvAmplitude)
```

**Description:** Gets the currently set amplitude or modulation index for the specified channel-node on the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- pvAmplitude – Pointer to variable to receive amplitude value in Volts or modulation index in percentage.

**FDwfAnalogOutNodeOffsetInfo (****HDWF hdwf, int idxChannel, AnalogOutNode node, double \*pvMin, double \*pvMax)**

**Description:** Retrieves available the offset range. For carrier node in units of volts, and in percentage units for AM/FM nodes.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- pvMin – Minimum offset voltage or modulation offset percentage.
- pvMax – Maximum offset voltage or modulation offset percentage.

**FDwfAnalogOutNodeOffsetSet (****HDWF hdwf, int idxChannel, AnalogOutNode node, double vOffset)**

**Description:** Sets the offset value for the specified channel-node on the instrument. With channel index -1, each enabled Analog Out channel offset will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- vOffset – Value to set voltage offset in Volts or modulation offset percentage.

**FDwfAnalogOutNodeOffsetGet (HDWF hdwf, int idxChannel, AnalogOutNode node, double \*pvOffset)**

**Description:** Gets the current offset value for the specified channel-node on the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- pvOffset – Pointer to variable to receive offset value in Volts or modulation offset percentage.



**FDwfAnalogOutNodeSymmetryInfo**(HDWF hdwf, `int` idxChannel, AnalogOutNode node, `double` \*ppercentageMin, `double` \*ppercentageMax)

**Description:** Obtains the symmetry (or duty cycle) range (0 ... 100). This symmetry is supported for standard signal types. It the pulse duration divided by the pulse period.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- ppercentageMin – Minimum value of Symmetry percentage.
- ppercentageMax – Maximum value of Symmetry percentage.

**FDwfAnalogOutNodeSymmetrySet**(HDWF hdwf, `int` idxChannel, AnalogOutNode node, `double` percentageSymmetry)

**Description:** Sets the symmetry (or duty cycle) for the specified channel-node on the instrument. With channel index -1, each enabled Analog Out channel symmetry will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- percentageSymmetry – Value of percentage of Symmetry (duty cycle).

**FDwfAnalogOutNodeSymmetryGet** (  
HDWF hdwf, `int` idxChannel, AnalogOutNode node, `double` \*ppercentageSymmetry)

**Description:** Gets the currently set symmetry (or duty cycle) for the specified channel-node of the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- ppercentageSymmetry – Pointer to variable to receive value of Symmetry (duty cycle).

**FDwfAnalogOutNodePhaseInfo**(HDWF hdwf, `int` idxChannel, AnalogOutNode node, `double` \*pdegreeMin, `double` \*pdegreeMax)

**Description:** Retrieves the phase range (in degrees 0 ... 360) for the specified channel-node of the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- pdegreeMin – Minimum value of Phase (in degrees).
- pdegreeMax – Maximum value of Phase (in degrees).

```
FDwfAnalogOutNodePhaseSet (  
HDWF hdwf, int idxChannel, AnalogOutNode node, double degreePhase)
```

**Description:** Sets the phase for the specified channel-node on the instrument. With channel index -1, each enabled Analog Out channel phase will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- degreePhase – Value of Phase in degrees.

```
FDwfAnalogOutNodePhaseGet (HDWF hdwf, int idxChannel, AnalogOutNode node,  
double *pdegreePhase)
```

**Description:** Gets the current phase for the specified channel-node on the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- pdegreePhase – Pointer to variable to receive Phase value (in degrees).

```
FDwfAnalogOutNodeDataInfo (HDWF hdwf, int idxChannel, AnalogOutNode node,  
int *pnSamplesMin, double *pnSamplesMax)
```

**Description:** Retrieves the minimum and maximum number of samples allowed for custom data generation.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- pnSamplesMin – Minimum number of samples available for custom data.
- pnSamplesMax – Maximum number of samples available for custom data.

```
FDwfAnalogOutNodeDataSet (
  HDWF hdwf, int idxChannel, AnalogOutNode node, double *rgdData, int cdData)
```

**Description:** Set the custom data or to prefill the buffer with play samples. The samples are double precision floating point values (rgdData) normalized to  $\pm 1$ .

With the custom function option, the data samples (cdData) will be interpolated to the device buffer size. The output value will be Offset + Sample\*Amplitude, for instance:

- 0 value sample will output: Offset.
- +1 value sample will output: Offset + Amplitude.
- -1 value sample will output: Offset – Amplitude.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- node – Node index.
- rgdData – Buffer of samples to set.
- cdData – Number of samples to set in rgdData.

-

```
FDwfAnalogOutLimitationInfo (
  HDWF hdwf, int idxChannel, double *pvMin, double *pvMax)
```

**Description:** Retrieves the limitation range supported by the channel. This option is supported on Electronics Explorer Analog Out Channel 3 and 4, Positive and Negative Power supplies, to set current or voltage limitation.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pvMin – Minimum limitation value.
- pvMax – Maximum offset voltage or modulation offset percentage.

```
FDwfAnalogOutLimitationSet (HDWF hdwf, int idxChannel, double limit)
```

**Description:** Sets the limitation value for the specified channel on the instrument. With channel index -1, each enabled Analog Out channel limitation will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- limit – Value to set voltage offset in Volts or modulation offset percentage.

```
FDwfAnalogOutLimitationGet(HDWF hdwf, int idxChannel, double *plimit)
```

**Description:** Gets the current limitation value for the specified channel on the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- limit – Pointer to variable to receive offset value in Volts or modulation offset percentage.

```
FDwfAnalogOutModeSet(HDWF hdwf, int idxChannel, DwfAnalogOutMode mode)
```

**Description:** Set the generator output mode for the specified instrument channel. With channel index -1, each enabled Analog Out channel mode will be configured to use the same, new option. This option is supported on Electronics Explorer Analog Out Channel 3 and 4, Positive and Negative Power supplies, to set current or voltage waveform generator mode.

The DwfAnalogOutMode options are:

- DwfAnalogOutModeVoltage
- DwfAnalogOutModeCurrent

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- mode – Generator mode option to set.

```
FDwfAnalogOutModeGet(HDWF hdwf, int idxChannel, DwfAnalogOutMode *pmode)
```

**Description:** Retrieves the current generator mode option for the specified instrument channel.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- mode – Pointer to variable to receive the generator mode option.

| DwfAnalogOutIdle               | Idle Constant Capabilities                                                                                                                                                                    |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DwfAnalogOutIdleDisable</b> | Supported on Electronics Explorer:<br>Channel 1&2 0V output<br>Channel 3&4. having 1 kΩ resistor to GND and diode                                                                             |
| <b>DwfAnalogOutIdleOffset</b>  | The idle output is the configured Offset level.                                                                                                                                               |
| <b>DwfAnalogOutIdleInitial</b> | The idle output voltage level has the initial waveform value of the current configuration. This depends on the selected signal type, Offset, Amplitude and Amplitude Modulator configuration. |
| <b>DwfAnalogOutIdleHold</b>    | Like initial but holds the last output level in Done state. This can be used to create ramp signals like settings Run length to period (Auto) and Repeat once.                                |

---

**FDwfAnalogOutIdleInfo**(HDWF hdwf, int idxChannel, int \*pfsidle)

**Description:** Returns the supported generator idle output options. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the DwfAnalogOutIdle constants in dwf.h.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pfsidle – Variable to receive the supported generator idle options.

---

**FDwfAnalogOutIdleSet**(HDWF hdwf, int idxChannel, AnalogOutNode idle)

**Description:** Sets the generator idle output for the specified instrument channel. The idle output selects the output while not running, Ready, Stopped, Done, or Wait states.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- idle – Generator function option to set.

---

**FDwfAnalogOutIdleGet**(HDWF hdwf, int idxChannel, AnalogOutNode \*pidle)

**Description:** Retrieves the generator idle output option for the specified instrument channel.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pidle – Pointer to variable to receive the generator function option.

## 6.3 States

See the description of **FDwfDeviceTriggerInfo**.

```
FDwfAnalogOutTriggerSourceSet(HDWF hdwf, int idxChannel, TRIGSRC trigsrc)
```

**Description:** Sets the trigger source for the channel on instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- trigsrc – Trigger source to set.

```
FDwfAnalogOutTriggerSourceGet(HDWF hdwf, int idxChannel, TRIGSRC *ptrigsrc)
```

**Description:** Gets the current trigger source setting for the channel on instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- ptrigsrc – Pointer to variable to receive the trigger source.

```
FDwfAnalogOutTriggerSlopeSet(  
HDWF hdwf, int idxChannel, DwfTriggerSlope slope)
```

**Description:** Sets the trigger slope for the channel on instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- slope – Trigger slope to set.

```
FDwfAnalogOutTriggerSlopeGet(  
HDWF hdwf, int idxChannel, DwfTriggerSlope *pslope)
```

**Description:** Gets the current trigger slope setting for the channel on instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pslope – Pointer to variable to receive the trigger slope.

```
FDwfAnalogOutRunInfo (  
HDWF hdwf, int idxChannel, double *psecMin, double *psecMax)
```

**Description:** Returns the supported run length range for the instrument in Seconds. Zero values represent an infinite (or continuous) run. Default value is zero.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- psecMin – Variable to receive the supported minimum run length.
- psecMax – Variable to receive the supported maximum run length.

```
FDwfAnalogOutRunSet (HDWF hdwf, int idxChannel, double secRun)
```

**Description:** Sets the run length for the instrument in Seconds.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- secRun – Run length to set expressed in seconds.

```
FDwfAnalogOutRunGet (HDWF hdwf, int idxChannel, double *psecRun)
```

**Description:** Reads the configured run length for the instrument in Seconds.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- psecRun – Pointer to variable to receive the run length.

```
FDwfAnalogOutRunStatus (HDWF hdwf, int idxChannel, double *psecRun)
```

**Description:** Reads the remaining run length. It returns data from the last FDwfAnalogOutStatus call.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- psecRun – Pointer to variable to receive the remaining run length.

```
FDwfAnalogOutWaitInfo (  
HDWF hdwf, int idxChannel, double *psecMin, double *psecMax)
```

**Description:** Returns the supported wait length range in Seconds. The wait length is how long the instrument waits after being triggered to generate the signal. Default value is zero.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- psecMin/Max – Variable to receive the supported minimum/maximum wait length.

```
FDwfAnalogOutWaitSet (HDWF hdwf, int idxChannel, double secWait)
```

**Description:** Sets the wait length for the channel on instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- secWait – Wait length to set expressed in seconds.

```
FDwfAnalogOutWaitGet (HDWF hdwf, int idxChannel, double *psecWait)
```

**Description:** Gets the current wait length for the channel on instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- psecWait – Pointer to variable to receive the wait length.



---

**FDwfAnalogOutRepeatInfo**(HDWF hdwf, int idxChannel, int \*pnMin, int \*pnMax)

**Description:** Returns the supported repeat count range. This is how many times the generated signal will be repeated upon. Zero value represents infinite repeat. Default value is one.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pnMin – Variable to receive the supported minimum repeat count.
- pnMax – Variable to receive the supported maximum repeat count.

---

**FDwfAnalogOutRepeatSet**(HDWF hdwf, int idxChannel, int cRepeat)

**Description:** Sets the repeat count.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- cRepeat – Repeat count to set.

---

**FDwfAnalogOutRepeatGet**(HDWF hdwf, int idxChannel, int \*pcRepeat)

**Description:** Reads the current repeat count.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pcRepeat – Pointer to variable to receive the repeat count.

---

**FDwfAnalogOutRepeatStatus**(HDWF hdwf, int idxChannel, int \*pcRepeat)

**Description:** Reads the remaining repeat counts. It only returns information from the last FDwfAnalogOutStatus function call, it does not read from the device.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pcRepeat – Pointer to variable to receive the remaining repeat counts.

```
FDwfAnalogOutRepeatTriggerSet(HDWF hdwf, int idxChannel, int fRepeatTrigger)
```

**Description:** Sets the repeat trigger option. To include the trigger in wait-run repeat cycles, set fRepeatTrigger to TRUE. It is disabled by default.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- fRepeatTrigger – Boolean used to specify if the trigger should be included in a repeat cycle.

```
FDwfAnalogOutRepeatTriggerGet(  
HDWF hdwf, int idxChannel, int *pfRepeatTrigger)
```

**Description:** Verifies if the trigger has been included in wait-run repeat cycles.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pfRepeatTrigger – Pointer to variable to receive the repeat trigger option.

```
FDwfAnalogOutMasterSet(HDWF hdwf, int idxChannel, int idxMaster)
```

**Description:** Sets the state machine master of the channel generator. With channel index -1, each enabled Analog Out channel will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- idxMaster – Node index.

```
FDwfAnalogOutMasterGet(HDWF hdwf, int idxChannel, int *pidxMaster)
```

**Description:** Verifies the parameter set by FDwfAnalogOutMasterSet.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pidxMaster – Pointer to variable to receive parameter.

## 7 Analog I/O

The AnalogIO functions are used to control the power supplies, reference voltage supplies, voltmeters, ammeters, thermometers, and any other sensors on the device. These are organized into channels which contain a number of nodes. For instance, a power supply channel might have three nodes: an enable setting, a voltage level setting/reading, and current limitation setting/reading.

See the AnalogIO.py example.

---

**FDwfAnalogIOReset** (HDWF hdwf)

---

**Description:** Resets and configures (by default, having auto configure enabled) all AnalogIO instrument parameters to default values.

**Parameters:**

- hdwf – Open interface handle on a device.

---

**FDwfAnalogIOConfigure** (HDWF hdwf)

---

**Description:** Configures the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.

---

**FDwfAnalogIOStatus** (HDWF hdwf)

---

**Description:** Reads the status of the device and stores it internally. The following status functions will return the information that was read from the device when the function above was called.

**Parameters:**

- hdwf – Open interface handle on a device.

---

**FDwfAnalogIOEnableInfo** (HDWF hdwf, int \*pfSet, int \*pfStatus)

---

**Description:** Verifies if Master Enable Setting and/or Master Enable Status are supported for the AnalogIO instrument. The Master Enable setting is essentially a software switch that “enables” or “turns on” the AnalogIO channels. If supported, the status of this Master Enable switch (Enabled/Disabled) can be queried by calling `FDwfAnalogIOEnableStatus`.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfSet – Returns true when the master enable setting is supported.
- pfStatus – Return true when the status of the master enable can be read.

```
FDwfAnalogIOEnableSet(HDWF hdwf, int fMasterEnable)
```

**Description:** Sets the master enable switch.

**Parameters:**

- hdwf – Open interface handle on a device.
- fMasterEnable – Set TRUE to enable the master switch; FALSE to disable the master switch.

```
FDwfAnalogIOEnableGet(HDWF hdwf, int *pfMasterEnable)
```

**Description:** Returns the current state of the master enable switch. This is not obtained from the device.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfMasterEnable – Pointer to variable to return the enabled configuration.

```
FDwfAnalogIOEnableStatus(HDWF hdwf, int *pfMasterEnable)
```

**Description:** Returns the master enable status (if the device supports it). This can be a switch on the board or an overcurrent protection circuit state.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfMasterEnabled – Pointer to variable to return the active status.

```
FDwfAnalogIOChannelCount(HDWF hdwf, int *pnChannel)
```

**Description:** Returns the number of AnalogIO channels available on the device.

**Parameters:**

- hdwf – Open interface handle on a device.
- pnChannel – Pointer to variable to return the number of channels.

```
FDwfAnalogIOChannelName(  
HDWF hdwf, int idxChannel, char szName[32], char szLabel[16])
```

**Description:** Returns the name (long text) and label (short text, printed on the device) for a channel.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- szName – Pointer to character array to return the user name.
- szLabel – Pointer to character array to return the label.

```
FDwfAnalogIOChannelInfo(HDWF hdwf, int idxChannel, int *pnNodes)
```

**Description:** Returns the number of nodes associated with the specified channel.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pnNodes – Pointer to variable to return the number of node .

```
FDwfAnalogIOChannelNodeName (  
HDWF hdwf, int idxChannel, int idxNode,  
char szNodeName[32], char szUnits[16])
```

**Description:** Returns the node name (“Voltage”, “Current”...) and units (“V”, “A”) for an Analog I/O node .

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- idxNode – Node index.
- szNodeName – Pointer to character array to return the node name.
- szUnits – Pointer to character array to return the value units.

```
FDwfAnalogIOChannelNodeInfo (  
HDWF hdwf, int idxChannel, int idxNode, ANALOGIO *panalogio)
```

**Description:** Returns the supported channel nodes. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the *ANALOGIO* constants in dwf.h. The acquisition mode selects one of the following modes, *ANALOGIO*:

| <i>ANALOGIO</i> Modes      | <i>ANALOGIO</i> Mode Functions                                                |
|----------------------------|-------------------------------------------------------------------------------|
| <b>analogioEnable</b>      | Enable I/O node; used to enable a power supply, reference voltage, etc.       |
| <b>analogioVoltage</b>     | Voltage I/O node; used to input/output voltage levels.                        |
| <b>analogioCurrent</b>     | Current I/O node; used to input/output current levels.                        |
| <b>analogioTemperature</b> | Temperature I/O node; used to retrieve read values from a temperature sensor. |

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- idxNode – Node index.
- panalogio – Pointer to variable to return the node type.

```
FDwfAnalogIOChannelNodeSetInfo(HDWF hdwf, int idxChannel, int idxNode,  
double *pmin, double *pmax, int *pnSteps)
```

**Description:** Returns node value limits. Since a Node can represent many things (Power supply, Temperature sensor, etc.), the *Minimum*, *Maximum*, and *Steps* parameters also represent different types of values. In broad terms, the (Maximum – Minimum)/Steps = the number of specific input/output values.

**FDwfAnalogIOChannelNodeInfo** returns the type of values to expect and

**FDwfAnalogIOChannelNodeName** returns the units of these values.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Analog I/O channel index of the device.
- idxNode – Node index.
- pmin – Minimum settable value.
- pmax – Maximum settable value.
- pnSteps – Number of steps between minimum and maximum values.

```
FDwfAnalogIOChannelNodeSet(  
HDWF hdwf, int idxChannel, int idxNode, double value)
```

**Description:** Sets the node value for the specified node on the specified channel.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxNode – Node index.
- idxChannel – Analog I/O channel index of the device.
- value – Value to set.

```
FDwfAnalogIOChannelNodeGet(  
HDWF hdwf, int idxChannel, int idxNode, double *pvalue)
```

**Description:** Returns the currently set value of the node on the specified channel.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Analog I/O channel index of the device.
- idxNode – Node index.
- pvalue – Pointer to variable to return the configured value.

**FDwfAnalogIOChannelNodeStatusInfo (**

HDWF hdwf, **int** idxChannel, **int** idxNode, **double** \*pmin, **double** \*pmax, **int** \*pnSteps)

**Description:** Returns node the range of reading values available for the specified node on the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- idxNode – Node index.
- pmin – Minimum reading value.
- pmax – Maximum reading value.
- pnSteps – Number of steps between minimum and maximum values.

**FDwfAnalogIOChannelNodeStatus (**

HDWF hdwf, **int** idxChannel, **int** idxNode, **double** \*pvalue)

**Description:** Returns the value reading of the node.

**Parameters:**

- hdwf –Interface handle.
- idxChannel – Channel index.
- idxNode – Node index.
- pvalue – Pointer to variable to return the value reading.

## 8 Digital I/O

The digital IO signals are shared in the device between Digital-IO, Out and In functions. The Digital-IO has priority over Digital-Out. The Digital-Out is only applied for a signal when the Digital-IO Enable and Output are zero for the respective bit.

See the DigitalIO.py example.

---

**FDwfDigitalIOReset** (HDWF hdwf)

---

**Description:** Resets and configures (by default, having auto configure enabled) all DigitalIO instrument parameters to default values. It sets the output enables to zero (tri-state), output value to zero, and configures the DigitalIO instrument.

**Parameters:**

- hdwf – Open interface handle on a device.

---

**FDwfDigitalIOConfigure** (HDWF hdwf)

---

**Description:** Configures the DigitalIO instrument. This doesn't have to be used if AutoConfiguration is enabled.

**Parameters:**

- hdwf – Open interface handle on a device.

---

**FDwfDigitalIOStatus** (HDWF hdwf)

---

**Description:** Reads the status and input values, of the device DigitalIO to the PC. The status and values are accessed from the `FDwfDigitalIOInputStatus` function.

**Parameters:**

- hdwf – Open interface handle on a device.

The functions with 32 bit arguments are the following.

---

**FDwfDigitalIOOutputEnableInfo** (HDWF hdwf,  
 unsigned long long \*pfsOutputEnableMask)

---

**Description:** Returns the output enable mask (bit set) that can be used on this device. These are the pins that can be used as outputs on the device.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsOutputEnableMask – Variable to return the OE mask bit field.



---

**FDwfDigitalIOOutputEnableSet**(HDWF hdwf, unsigned int fsOutputEnable)

**Description:** Enables specific pins for output. This is done by setting bits in the fsOutEnable bit field (1 for enabled, 0 for disabled).

**Parameters:**

- hdwf – Open interface handle on a device.
- fsOutputEnable – Output enable bit set.

---

**FDwfDigitalIOOutputEnableGet**(HDWF hdwf, unsigned int \*pfsOutputEnable)

**Description:** Returns a bit field that specifies which output pins have been enabled.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsOutputEnable – Pointer to variable to returns output enable bit set.

---

**FDwfDigitalIOOutputInfo**(HDWF hdwf, unsigned int \*pfsOutputMask)

**Description:** Returns the settable output value mask (bit set) that can be used on this device.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsOutputMask – Variable to return the output value mask.

---

**FDwfDigitalIOOutputSet**(HDWF hdwf, unsigned int fsOutput)

**Description:** Sets the output logic value on all output pins.

**Parameters:**

- hdwf – Open interface handle on a device.
- fsOutput – Output bit set.

---

**FDwfDigitalIOOutputGet**(HDWF hdwf, unsigned int \*pfsOutput)

**Description:** Returns the currently set output values across all output pins.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsOutput – Pointer to variable to returns output bit set.

---

**FDwfDigitalIOInputInfo** (HDWF hdwf, unsigned int \*pfsInputMask)
 

---

**Description:** returns the readable input value mask (bit set) that can be used on the device.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsInputMask – Variable to return the input value mask.

---

**FDwfDigitalIOInputStatus** (HDWF hdwf, unsigned int \*pfsInput)
 

---

**Description:** Returns the input states of all I/O pins. Before calling the function above, call the FDwfDigitalIOStatus function to read the Digital I/O states from the device.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsInput – Variable to return the input value.

---

**FDwfDigitalIOPullInfo** (HDWF hdwf, unsigned int \*pfsUp, unsigned int \*pfsDown)
 

---

**Description:** Returns the settable output pullup and down mask that can be used on this device.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsUp – Variable to return the pullup value mask.
- pfsDown – Variable to return the pulldown value mask.

---

**FDwfDigitalIOPullSet** (HDWF hdwf, unsigned int fsUp, unsigned int fsDown)
 

---

**Description:** Sets the output logic value on all output pins.

**Parameters:**

- hdwf – Open interface handle on a device.
- fsUp – Pullup bit set.
- fsDown – Pulldown bit set.

---

**FDwfDigitalIOPullGet** (HDWF hdwf, unsigned int \*pfsUp, unsigned int \*pfsDown)
 

---

**Description:** Returns the currently set output values across all output pins.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsUp – Pointer to variable to returns pullup bit set.
- pfsDown – Pointer to variable to returns pulldown bit set.

```
FDwfDigitalIODriveInfo(HDWF hdwf, int channel,  
double *ampMin, double *ampMax, int *ampSteps, int *slewSteps)
```

**Description:** Returns information about the settable drive current and slew that can be used on this device channels.

**Parameters:**

- hdwf – Open interface handle on a device.
- channel – Channel index, not yet used by any device.
- ampMin – Variable to return the minimum drive value in ampere.
- ampMax – Variable to return the maximum drive value in ampere.
- ampSteps – Variable to return the supported drive steps.
- slewSteps – Variable to return the supported slew steps.

```
FDwfDigitalIODriveSet(HDWF hdwf, int channel, double amp, int slew)
```

**Description:** Sets the output logic value on all output pins.

**Parameters:**

- hdwf – Open interface handle on a device.
- channel – Channel index.
- amp – Drive strength in ampere.
- slew – Slew option.

```
FDwfDigitalIODriveGet(HDWF hdwf, double *pamp, int *pslew)
```

**Description:** Returns the currently set output values across all output pins.

**Parameters:**

- hdwf – Open interface handle on a device.
- pamp – Pointer to variable to return drive.
- pslew – Pointer to variable to return slew.

The functions with 64bit arguments are the following.

**FDwfDigitalIOOutputEnableInfo64** (HDWF hdwf, unsigned long long \*pfsOutputEnableMask)

**Description:** Returns the output enable mask (bit set) that can be used on this device. These are the pins that can be used as outputs on the device.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsOutputEnableMask – Variable to return the OE mask bit field.

**FDwfDigitalIOOutputEnableSet64** (HDWF hdwf, unsigned long long fsOutputEnable)

**Description:** Enables specific pins for output. This is done by setting bits in the fsOutEnable bit field (1 for enabled, 0 for disabled).

**Parameters:**

- hdwf – Open interface handle on a device.
- fsOutputEnable – Output enable bit set.

**FDwfDigitalIOOutputEnableGet64** (HDWF hdwf, unsigned long long \*pfsOutputEnable)

**Description:** Returns a bit field that specifies which output pins have been enabled.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsOutputEnable – Pointer to variable to returns output enable bit set.

**FDwfDigitalIOOutputInfo64** (HDWF hdwf, unsigned long long \*pfsOutputMask)

**Description:** Returns the settable output value mask (bit set) that can be used on this device.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsOutputMask – Variable to return the output value mask.

**FDwfDigitalIOOutputSet64** (HDWF hdwf, unsigned long long fsOutput)

**Description:** Sets the output logic value on all output pins.

**Parameters:**

- hdwf – Open interface handle on a device.
- fsOutput – Output bit set.

**FDwfDigitalIOOutputGet64** (HDWF hdwf, unsigned long long \*pfsOutput)

**Description:** Returns the currently set output values across all output pins.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsOutput – Pointer to variable to returns output bit set.

**FDwfDigitalIOInputInfo64** (HDWF hdwf, unsigned long long \*pfsInputMask)

**Description:** returns the readable input value mask (bit set) that can be used on the device.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsInputMask – Variable to return the input value mask.

**FDwfDigitalIOInputStatus64** (HDWF hdwf, unsigned long long \*pfsInput)

**Description:** Returns the input states of all I/O pins. Before calling the function above, call the FDwfDigitalIOStatus function to read the Digital I/O states from the device.

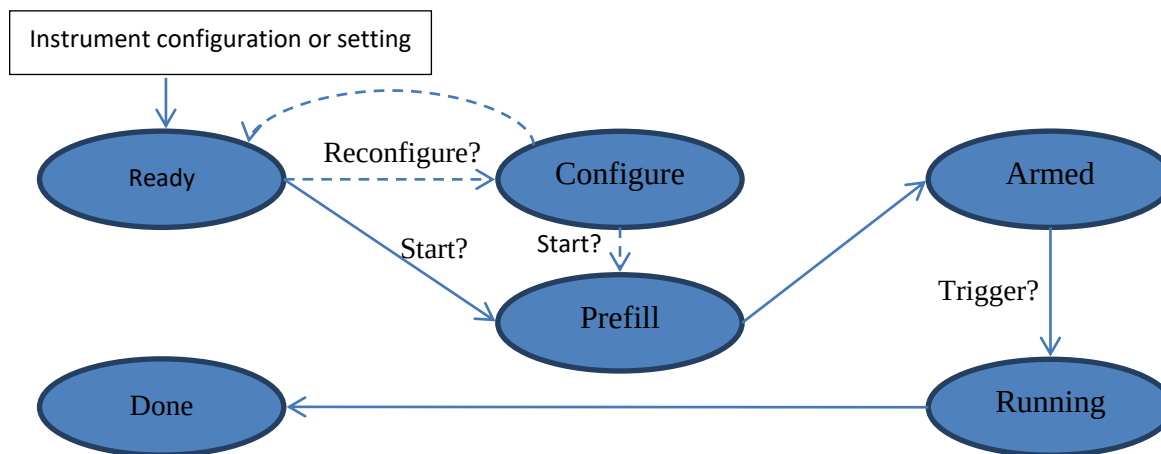
For Digital Discovery returns DIN[23-0] DIO[39-24]

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsInput – Variable to return the input value.

## 9 Digital In (Logic Analyzer)

The Digital In instrument states:



The states are defined in `dwf.h DwfState` type.

- **Ready:** Initial state. After `FDwfDigitalInConfigure` or any `FDwfDigitalIn*Set` function call goes to this state. With `FDwfDigitalInConfigure`, reconfigure goes to Configure state.
- **Configure:** The digital in auto trigger is reset.
- **Prefill:** Prefills the buffer with samples need before trigger.
- **Armed:** It waits for trigger.
- **Running:** For single acquisition mode remains in this state to acquire samples after trigger set by `FDwfDigitalInTriggerPositionSet`. Scan screen and shift modes remain until configure or any set function of this instrument.
- **Done:** Final state.

See the following examples: `DigitalIn_Acquisition/Record.py`.

## 9.1 Control

**FDwfDigitalInReset**(HDWF hdwf)

**Description:** Resets and configures (by default, having auto configure enabled) all DigitalIn instrument parameters to default values.

**Parameters:**

- hdwf – Interface handle.

**FDwfDigitalInConfigure**(HDWF hdwf, int fReconfigure, int fStart)

**Description:** Configures the instrument and start or stop the acquisition. To reset the Auto trigger timeout, set fReconfigure to TRUE.

**Parameters:**

- hdwf – Interface handle.
- fReconfigure – Configure the device.
- fStart – Start the acquisition.

**FDwfDigitalInStatus**(HDWF hdwf, int fReadData, DwfState \*psts)

**Description:** Checks the state of the instrument. To read the data from the device, set fReadData to TRUE. For single acquisition mode, the data will be read only when the acquisition is finished.

**Parameters:**

- hdwf – interface handle.
- fReadData – TRUE if data should be read.
- psts – Variable to receive the acquisition state.

**FDwfDigitalInStatusSamplesLeft**(HDWF hdwf, int \*pcSamplesLeft)

**Description:** Retrieves the number of samples left in the acquisition.

**Parameters:**

- hdwf – Interface handle.
- pcSamplesLeft – Variable to receive the remaining samples to acquire.

**FDwfDigitalInStatusSamplesValid**(HDWF hdwf, int \*pcSamplesValid)

**Description:** Retrieves the number of valid/acquired data samples.

**Parameters:**

- hdwf – Interface handle.
- pcSamplesValid – Variable to receive the number of valid samples.

---

**FDwfDigitalInStatusIndexWrite**(HDWF hdwf, int \*pidxWrite)

**Description:** Retrieves the buffer write pointer. This is needed in ScanScreen acquisition mode to display the scan bar.

**Parameters:**

- hdwf – Interface handle.
- pidxWrite – Variable to receive the position of the acquisition.

---

**FDwfDigitalInStatusAutoTriggered**(HDWF hdwf, int \*pfAuto)

**Description:** Verifies if the acquisition is auto triggered.

**Parameters:**

- hdwf – Interface handle.
- pfAuto – Returns TRUE if the acquisition was auto triggered.

---

**FDwfDigitalInStatusTime**(HDWF hdwf, unsigned int \*psecUtc, unsigned int \*ptick, unsigned int \*pticksPerSecond)

**Description:** Retrieves instrument trigger time information.

With ADP3000 returns a high precision device count value. This is not accurately synchronized between devices but it can be used to measure trigger distances at system frequency like 100 or 125MHz, at 10 or 8ns resolution.

With other devices returns the host time.

**Parameters:**

- hdwf – Interface handle.
- psecUtc –Seconds elapsed since Epoch 1970-01-01T00:00:00Z in local time zone.
- pTick – Additional ticks since the second count, like 0 to 999,999,999 or 124,999,999
- pticksPerSecond – Number of ticks in second like 100M or 125M.

---

**FDwfDigitalInStatusData**(HDWF hdwf, void \*rgData, int countOfDataBytes)

**Description:** It copies the data samples to the provided buffer. The sample format is specified by **FDwfDigitalInSampleFormatSet** function.

**Parameters:**

- hdwf – Interface handle.
- rgData – Pointer to allocated buffer to copy the acquisition data.
- countOfDataBytes – Number of bytes to copy.



```
FDwfDigitalInStatusData2(HDWF hdwf,  
void *rgData, int idxSample, int countOfDataBytes)
```

**Description:** It copies the data samples to the provided buffer. The sample format is specified by **FDwfDigitalInSampleFormatSet** function.

**Parameters:**

- hdwf – Interface handle.
- rgData – Pointer to allocated buffer to copy the acquisition data.
- idxSample – First sample index to copy.
- countOfDataBytes – Number of bytes to copy.

```
FDwfDigitalInStatusNoise2(HDWF hdwf,  
void *rgData, int idxSample, int countOfDataBytes)
```

**Description:** It copies the noise samples to the provided buffer.  
The noise capture can be enabled with **DwfDigitalInSampleModeNoise**.

**Parameters:**

- hdwf – Interface handle.
- rgData – Pointer to allocated buffer to copy the acquisition data.
- idxSample – First sample index to copy.
- countOfDataBytes – Number of bytes to copy.

```
FDwfDigitalInStatusRecord(  
HDWF hdwf, int *pcdDataAvailable, int *pcdDataLost, int *pcdDataCorrupt)
```

**Description:** Retrieves information about the recording process. The data loss occurs when the device acquisition is faster than the read process to PC. In this case, the device recording buffer is filled and data samples are overwritten. Corrupt samples indicate that the samples have been overwritten by the acquisition process during the previous read. In this case, try optimizing the loop process for faster execution or reduce the acquisition frequency or record length to be less than or equal to the device buffer size (record length <= buffer size/frequency).

**Parameters:**

- hdwf – Interface handle.
- pcdDataAvailable – Pointer to variable to receive the available number of samples.
- pcdDataLost – Pointer to variable to receive the lost samples after the last check.
- pcdDataCorrupt – Pointer to variable to receive the number of samples that could be corrupt.

## 9.2 Configuration

**FDwfDigitalInInternalClockInfo**(HDWF hdwf, double \*phzFreq)

**Description:** Retrieves the internal clock frequency.

**Parameters:**

- hdwf – Interface handle.
- phzFreq – Pointer to return the internal clock frequency.

**FDwfDigitalInClockSourceInfo**(HDWF hdwf, int \*pfsDwfDigitalInClockSource)

**Description:** Returns the supported clock sources for Digital In instrument. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the *DwfDigitalInClockSource* constants in dwf.h:

- DwfDigitalInClockSourceInternal: Internal clock.
- DwfDigitalInClockSourceExternal: External clock source.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfsDwfDigitalInClockSource – Pointer to variable to return the available clock source options.

**FDwfDigitalInClockSourceSet**(HDWF hdwf, DwfDigitalInClockSource v)

**Description:** Sets the clock source of instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- v – Clock source.

**FDwfDigitalInClockSourceGet**(HDWF hdwf, DwfDigitalInClockSource \*pv)

**Description:** Gets the clock source of instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- pv – Pointer to variable to return the configured value.

---

**FDwfDigitalInDividerInfo**(HDWF hdwf, unsigned int \*pdivMax)

---

**Description:** Returns the maximum supported clock divider value. This specifies the sample rate.**Parameters:**

- hdwf – Interface handle.
- pdivMax – Pointer to variable to return the available maximum divider value.

---

**FDwfDigitalInDividerSet**(HDWF hdwf, unsigned int div)

---

**Description:** Sets the clock divider value.**Parameters:**

- hdwf – Interface handle.
- div – Divider value.

---

**FDwfDigitalInDividerGet**(HDWF hdwf, unsigned int \*pdiv)

---

**Description:** Gets the configured clock divider value.**Parameters:**

- hdwf – Interface handle.
- pdiv – Pointer to return configured value.

---

**FDwfDigitalInBitsInfo**(HDWF hdwf, int \*pnBits)

---

**Description:** Returns the number of Digital In bits.**Parameters:**

- hdwf – Interface handle.
- pnBits – Pointer to variable to return the number of bits.

---

**FDwfDigitalInInputOrderSet**(HDWF hdwf, int fDioFirst)

---

**Description:** Configures the order of values stored in the sampling array. If fDIOFirst = true DIO24..39 are placed at the beginning of the array followed by DIN0..23. With fDIOFirst = false DIN0..23 are placed at the beginning followed by DIO24..31. Valid only for Digital Discovery device.**Parameters:**

- hdwf – Interface handle.
- fDioFirst – DIO or DIN lines start from index zero.

```
FDwfDigitalInSampleFormatSet(HDWF hdwf, int nBits)
```

**Description:** Sets the sample format, the number of bits starting from least significant bit. Valid options are 8, 16, and 32.

**Parameters:**

- hdwf – Interface handle.
- nBits – Sample format.

```
FDwfDigitalInSampleFormatGet(HDWF hdwf, int *pnBits)
```

**Description:** Returns the configured sample format.

**Parameters:**

- hdwf – Interface handle.
- pnBits – Pointer to return configured value.

```
FDwfDigitalInBufferSizeInfo(HDWF hdwf, int *pnSizeMax)
```

**Description:** Returns the Digital In maximum buffer size.

**Parameters:**

- hdwf – Interface handle.
- pnSizeMax – Pointer to variable to return maximum buffer size.

```
FDwfDigitalInBufferSizeSet(HDWF hdwf, int nSize)
```

**Description:** Set the buffer size.

**Parameters:**

- hdwf – Interface handle.
- nSize – Buffer size.

```
FDwfDigitalInBufferSizeGet(HDWF hdwf, int *pnSize)
```

**Description:** Returns the configured buffer size.

**Parameters:**

- hdwf – Interface handle.
- nSize – Pointer to return configured value.

---

**FDwfDigitalInSampleModeInfo**(HDWF hdwf, int \*pfsDwfDigitalInSampleMode)

**Description:** Returns the supported sample modes. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the DwfDigitalInSampleMode constants in dwf.h:

- DwfDigitalInSampleModeSimple: Stores one sample on every divider clock pulse.
- DwfDigitalInSampleModeNoise: Stores alternating noise and sample values, where noise is more than one transition between two samples. This could indicate glitches or ringing. It is available when sample rate is less than maximum clock frequency, divider is greater than one.

**Parameters:**

- hdwf – Interface handle.
- pfsDwfDigitalInSampleMode – Pointer to return the supported sample modes.

The function above

---

**FDwfDigitalInSampleModeSet**(HDWF hdwf, DwfDigitalInSampleMode v)

**Description:** Set the sample mode.

**Parameters:**

- hdwf – Open interface handle on a device.
- v – Sample mode.

---

**FDwfDigitalInSampleModeGet**(HDWF hdwf, DwfDigitalInSampleMode \*pv)

**Description:** Return the configured sample mode.

**Parameters:**

- hdwf – Open interface handle on a device.
- pv – Pointer to return configured value.

---

**FDwfDigitalInAcquisitionModeInfo**(HDWF hdwf, [int](#) \*pfsacqmode)

- 
- Description:** Returns the supported DigitalIn acquisition modes. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the ACQMODE constants in DWF.h. The acquisition mode selects one of the following modes, ACQMODE:

**Parameters:**

- hdwf – Interface handle.

---

**FDwfDigitalInAcquisitionModeSet**(HDWF hdwf, [ACQMODE](#) acqmode)

---

**Description:** Sets the acquisition mode.

**Parameters:**

- hdwf – Interface handle.
- acqmode – Acquisition mode to set.

---

**FDwfDigitalInAcquisitionModeGet**(HDWF hdwf, ACQMODE \*pacqmode)

---

**Description:** Retrieves the acquisition mode.

**Parameters:**

- hdwf – Interface handle.
- pacqmode – Variable to receive the current acquisition mode.

---

**FDwfDigitalInSampleSensibleSet**(HDWF hdwf, [unsigned int](#) fs)

---

**Description:** Selects the signals to be used for data compression in record acquisition mode.

**Parameters:**

- hdwf – Interface handle.
- fs – bit field set of signals to look for compression.

---

**FDwfDigitalInSampleSensibleGet**(HDWF hdwf, [unsigned int](#) \*pfs)

---

**Description:** Retrieves the signals being used for data compression in record acquisition mode.

**Parameters:**

- hdwf – Interface handle.
- pfs – Pointer to variable to receive configured value

---

**FDwfDigitalInBuffersInfo**(HDWF hdwf, int \*pMax)

---

**Description:** Returns the maximum supported device buffers (memory segmentation) for acqmodeSingle, Oversample for low latency sequential captures.

**Parameters:**

- hdwf – Interface handle.
- pnMax – Pointer to return the maximum buffers.

---

**FDwfDigitalInBuffersSet**(HDWF hdwf, int n)

---

**Description:** Sets the desired number of buffers.

**Parameters:**

- hdwf – Interface handle.
- n – number of device buffers, 0 and 1 disables the device buffering, -1 sets maximum

---

**FDwfDigitalInBuffersGet**(HDWF hdwf, int \*pn)

---

**Description:** Returns the number of device buffers.

**Parameters:**

- hdwf – Interface handle.
- pn – Variable to receive the current number of device buffers.

---

**FDwfDigitalInBuffersStatus**(HDWF hdwf, int \*pn)

---

**Description:** Returns the number of filled device buffers.

**Parameters:**

- hdwf – Interface handle.
- pn – Variable to receive the current number of filled device buffers.

## 9.3 Trigger

See the description of **FDwfDeviceTriggerInfo**.

---

```
FDwfDigitalInTriggerSourceSet(HDWF hdwf, TRIGSRC trigsrc)
```

---

**Description:** Sets the trigger source for the instrument.

**Parameters:**

- hdwf – Interface handle.
- trigsrc – Trigger source to set.

---

```
FDwfDigitalInTriggerSourceGet(HDWF hdwf, TRIGSRC *ptrigsrc)
```

---

**Description:** Gets the current trigger source setting for the instrument.

**Parameters:**

- hdwf – Interface handle.
- ptrigsrc – Pointer to variable to receive the trigger source.

---

```
FDwfDigitalInTriggerSlopeSet(HDWF hdwf, DwfTriggerSlope slope)
```

---

**Description:** Sets the trigger slope for the instrument.

**Parameters:**

- hdwf – Interface handle.
- slope – Trigger source to set.

---

```
FDwfDigitalInTriggerSlopeGet(HDWF hdwf, DwfTriggerSlope *pslope)
```

---

**Description:** Gets the current trigger source setting for the instrument.

**Parameters:**

- hdwf – Interface handle.
- pslope – Pointer to variable to receive the trigger slope.

---

```
FDwfDigitalInTriggerPositionInfo(  
HDWF hdwf, unsigned int *pnSamplesAfterTriggerMax)
```

---

**Description:** Returns maximum values of the trigger position in samples. This can be greater than the specified buffer size.

**Parameters:**

- hdwf – Interface handle.
- pnSamplesAfterTriggerMax – Variable to receive the maximum trigger position.



---

**FDwfDigitalInTriggerPositionSet**(HDWF hdwf, `unsigned int` cSamplesAfterTrigger)

---

**Description:** Sets the number of samples to acquire after trigger.**Parameters:**

- hdwf – Interface handle.
- cSamplesAfterTrigger – Samples after trigger.

---

**FDwfDigitalInTriggerPositionGet**(  
HDWF hdwf, `unsigned int` \*pcSamplesAfterTrigger)

---

**Description:** Gets the configured trigger position.**Parameters:**

- hdwf – Interface handle.
- pcSamplesAfterTrigger – Pointer to variable to receive configured value

---

**FDwfDigitalInTriggerPrefillSet**(HDWF hdwf, `unsigned int` cSamplesBeforeTrigger)

---

**Description:** Sets the number of samples to acquire before arming in Record acquisition mode. The prefill is used for record with trigger to make sure at last the required number of samples are collected before arming, before looking for trigger event.

With prefill 0 the recording process will stream data only after trigger event.

With prefill more than zero the recording will stream until trigger occurs plus the samples specified by trigger position.

**Parameters:**

- hdwf – Interface handle.
- cSamplesBeforeTrigger – Samples before trigger.

---

**FDwfDigitalInTriggerPrefillGet**(  
HDWF hdwf, `unsigned int` \*pcSamplesBeforeTrigger)

---

**Description:** Gets the configured trigger prefill.**Parameters:**

- hdwf – Interface handle.
- pcSamplesBeforeTrigger – Pointer to variable to receive configured value

```
FDwfDigitalInTriggerAutoTimeoutInfo(  
HDWF hdwf, double *psecMin, double *psecMax, double *pnSteps)
```

**Description:** Returns the minimum and maximum auto trigger timeout values, and the number of adjustable steps.

**Parameters:**

- hdwf – Interface handle.
- psecMin – Variable to receive the minimum timeout.
- psecMax – Variable to receive the maximum timeout.
- pnSteps – Variable to return the number of steps.

```
FDwfDigitalInTriggerAutoTimeoutSet(HDWF hdwf, double secTimeout)
```

**Description:** Configures the auto trigger timeout value in seconds.

**Parameters:**

- hdwf – Interface handle.
- secTimeout – Timeout to set.

```
FDwfDigitalInTriggerAutoTimeoutGet(HDWF hdwf, double *psecTimeout)
```

**Description:** Returns the configured auto trigger timeout value in seconds. The acquisition is auto triggered when the specified time elapses. With zero value the timeout is disabled, performing “Normal” acquisitions.

**Parameters:**

- hdwf – Interface handle.
- psecTimeout – Variable to receive the current timeout.

See DigitalIn\_Counter.py example.

```
FDwfDigitalInCounterInfo(HDWF hdwf, double *pcntMax, double *psecMax)
```

**Description:** Returns the supported maximum count and timeout values.

**Parameters:**

- hdwf – Interface handle.
- pcntMax – Variable to receive the maximum count value.
- psecMax – Variable to receive the maximum timeout value.

```
FDwfDigitalInCounterSet(HDWF hdwf, double sec)
```

**Description:** Sets the timeout value, frequency measurement refresh rate.

**Parameters:**

- hdwf – Interface handle.
- sec – Timeout value in seconds.

```
FDwfDigitalInCounterGet(HDWF hdwf, double *psec)
```

**Description:** Gets the current timeout value.

**Parameters:**

- hdwf – Interface handle.
- psec – Variable to receive the current timeout value.

```
FDwfDigitalInCounterStatus(HDWF hdwf,  
double *pcnt, double *pfreq, double *ptick)
```

**Description:** Returns the count, frequency and tick values.

**Parameters:**

- hdwf – Interface handle.
- pcnt – Variable to receive the count value.
- pfreq – Variable to receive the frequency value in hertz.
- ptick – Variable to receive the tick value. This changes on each new measurement.

## 9.4 Trigger Detector

In order to use trigger on digital in pins, set trigger source with `FDwfDigitalInTriggerSourceSet` to `trigsrcDetectorDigitalIn`.

```
FDwfDigitalInTriggerInfo(HDWF hdwf,
unsigned int *pfsLevelLow, unsigned int *pfsLevelHigh,
unsigned int *pfsEdgeRise, unsigned int *pfsEdgeFall)
```

**Description:** Returns the supported digital in triggers. The bits of the arguments represent pins.

**Parameters:**

- hdwf – Interface handle.
- pfsLevelLow – Variable to receive the supported low state triggers.
- pfsLevelHigh – Variable to receive the supported low state triggers.
- pfsEdgeRise – Variable to receive the supported rising edge triggers.
- pfsEdgeFall – Variable to receive the supported falling edge triggers.

```
FDwfDigitalInTriggerSet(HDWF hdwf,
unsigned int fsLevelLow, unsigned int fsLevelHigh,
unsigned int fsEdgeRise, unsigned int fsEdgeFall)
```

**Description:** Configures the digital in trigger detector.

The logic for the trigger bits is: *Low and High and (Rise or Fall)*. Setting a bit in both rise and fall will trigger on any edge, any transition. For instance `FDwfDigitalInTriggerInfo(hdwf, 1, 2, 4, 8)` will generate trigger when DIO-0 is low and DIO-1 is high and DIO-2 is rising or DIO-3 is falling.

**Parameters:**

- hdwf – Interface handle.
- fsLevelLow – Set low state condition.
- fsLevelHigh – Set high state condition.
- fsEdgeRise – Set rising edge condition.
- fsEdgeFall – Set falling edge condition.

```
FDwfDigitalInTriggerGet(HDWF hdwf,
unsigned int *pfsLevelLow, unsigned int *pfsLevelHigh,
unsigned int *pfsEdgeRise, unsigned int *pfsEdgeFall)
```

**Description:** Returns the configured digital in trigger detector option.

**Parameters:**

- hdwf – Interface handle.
- pfsLevelLow – Variable to receive the configured value.
- pfsLevelHigh – Variable to receive the configured value.
- pfsEdgeRise – Variable to receive the configured value.
- pfsEdgeFall – Variable to receive the configured value.

```
FDwfDigitalInTriggerResetSet(HDWF hdwf,  
unsigned int fsLevelLow, unsigned int fsLevelHigh,  
unsigned int fsEdgeRise, unsigned int fsEdgeFall)
```

**Description:** Configures the digital in trigger reset condition.

**Parameters:**

- hdwf – Interface handle.
- fsLevelLow – Set low state condition.
- fsLevelHigh – Set high state condition.
- fsEdgeRise – Set rising edge condition.
- fsEdgeFall – Set falling edge condition.

```
FDwfDigitalInTriggerCountSet(HDWF hdwf, int cCount, int fRestart)
```

**Description:** Configures the trigger counter.

**Parameters:**

- hdwf – Interface handle.
- cCount – Set event count.
- fRestart – Set to restart counter after expires or not and wait for next reset condition.

```
FDwfDigitalInTriggerLengthSet(HDWF hdwf,  
double secMin, double secMax, int idxSync)
```

**Description:** Configures the trigger timing. The synchronization modes are the following:

0 – Normal

1 – Timing: use for UART, CAN. The min length specifies bit length and max the timeout length.

2 – PWM: use for 1-Wire. The min length specifies sampling time and max the timeout length.

**Parameters:**

- hdwf – Interface handle.
- secMin – Set minimum length in seconds, up to 20sec.
- secMax – Set maximum length in seconds, up to 20sec.
- idxSync – Set synchronization mode.

```
FDwfDigitalInTriggerMatchSet(HDWF hdwf,
int iPin, unsigned int fsMask, unsigned int fsValue, int cBitStuffing)
```

**Description:** Configure the deserializer. The bits are left shifted. The mask and value should be specified according to this, in MSBit first order. Like to trigger on first to fourth bits received from a sequence of 8, b1010XXXX, set mask to 0x000000F0 and value to 0x000000A0.

**Parameters:**

- hdwf – Interface handle.
- iPin – Set pin to deserialize.
- fsMask – Set bit mask pattern.
- fsValue – Set bit match pattern.
- cBitStuffing – Set bit stuffing count.

To trigger on a pulse configure the following:

```
FDwfDigitalInTriggerCountSet(hdwf, 1, 0);
```

- For positive pulse specify rising and for negative falling edge reset.

```
FDwfDigitalInTriggerResetSet(hdwf, 0, 0, positive?1<<dio:0, negative?1<<dio:0);
```

- For positive pulse specify high and for negative low level trigger.

```
FDwfDigitalInTriggerSet(hdwf, negative?1<<dio:0, positive?1<<dio:0, 0, 0);
```

**Glitch/Less:** To trigger on glitch, a pulse length at most the specified value:

```
FDwfDigitalInTriggerLengthSet(hdwf, 0, max, 0); // maximum pulse length in seconds
```

**Timeout:** To trigger on pulse timeout, on a pulse after the specified minimum time expires:

```
FDwfDigitalInTriggerLengthSet(hdwf, min, 0, 0); // minimum pulse length in seconds
```

**More:** To trigger on pulse ending slope which is longer than the specified minimum time:

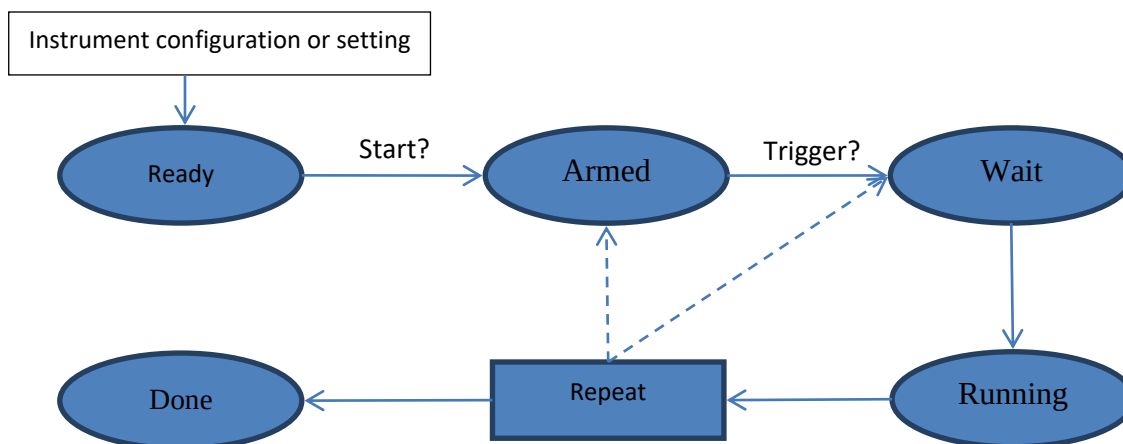
```
FDwfDigitalInTriggerLengthSet(hdwf, min, -1, 0); // minimum pulse length in seconds
```

**Length:** To trigger on a pulse length with the specified minimum and maximum lengths use:

```
FDwfDigitalInTriggerLengthSet(hdwf, min, max, 0); // min/max pulse length in seconds
```

## 10 Digital Out (Pattern Generator)

The DigitalOut instrument states:



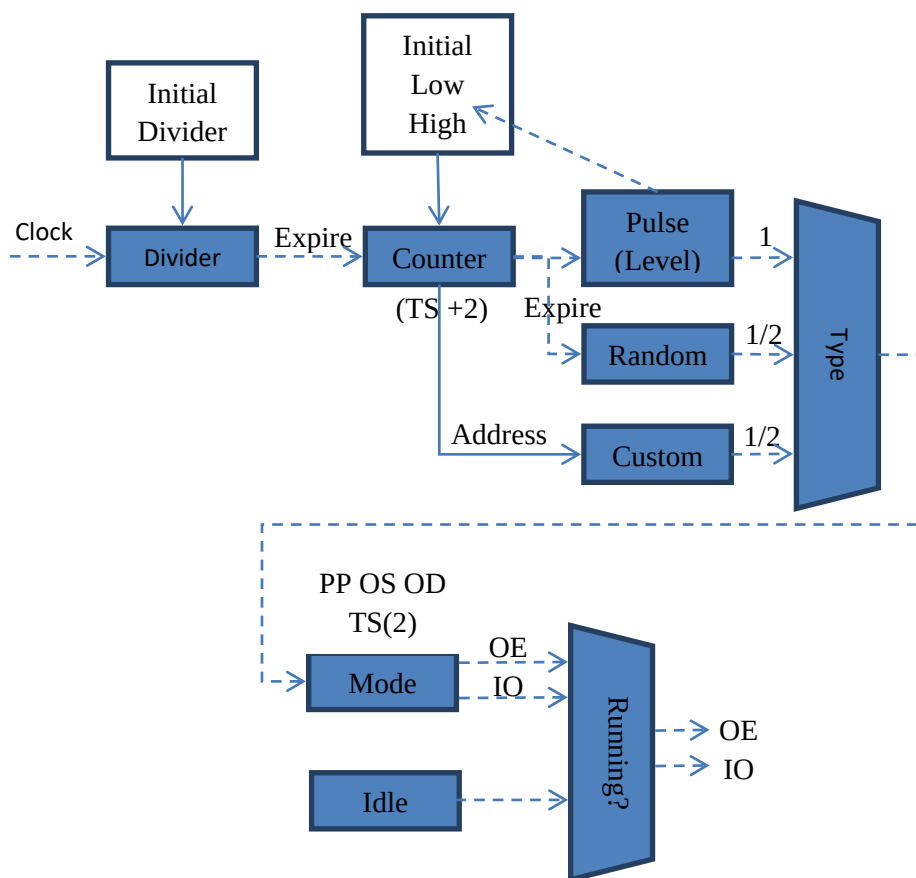
The states are described defined in dwf.h DwfState type.

- **Ready**: Initial state. After *FDwfDigitalOutConfigure* or any *FDwfDigitalOut\*Set* function call goes to this state. With digital out, configure start command goes to Armed state
- **Armed**: It waits for trigger.
- **Wait**: Remains in this state for the time period specified by *FDwfDigitalOutWaitSet* function.
- **Running**: Remains in this state for the time period specified by *FDwfDigitalOutRunSet* function.
- **Repeat**: Goes to Armed or Wait state according to the *FDwfDigitalOutRepeatTriggerSet* setting for the number of times specified by *FDwfDigitalOutRepeatSet*.
- **Done**: Final state.

This states machine controls all digital out channels.

See the following examples: [DigitalOut\\_BinaryCounter/Pins.py](#)

Channel configuration:



The initial values, for divider and counter, specify the initially loaded values, initial delay, when entering in Running state. The Divider specifies the clock division. This rate will be the custom sample frequency and step for the counter. When entering Running state, the initial value specified with *FDwfDigitalOutDividerInitSet* is loaded. When this expires, the value specified by *FDwfDigitalOutDividerSet* will be loaded upon each expiration. The Counter initial value is set by *FDwfDigitalOutCounterInitSet* function. This function also sets the initial level. When this expires the level values specified by *FDwfDigitalOutCounterSet* are loaded upon further expiration. On counter expiration the level is toggled, and this directs the low or high value loading. In case one of these is zero, the level is not toggled.

The Counter is used for:

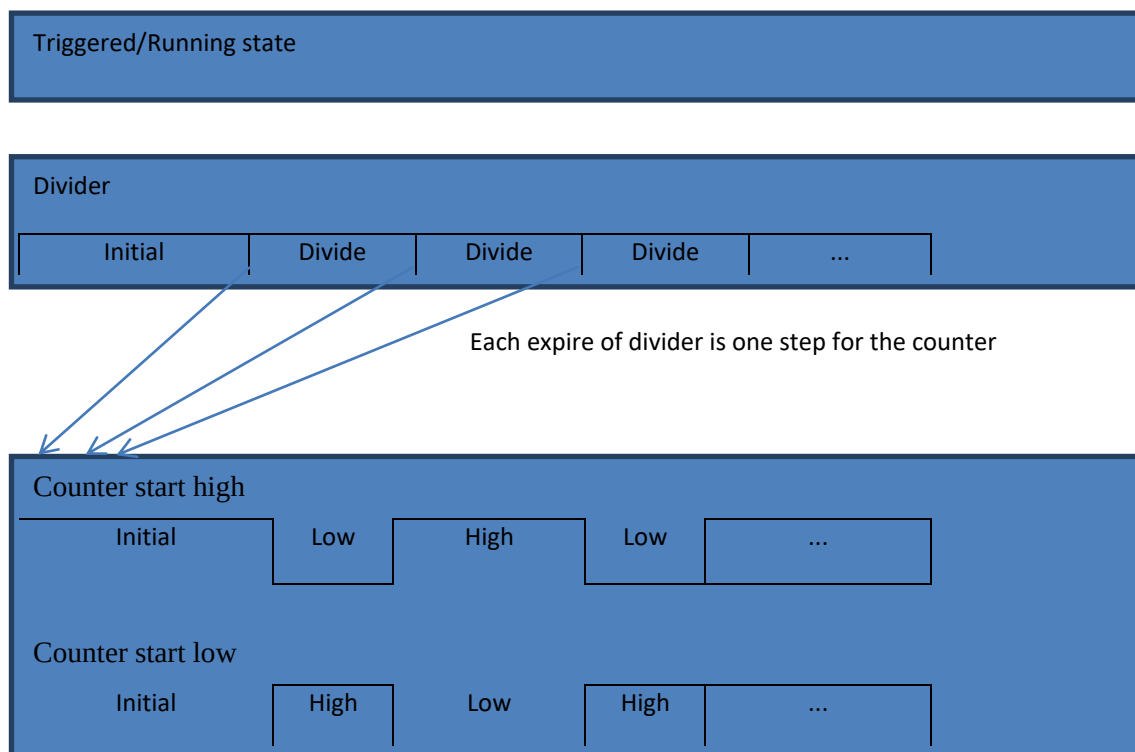
- Pulse to generate the low and high state lengths.
- Random to set update rate.
- Custom to address buffer. The samples are configured by *FDwfDigitalOutDataSet* function. This also configures the counter low/high according *countOfBits* parameter. In TS mode the counter step is double, providing two bits of samples for output: value and enable.

The output Mode (*FDwfDigitalOutModeSet*) selects between: PP, OS, OD and TS.

The Idle output (*FDwfDigitalOutIdleSet*) selects the output while not in *Running* state.



Pulse signal:



For pulse signal the initial level and initial value are specified with *FDwfDigitalOutCounterInitSet* function. These are loaded when entering Running state.

## 10.1 Control

**FDwfDigitalOutReset** (HDWF hdwf)

**Description:** Resets and configures (by default, having auto configure enabled) all the instrument parameters to default values.

**Parameters:**

- hdwf – Interface handle.

**FDwfDigitalOutConfigure** (HDWF hdwf, int fStart)

**Description:** Starts or stops the instrument.

**Parameters:**

- hdwf – Interface handle.

- fStart – Start the instrument. To stop, set to FALSE.

**FDwfDigitalOutStatus** (HDWF hdwf, DwfState \*psts)

**Description:** Checks the state of the instrument.

**Parameters:**

- hdwf – Interface handle.

- psts – Pointer to variable to return the state.

## 10.2 Configuration

**FDwfDigitalOutInternalClockInfo** (HDWF hdwf, double \*phzFreq)

**Description:** Retrieves the internal clock frequency.

**Parameters:**

- hdwf – Interface handle.

- phzFreq – Pointer to return the internal clock frequency.

See the description of **FDwfDeviceTriggerInfo**.

**FDwfDigitalOutTriggerSourceSet** (HDWF hdwf, TRIGSRC trigsrcc)

**Description:** Sets the trigger source for the instrument. Default setting is trigsrccNone.

**Parameters:**

- hdwf – Interface handle.

- trigsrcc – Trigger source to set.

---

**FDwfDigitalOutTriggerSourceGet**(HDWF hdwf, TRIGSRC \*ptrigsrc)

---

**Description:** Gets the current trigger source setting for the instrument.**Parameters:**

- hdwf – Interface handle.
- ptrigsrc – Pointer to variable to receive the trigger source.

---

**FDwfDigitalOutTriggerSlopeSet**(HDWF hdwf, DwfTriggerSlope slope)

---

**Description:** Sets the trigger slope for the instrument.**Parameters:**

- hdwf – Interface handle.
- slope – Trigger source to set.

---

**FDwfDigitalOutTriggerSlopeGet**(HDWF hdwf, DwfTriggerSlope \*pslope)

---

**Description:** Gets the current trigger source setting for the instrument.**Parameters:**

- hdwf – Interface handle.
- pslope – Pointer to variable to receive the trigger slope.

---

**FDwfDigitalOutRunInfo**(HDWF hdwf, double \*psecMin, double \*psecMax)

---

**Description:** Returns the supported run length range for the instrument in seconds. Zero value (default) represent an infinite (or continuous) run.**Parameters:**

- hdwf – Interface handle.
- psecMin – Variable to receive the supported minimum run length.
- psecMax – Variable to receive the supported maximum run length.

---

**FDwfDigitalOutRunSet**(HDWF hdwf, double secRun)

---

**Description:** Sets the run length for the instrument in Seconds.**Parameters:**

- hdwf – Interface handle.
- secRun – Run length to set expressed in seconds.

---

**FDwfDigitalOutRunGet**(HDWF hdwf, double \*psecRun)

---

**Description:** Reads the configured run length for the instrument in seconds.**Parameters:**

- hdwf – Interface handle.
- psecRun – Pointer to variable to receive the run length.

---

**FDwfDigitalOutRunStatus**(HDWF hdwf, double \*psecRun)

---

**Description:** Reads the remaining run length. It returns data from the last FDwfDigitalOutStatus call.**Parameters:**

- hdwf – Interface handle.
- psecRun – Pointer to variable to receive the remaining run length.

---

**FDwfDigitalOutWaitInfo**(HDWF hdwf, double \*psecMin, double \*psecMax)

---

**Description:** Returns the supported wait length range in seconds. The wait length is how long the instrument waits after being triggered to generate the signal. Default value is zero.**Parameters:**

- hdwf – Interface handle.
- psecMin – Variable to receive the supported minimum wait length.
- psecMax – Variable to receive the supported maximum wait length.

---

**FDwfDigitalOutWaitSet**(HDWF hdwf, double secWait)

---

**Description:** Sets the wait length.**Parameters:**

- hdwf – Interface handle.
- secWait – Wait length to set expressed in seconds.

---

**FDwfDigitalOutWaitGet**(HDWF hdwf, double \*psecWait)

---

**Description:** Gets the current wait length.**Parameters:**

- hdwf – Interface handle.
- psecWait – Pointer to variable to receive the wait length.

---

**FDwfDigitalOutRepeatInfo**(HDWF hdwf, unsigned int \*pnMin, unsigned int \*pnMax)

---

**Description:** Returns the supported repeat count range. This is how many times the generated signal will be repeated. Zero value represents infinite repeats. Default value is one.

**Parameters:**

- hdwf – Interface handle.
- pnMin – Variable to receive the supported minimum repeat count.
- pnMax – Variable to receive the supported maximum repeat count.

---

**FDwfDigitalOutRepeatSet**(HDWF hdwf, unsigned int cRepeat)

---

**Description:** Sets the repeat count.

**Parameters:**

- hdwf – Interface handle.
- cRepeat – Repeat count to set.

---

**FDwfDigitalOutRepeatGet**(HDWF hdwf, unsigned int \*pcRepeat)

---

**Description:** Reads the current repeat count.

**Parameters:**

- hdwf – Interface handle.
- pcRepeat – Pointer to variable to receive the repeat count.

---

**FDwfDigitalOutRepeatStatus**(HDWF hdwf, unsigned int \*pcRepeat)

---

**Description:** Reads the remaining repeat counts. It only returns information from the last FDwfDigitalOutStatus function call, it does not read from the device.

**Parameters:**

- hdwf – Interface handle.
- pcRepeat – Pointer to variable to receive the remaining repeat counts.

---

**FDwfDigitalOutRepeatTriggerSet**(HDWF hdwf, int fRepeatTrigger)

---

**Description:** Sets the repeat trigger option. To include the trigger in wait-run repeat cycles, set fRepeatTrigger to TRUE. It is disabled by default.

**Parameters:**

- hdwf – Interface handle.
- fRepeatTrigger – Boolean used to specify if the trigger should be included in a repeat cycle.

```
FDwfDigitalOutRepeatTriggerGet(HDWF hdwf, int *pfRepeatTrigger)
```

**Description:** Verifies if the trigger has been included in wait-run repeat cycles.

**Parameters:**

- hdwf – Open interface handle on a device.
- pfRepeatTrigger – Pointer to variable to receive the repeat trigger option.

```
FDwfDigitalOutCount(HDWF hdwf, int *pcChannel)
```

**Description:** Returns the number of Digital Out channels by the device specified by hdwf.

**Parameters:**

- hdwf – Interface handle.
- pcChannel – Pointer to variable to receive the number of channels in the instrument.

```
FDwfDigitalOutEnableSet(HDWF hdwf, int idxChannel, int fEnable)
```

**Description:** Enables or disables the channel specified by idxChannel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- fEnable – TRUE to enable, FALSE to disable.

```
FDwfDigitalOutEnableGet(HDWF hdwf, int idxChannel, int *pfEnable)
```

**Description:** Verifies if a specific channel is enabled or disabled.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pfEnable – Pointer to variable to receive enabled state.

```
FDwfDigitalOutOutputInfo(
  HDWF hdwf, int idxChannel, int *pfsDwfDigitalOutOutput)
```

**Description:** Returns the supported output modes of the channel. They are returned (by reference) as a bit field.

This bit field can be parsed using the `IsBitSet` Macro. Individual bits are defined using the `DwfDigitalOutOutput` constants in `DWF.h`:

- **DwfDigitalOutOutputPushPull**: Default setting.
- **DwfDigitalOutOutputOpenDrain**: External pull needed.
- **DwfDigitalOutOutputOpenSource**: External pull needed.
- **DwfDigitalOutOutputThreeState**: Available with custom and random types.

**Parameters:**

- `hdwf` – Interface handle.
- `idxChannel` – Channel index.
- `pfsDwfDigitalOutOutput` – Pointer to variable to receive the supported output modes.

```
FDwfDigitalOutOutputSet(HDWF hdwf, int idxChannel, DwfDigitalOutOutput v)
```

**Description:** Specifies output mode of the channel.

**Parameters:**

- `hdwf` – Interface handle.
- `idxChannel` – Channel index.
- `v` – Output mode.

```
FDwfDigitalOutOutputGet(HDWF hdwf, int idxChannel, DwfDigitalOutOutput *pv)
```

**Description:** Verifies if a specific channel output mode.

**Parameters:**

- `hdwf` – Interface handle.
- `idxChannel` – Channel index.
- `pfEnable` – Pointer to variable to receive enabled state.

---

**FDwfDigitalOutTypeInfo**(HDWF hdwf, int idxChannel, int \*pfsDwfDigitalOutType)
 

---

**Description:** Returns the supported types of the channel. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the DwfDigitalOutType constants in dwf.h:

- **DwfDigitalOutTypePulse:** Frequency = internal frequency/divider/(low + high counter).
- **DwfDigitalOutTypeCustom:** Sample rate = internal frequency / divider.
- **DwfDigitalOutTypeRandom:** Random update rate = internal frequency/divider/counter alternating between low and high values.
- **DwfDigitalOutTypeROM:** ROM logic, the DIO input value is used as address for output value
- **DwfDigitalOutTypePlay:** Supported with Digital Discovery.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pfsDwfDigitalOutType – Pointer to variable to receive the supported output types.

---

**FDwfDigitalOutTypeSet**(HDWF hdwf, int idxChannel, DwfDigitalOutType v)
 

---

**Description:** Sets the output type of the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- v – Output mode.

---

**FDwfDigitalOutTypeInfoGet**(HDWF hdwf, int idxChannel, DwfDigitalOutType \*pv)
 

---

**Description:** Verifies the type of a specific channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pfEnable – Pointer to variable to receive enabled state.

---

**FDwfDigitalOutIdleInfo**(HDWF hdwf, int idxChannel, int \*pfsDwfDigitalOutIdle)
 

---

**Description:** Returns the supported idle output types of the channel. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the DwfDigitalOutIdle constants in dwf.h. Output while not running:

- **DwfDigitalOutIdleInit:** Output initial value.
- **DwfDigitalOutIdleLow:** Low level.
- **DwfDigitalOutIdleHigh:** High level.
- **DwfDigitalOutIdleZet:** Three state.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pfsDwfDigitalOutIdle – Pointer to variable to receive the supported idle output types.



```
FDwfDigitalOutIdleSet(HDWF hdwf, int idxChannel, DwfDigitalOutIdle v)
```

**Description:** Sets the idle output of the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- v – Value to set idle output.

```
FDwfDigitalOutIdleGet(HDWF hdwf, int idxChannel, DwfDigitalOutIdle *pv)
```

**Description:** Verifies the idle output of a specific channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pv – Pointer to variable to receive configured value.

```
FDwfDigitalOutDividerInfo(  
HDWF hdwf, int idxChannel, unsigned int *vMin, unsigned int *vMax)
```

**Description:** Returns the supported clock divider value range.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pnMin – Variable to receive the supported minimum divider value.
- pnMax – Variable to receive the supported maximum divider value.

```
FDwfDigitalOutDividerInitSet(HDWF hdwf, int idxChannel, unsigned int v)
```

**Description:** Sets the initial divider value of the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- v – Divider initial value.

```
FDwfDigitalOutDividerInitGet(HDWF hdwf, int idxChannel, unsigned int *pv)
```

**Description:** Verifies the initial divider value of the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pv – Pointer to variable to receive configured value.

```
FDwfDigitalOutDividerSet(HDWF hdwf, int idxChannel, unsigned int v)
```

**Description:** Sets the divider value of the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- v – Divider value.

```
FDwfDigitalOutDividerGet(HDWF hdwf, int idxChannel, unsigned int *pv)
```

**Description:** Verifies the divider value of the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pv – Pointer to variable to receive configured value.

```
FDwfDigitalOutCounterInfo(  
HDWF hdwf, int idxChannel, unsigned int *vMin, unsigned int *vMax)
```

**Description:** Returns the supported counter value range.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pnMin – Variable to receive the supported minimum counter value.
- pnMax – Variable to receive the supported maximum counter value.

```
FDwfDigitalOutCounterInitSet(  
HDWF hdwf, int idxChannel, int fHigh, unsigned int v)
```

**Description:** Sets the initial state and counter value of the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- fHigh – Start high.
- v – Counter initial value.

```
FDwfDigitalOutCounterInitGet(
  HDWF hdwf, int idxChannel, int *pfHigh, unsigned int *pv)
```

**Description:** Retrieves the initial state and counter value for the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pfHigh – Pointer to variable to receive configured value.
- pv – Pointer to variable to receive configured value.

```
FDwfDigitalOutCounterSet(
  HDWF hdwf, int idxChannel, unsigned int vLow, unsigned int vHigh)
```

**Description:** Sets the counter low and high values for the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- vLow – Counter low value.
- vHigh – Counter high value.

```
FDwfDigitalOutCounterGet(
  HDWF hdwf, int idxChannel, unsigned int *pvLow, unsigned int *pvHigh)
```

**Description:** Verifies the low and high counter value of the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pvLow – Pointer to variable to receive configured value.
- pvHigh – Pointer to variable to receive configured value.

**Repetition** specifies how many times the counter should be reloaded. For pulse signals set twice the desired value since each pulse is generated by two counter loads, low and high.

It is available with ADP3X50 and newer devices.

```
FDwfDigitalOutRepetitionInfo(HDWF hdwf, int idxChannel, unsigned int *pnMax)
```

**Description:** Returns the supported repetition value range.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pnMax – Variable to receive the supported maximum value,  $2^{31}$

```
FDwfDigitalOutRepetitionSet(HDWF hdwf, int idxChannel, unsigned int cRepeat)
```

**Description:** Sets the counter low and high values for the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- cRepeat – Repetition value for low and high counts. Use twice the repeat for pulse, like 2 to repeat once.

```
FDwfDigitalOutRepetitionGet(HDWF hdwf, int idxChannel, unsigned int *cRepeat)
```

**Description:** Verifies the low and high counter value of the specified channel.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pvLow – Pointer to variable to receive configured value.
- pvHigh – Pointer to variable to receive configured value.

```
FDwfDigitalOutDataInfo(  
HDWF hdwf, int idxChannel, unsigned int *pcountOfBitsMax)
```

**Description:** Returns the maximum buffers size, the number of custom data bits.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- pcountOfBitsMax – Variable to receive the maximum number of bits.

```
FDwfDigitalOutDataSet(  
HDWF hdwf, int idxChannel, void *rgBits, unsigned int countOfBits)
```

**Description:** Sets the custom data bits. The function also sets the counter initial, low and high value, according the number of bits. The data bits are sent out in LSB first order. For TS output, the count of bits is the total number of output value (I/O) and output enable (OE) bits, which should be an even number.

| Custom Data Bits |        |       |        |       |     |       |        |       |     |
|------------------|--------|-------|--------|-------|-----|-------|--------|-------|-----|
| <b>BYTE</b>      | 0      |       |        |       |     |       | 1      |       |     |
| <b>Bits</b>      | 0      | 1     | 2      | 3     | ... | 7     | 0      | 1     | ... |
| <b>Output</b>    | I/O(0) | OE(0) | I/O(1) | OE(1) | ... | OE(3) | I/O(4) | OE(4) | ... |

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- rgBits – Custom data array.
- countOfBits – Number of bits.

The **Play** functions are supported by Digital Discovery.

---

```
FDwfDigitalOutPlayRateSet(HDWF hdwf, double hzRate)
```

---

**Description:** Sets the play rate.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Channel index.
- hzRate – Sample rate.

---

```
FDwfDigitalOutPlayDataSet(HDWF hdwf, unsigned char *rgBits,  
unsigned int bitPerSample, unsigned int countOfSamples)
```

---

**Description:** Sets the data bits to play. The output can be DwfDigitalOutOutputPushPull, OpenDrain or OpenSource. ThreeState is not supported.

**Parameters:**

- hdwf – Interface handle.
- rgBits – Data array to play.
- bitPerSample – Bits per sample, should be 1, 2, 4, 8 or 16. The expected bit order in rgBits:
  - 1: rgBits[DIO-0-S0, DIO-0-S1, DIO-0-S2, ...]
  - 2: rgBits[DIO-0-S0, DIO-1-S0, DIO-0-S1, DIO-1-S1, ...]
  - 4: rgBits[DIO-0-S0, DIO-1-S0, DIO-2-S0, DIO-3-S0, DIO-0-S1, DIO-1-S1, DIO-2-S1, DIO-3-S1, ...]
  - 8: rgBits[DIO-0-S0, DIO-1-S0, ..., DIO-7-S0, DIO-0-S1, DIO-1-S1, ..., DIO-7-S1, ...]
  - 16: rgBits[DIO0-S0, DIO-1-S0, ..., DIO-15-S0, DIO-0-S1, DIO-1-S1, ..., DIO-15-S1, ...]
- countOfSamples – Number of samples

---

```
FDwfDigitalOutPlayUpdateSet(HDWF hdwf, unsigned char *rgBits,  
unsigned int indexOfSample, unsigned int countOfSamples)
```

---

**Description:** Changes the data bits configured to play.

**Parameters:**

- hdwf – Interface handle.
- rgBits – Data array with new values.
- indexOfSample – Start index were to insert the new value.
- countOfSamples – Number of new samples.

## 11 Miscellaneous

See examples: AnalogIn\_FFT.py AnalogIn\_Frequency.py AnalogIn\_Spectrum.py

```
FDwfSpectrumWindow (double * rgdWin, int cdWin, DwfWindow iWindow,
double vBeta, double * vNEBW)
```

**Description:** Generates a window function.

**Parameters:**

- rgdWin – Data array for window data.
- cdWin – Data array size (N).
- iWindow – Window type, DwfWindow:
  - DwfWindowRectangular 0
  - DwfWindowTriangular 1  $W_x = (2/(N+1)) * ((N+1)/2 - \text{abs}(X - (N-1)/2))$
  - DwfWindowHamming 2  $W_x = 0.54 - 0.46 * \cos(2 * \text{PI} * X / (N-1))$
  - DwfWindowHann 3  $W_x = 0.5 - 0.5 * \cos(2 * \text{PI} * X / (N-1));$
  - DwfWindowCosine 4  $W_x = \sin(\text{PI} * X / (N-1))$
  - DwfWindowBlackmanHarris 5
  $W_x = 0.35875 - 0.48829 * \cos(2 * \text{PI} * X / (N-1)) + 0.14128 * \cos(4 * \text{PI} * X / (N-1)) - 0.01168 * \cos(6 * \text{PI} * X / (N-1))$
  - DwfWindowFlatTop 6
  $W_x = 1 - 1.93 * \cos(2 * \text{PI} * X / (N-1)) + 1.29 * \cos(4 * \text{PI} * X / (N-1)) - 0.388 * \cos(6 * \text{PI} * X / (N-1)) + 0.028 * \cos(8 * \text{PI} * X / (N-1))$
  - DwfWindowKaiser 7  $W_x = \text{bessel\_i0}(\text{beta} * \sqrt{1 - ((2 * X - (N-1)) / n(N-1)m))^2})$
  - DwfWindowBlackman 8
  $W_x = 0.42 - 0.5 * \cos(2 * \text{PI} * X / (N-1)) + 0.08 * \cos(4 * \text{PI} * X / (N-1))$
  - DwfWindowFlatTopM 9
  $W_x = 1 - 1.93 * \cos(2 * \text{PI} * X / (N-1)) + 1.29 * \cos(4 * \text{PI} * X / (N-1)) - 0.388 * \cos(6 * \text{PI} * X / (N-1)) + 0.0322 * \cos(8 * \text{PI} * X / (N-1))$
- vBeta – Beta parameter used by Kaiser window.
- vNEBW – Noise equivalent bandwidth (optional). ( $= N * \text{sum}(w^2) / \text{sum}(w)^2$ )

```
FDwfSpectrumFFT (const double * rgdData, int cdData,
double * rgdBin, double * rgdPhase, int cdBin)
```

**Description:** Performs FFT on data array and returns BINs and/or Phase.

**Parameters:**

- rgdData – Data array input.
- cdData – Data array size. It should be power of two number, otherwise
- rgdBin – BIN array output in peak voltage unit (optional). DC to rate/2 with rate/2/(cdBin-1) resolution
- rgdPhase – Phase array output in radian unit (optional).
- cdBin – BIN and Phase array size. It must be cdData/2+1.

```
FDwfSpectrumTransform(const double * rgdData, int cdData,  
double * rgdBin, double * rgdPhase, int cdBin, double iFirst, double iLast)
```

**Description:** Performs FFT or CZT on data array and returns BINs and/or Phase.

**Parameters:**

- rgdData – Data array input.
- cdData – Data array size.
- rgdBin – BIN array output in peak voltage unit (optional). For dBV use  $20 \cdot \log_{10}(\text{BIN}/\sqrt{2})$
- rgdPhase – Phase array output in radian unit (optional).
- cdBin – BIN and Phase array size.
- iFirst – Start of frequency domain, between 0 and 1 interpreted as DC and sample rate/2.
- iLast – End of frequency domain, between 0 and 1 interpreted as DC to sample rate/2.

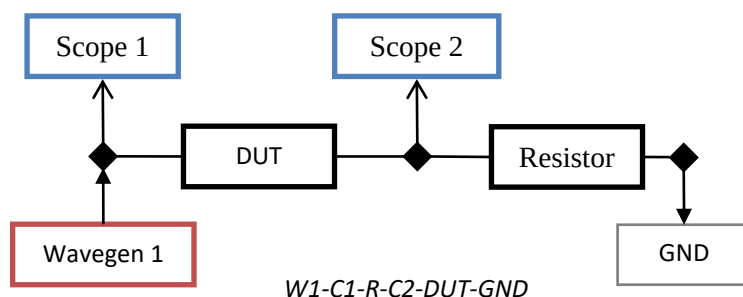
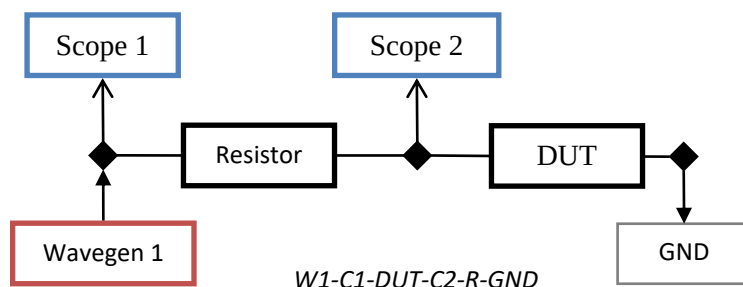
Like for 100MHz sample rate: for 0.0 and 1.0 will output 0Hz to 50MHz; 0.1 and 0.5 output 5MHz to 25MHz

## 12 Analog Impedance

The Analog Impedance functions use the Analog Input and Output channel 1.

When used with Impedance Analyzer module the Analog IO (Power Supplies) and Digital IO will be used too.

These functions can be used with [Impedance Analyzer for Analog Discovery](#) (specify this with value 8 in FDwfAnalogImpedanceModeSet function) or the analyzer circuit can be constructed in the following ways:



The DUT (Device Under Test) stands for the inductive or capacitive load to be analyzed and Resistor is the reference resistor. The resistor value depends on the load value and frequency.



See AnalogImpedance\_Meter.py, AnalogImpedance\_Compensation.py examples.

---

**FDwfAnalogImpedanceReset**(HDWF hdwf)

---

**Description:** Resets the AI configuration to default value.

**Parameters:**

- hdwf – Interface handle.

---

**FDwfAnalogImpedanceModeSet**(HDWF hdwf, int mode)

---

**Description:** Specifies the circuit to be used.

**Parameters:**

- hdwf – Interface handle.

- mode – circuit model to be used

- 0: W1-C1-DUT-C2-R-GND

- 1: W1-C1-R-C2-DUT-GND

- 8: Impedance Analyzer module for Analog Discovery

---

**FDwfAnalogImpedanceModeGet**(HDWF hdwf, int \*pmode)

---

**Description:** Returns the selected circuit model.

**Parameters:**

- hdwf – Interface handle.

- pmode – Pointer to variable to receive configured value.

---

**FDwfAnalogImpedanceReferenceSet**(HDWF hdwf, double ohms)

---

**Description:** Specifies the reference resistor to be used.

For AD IA module the resistor is selected by relays controlled by power supplies and digital IOs.

**Parameters:**

- hdwf – Interface handle.

- ohms – Reference resistor value.

---

**FDwfAnalogImpedanceReferenceGet**(HDWF hdwf, double \*pohms)

---

**Description:** Returns the reference resistor value.

**Parameters:**

- hdwf – Interface handle.

- pohms – Pointer to variable to receive configured value.

---

**FDwfAnalogImpedanceFrequencySet**(HDWF hdwf, double hz)

---

**Description:** Configures the stimulus frequency and restart the capture.**Parameters:**

- hdwf – Interface handle.
- hz – Frequency value.

---

**FDwfAnalogImpedanceFrequencyGet**(HDWF hdwf, double \*phz)

---

**Description:** Returns the frequency value.**Parameters:**

- hdwf – Interface handle.
- phz – Pointer to variable to receive configured value.

---

**FDwfAnalogImpedanceAmplitudeSet**(HDWF hdwf, double volts)

---

**Description:** Configures the stimulus signal amplitude, half of the peak to peak value.**Parameters:**

- hdwf – Interface handle.
- volts – Amplitude value.

---

**FDwfAnalogImpedanceAmplitudeGet**(HDWF hdwf, double \*pvolts)

---

**Description:** Returns the amplitude value.**Parameters:**

- hdwf – Interface handle.
- pvolts – Pointer to variable to receive configured value.

---

**FDwfAnalogImpedanceOffsetSet**(HDWF hdwf, double volts)

---

**Description:** Configures the stimulus signal offset.**Parameters:**

- hdwf – Interface handle.
- volts – Offset value.

---

**FDwfAnalogImpedanceOffsetGet**(HDWF hdwf, double \*pvolts)

---

**Description:** Returns the offset value.**Parameters:**

- hdwf – Interface handle.
- pvolts – Pointer to variable to receive configured value.

---

**FDwfAnalogImpedanceProbeSet**(HDWF hdwf, double ohmRes, double faradCap)

---

**Description:** Specifies the probe impedance that will be taken in consideration for measurements.

The default values are set specific for device when calling the FDwfAnalogImpedanceReset function.

**Parameters:**

- hdwf – Interface handle.
- ohmRes – Probe resistance.
- faradCap – Probe capacitance.

---

**FDwfAnalogImpedanceProbeGet**(HDWF hdwf, double \*pohmRes, double \*pfaradCap)

---

**Description:** Returns the probe impedance.**Parameters:**

- hdwf – Interface handle.
- pohmRes – Pointer to variable to receive configured value.
- pfaradCap – Pointer to variable to receive configured value.

---

**FDwfAnalogImpedancePeriodSet**(HDWF hdwf, int cMinPeriods)

---

**Description:** Specifies the minimum number of periods to be captured.**Parameters:**

- hdwf – Interface handle.
- cMinPeriods – Number of minimum periods, default 16.

---

**FDwfAnalogImpedancePeriodGet**(HDWF hdwf, int \*pcMinPeriods)

---

**Description:** Returns the periods value.**Parameters:**

- hdwf – Interface handle.
- pcMinPeriods – Pointer to variable to receive configured value.

---

**FDwfAnalogImpedanceCompReset**(HDWF hdwf)

---

**Description:** Resets the currently configured compensation parameters.**Parameters:**

- hdwf – Interface handle.

```
FDwfAnalogImpedanceCompSet(HDWF hdwf,  
double ohmOpenResistance, double ohmOpenReactance,  
double ohmShortResistance, double ohmShortReactance)
```

**Description:** Specifies the open and short compensation parameters. These values are specific for the circuit/adapter.

**Parameters:**

- hdwf – Interface handle.
- ohmOpenResistance – Open resistance value.
- ohmOpenReactance – Open reactance value.
- ohmShortResistance – Short resistance value.
- ohmShortReactance – Short reactance value.

```
FDwfAnalogImpedanceCompGet(HDWF hdwf,  
double *pohmOpenResistance, double *pohmOpenReactance,  
double *pohmShortResistance, double *pohmShortReactance)
```

**Description:** Returns the compensation parameters.

**Parameters:**

- hdwf – Interface handle.
- pohmOpenResistance – Pointer to variable to receive configured value.
- pohmOpenReactance – Pointer to variable to receive configured value.
- pohmShortResistance – Pointer to variable to receive configured value.
- pohmShortReactance – Pointer to variable to receive configured value.

```
FDwfAnalogImpedanceConfigure(HDWF hdwf, int fStart)
```

**Description:** Configures the instrument and start or stop the analysis.

**Parameters:**

- hdwf – Interface handle.
- fStart – Start the analysis.

```
FDwfAnalogImpedanceStatus(HDWF hdwf, DwfState *psts)
```

**Description:** Checks the state of the acquisition.

**Parameters:**

- hdwf – Interface handle.
- psts – Variable to receive the acquisition state.

```
FDwfAnalogImpedanceStatusInput(HDWF hdwf,  
int idxChannel, double *pgain, double *pradian)
```

**Description:** Read the raw input, for network analysis purpose. This returns the raw values without taking in consideration the probe characteristics or compensation parameters.

For scope channel 1 (idxChannel = 0) the gain is relative to Wavegen amplitude (Amplitude/Channel1) and the phase is zero. For further channels the gain and phase is relative to channel 1, gain = C1/C#  
The gain value is dimensionless, it represents the V/V ratio.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Scope channel index.
- pgain – Pointer to variable to receive the gain.
- pradian – Pointer to variable to receive the phase.

```
FDwfAnalogImpedanceStatusWarning(HDWF hdwf, int idxChannel, int *pWarning)
```

**Description:** This returns the warning if the scope input range is exceeded.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Scope channel index.
- pWarning– Pointer to variable to receive the warning status, 1 low, 2 high, 3 both.

```
FDwfAnalogImpedanceStatusMeasure(HDWF hdwf,  
DwfAnalogImpedance measure, double *pvalue)
```

**Description:** Read the DUT measurements. These take in account the scope probe characteristics and compensation parameters.

| DwfAnalogImpedance                    | int | Impedance measurement                                |
|---------------------------------------|-----|------------------------------------------------------|
| DwfAnalogImpedanceImpedance           | 0   | $ Z $ in Ohms = $\text{Sqrt}(R_s^2 + X_s^2)$         |
| DwfAnalogImpedanceImpedancePhase      | 1   | $\emptyset$ in Radians = $\text{Tan2}(X_s, R_s)$     |
| DwfAnalogImpedanceResistance          | 2   | $R_s$ in Ohms                                        |
| DwfAnalogImpedanceReactance           | 3   | $X_s$ in Ohms = $-1/(\omega * C_s) = \omega * L_s$   |
| DwfAnalogImpedanceAdmittance          | 4   | $ Y $ in Siemen = $\text{Sqrt}(G_p^2 + B_p^2)$       |
| DwfAnalogImpedanceAdmittancePhase     | 5   | $\emptyset$ in Radians = $\text{Tan2}(B_p, G_p)$     |
| DwfAnalogImpedanceConductance         | 6   | $G_p$ in Siemen                                      |
| DwfAnalogImpedanceSusceptance         | 7   | $B_p$ in Siemen = $\omega * C_p = -1/(\omega * L_p)$ |
| DwfAnalogImpedanceSeriesCapactance    | 8   | $C_s$ in Farad = $-1/(\omega * X_s)$                 |
| DwfAnalogImpedanceParallelCapacitance | 9   | $C_p$ in Farad = $B_p / \omega$                      |
| DwfAnalogImpedanceSeriesInductance    | 10  | $L_s$ in Henry = $X_s / \omega$                      |
| DwfAnalogImpedanceParallelInductance  | 11  | $L_p$ in Henry = $-1/(\omega * B_p)$                 |
| DwfAnalogImpedanceDissipation         | 12  | Dissipation factor = $R_s/X_s = G_p/B_p$             |
| DwfAnalogImpedanceQuality             | 13  | Quality factor = $X_s/R_s = B_p/G_p$                 |
| DwfAnalogImpedanceVrms                | 14  | RMS voltage on DUT                                   |
| DwfAnalogImpedanceVreal               | 15  | Real voltage on DUT                                  |
| DwfAnalogImpedanceVimag               | 16  | Imaginary voltage on DUT                             |
| DwfAnalogImpedanceIrms                | 17  | RMS current on DUT, resistor voltage/resistor value  |
| DwfAnalogImpedanceIreal               | 18  | Real current on DUT                                  |
| DwfAnalogImpedanceIimag               | 19  | Imaginary current on DUT                             |
|                                       |     | $\omega = 2 * \pi * \text{Hz}$                       |

**Parameters:**

- hdwf – Interface handle.
- measure – Select measurement to return.
- pvalue – Pointer to variable to receive the measurement.

## 13 Digital Protocols

The protocols use the Digital-In/Out device resources to create various communication protocols. Only one of the protocols can be used at a time. The Digital-Out is used to generate the output signals and the Digital-In to capture the received data. Overriding a signal used by the protocol with Digital-IO may mess up the communication.

Note:

The DIO channel indexing for Digital Discovery starts from 0, 0 is DIO-24, 1 is DIO-25...

### 13.1 UART

See Digital\_Uart.py example.

**FDwfDigitalUartReset**(HDWF hdwf)

**Description:** Resets the UART configuration to default value. Use FDwfDigitalOutReset to reset the output.

**Parameters:**

- hdwf – Interface handle.

**FDwfDigitalUartRateSet**(HDWF hdwf, double hz)

**Description:** Sets the data rate.

**Parameters:**

- hdwf – Interface handle.

- hz – data rate to set

**FDwfDigitalUartBitsSet**(HDWF hdwf, double cBits)

**Description:** Sets the character length, typically 8, 7, 6 or 5.

**Parameters:**

- hdwf – Interface handle.

- cBits – character length

**FDwfDigitalUartParitySet**(HDWF hdwf, int parity)

**Description:** Sets the parity bit: 0 no parity, 1 even, 2 odd, 3 mark (high), 4 space (low)

**Parameters:**

- hdwf – Interface handle.

- parity – parity mode to set

**FDwfDigitalUartPolaritySet**(HDWF hdwf, int polarity)

**Description:** Sets the polarity 0 normal and 1 inverted.

**Parameters:**

- hdwf – Interface handle.

- polarity – polarity to set

---

**FDwfDigitalUartStopSet**(HDWF hdwf, double cBits)
 

---

**Description:** Sets the stop length as number of bits.

**Parameters:**

- hdwf – Interface handle.
- cBits – stop length

---

**FDwfDigitalUartTxSet**(HDWF hdwf, int idxChannel)
 

---

**Description:** Specifies the DIO channel to use for transmission.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – DIO channel to use for TX

---

**FDwfDigitalUartRxSet**(HDWF hdwf, int idxChannel)
 

---

**Description:** Specifies the DIO channel to use for reception.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – DIO channel to use for RX

---

**FDwfDigitalUartTx**(HDWF hdwf, char \*szTx, int cTx)
 

---

**Description:** Transmits the specified characters.

**Parameters:**

- hdwf – Interface handle.
- szTX – array of characters to send
- cTX – number of characters to send

---

**FDwfDigitalUartRx**(HDWF hdwf, char \*szRx, int cRxMax, int \*pcRx, int \*pParity)
 

---

**Description:** Initializes the reception with cRxMax zero. Otherwise returns the received characters since the last call.

**Parameters:**

- hdwf – Interface handle.
- szRX – buffer to receive characters
- cRxMax – the maximum number of characters to receive, the buffer size
- pcRX – pointer to return the number of characters received
- pParity – pointer to return:
  - negative value for buffer overflow, in case the buffer got full since the previos call of this function
  - the first parity check failure as one based index
  - zero for no error



## 13.2 SPI

See examples: Digital\_Spi.py, Digital\_Spi\_Dual.py, Digital\_Spi\_Quad.py, Digital\_Spi\_Siso.py

The Select signal is controlled automatically for transfer functions if it is configured by the FDwfDigitalSpiSelectSet and it is not activated with the FDwfDigitalSpiSelect, having the same DIO and active level.

**FDwfDigitalSpiReset** (HDWF hdwf)

**Description:** Resets the SPI configuration to default value. Use FDwfDigitalOutReset to reset the outputs.

**Parameters:**

- hdwf – Interface handle.

**FDwfDigitalSpiFrequencySet** (HDWF hdwf, double hz)

**Description:** Sets the SPI frequency.

**Parameters:**

- hdwf – Interface handle.

- hz – bit rate to set (default 1kHz)

**FDwfDigitalSpiClockSet** (HDWF hdwf, int idxChannel)

**Description:** Specifies the DIO channel to use for SPI clock.

**Parameters:**

- hdwf – Interface handle.

- idxChannel – DIO channel to use for SPI clock (default DIO1)

**FDwfDigitalSpiDataSet** (HDWF hdwf, int idxDQ, int idxChannel)

**Description:** Specifies the DIO channels to use for SPI data.

**Parameters:**

- hdwf – Interface handle.

- idxDQ – specify data index to set, 0 = DQ0\_MOSI\_SISO, 1 = DQ1\_MISO, 2 = DQ2, 3 = DQ3

- idxChannel – DIO channel to use for SPI data

**FDwfDigitalSpiIdleSet** (HDWF hdwf, int idxDQ, DwfDigitalOutIdle idle)

**Description:** Specifies the DQ signal idle output state. DQ2 and 3 may be used for alternative purpose like for write protect (should driven low) or for hold (should be in high impedance).

**Parameters:**

- hdwf – Interface handle.

- idxDQ – specify data index

- idle – idle state of the channel

**FDwfDigitalSpiModeSet**(HDWF hdwf, `int` iMode)

**Description:** Sets the SPI mode.

| iMode | CPOL | CPHA |
|-------|------|------|
| 0     | 0    | 0    |
| 1     | 0    | 1    |
| 2     | 1    | 0    |
| 3     | 1    | 1    |

**Parameters:**

- hdwf – Interface handle.
- iMode – specify SPI mode, bit1 CPOL and bit0 CPHA. With bit2=1 the MISO will be sampled on the active edge.

**FDwfDigitalSpiOrderSet**(HDWF hdwf, `int` fMSBLSB)

**Description:** Sets the bit order for SPI data.

**Parameters:**

- hdwf – Interface handle.
- fMSB – Specify bit order, 1 MSB first (default), 0 LSB first

**FDwfDigitalSpiDelaySet**(HDWF hdwf, `int` cStart, `int` cCmd, `int` cWord, `int` cStop)

**Description:** Sets the delays used by FDwfDigitalSpiCmdWrite/Read\* functions.

**Parameters:**

- hdwf – Interface handle.
- cStart – Specify delay after start in clock cycle length.
- cCmd – Specify delay after command.
- cWord – Specify delay between words.
- cStop – Specify delay before stop.

**FDwfDigitalSpiSelectSet**(HDWF hdwf, `int` idxChannel, `int` fIdle)

**Description:** Specifies the SPI chip select signal.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Specify the DIO channel
- fIdle – Set the idle level: 0 low, 1 high

**FDwfDigitalSpiSelect**(HDWF hdwf, int idxChannel, int iLevel)

**Description:** Controls the SPI CS signal(s) FDwfDigitalSpiWrite/Read\* functions.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – Specify DIO channel for which to set the level
- iLevel – Set the channel level: 0 low, 1 high, -1 release (Z, high impedance)

**FDwfDigitalSpiWriteRead**(HDWF hdwf, int cDQ, int cBitPerWord, unsigned char \*rgTX, int cTX, unsigned char \*rgRX, int cRX)

**Description:** Performs SPI transfer of up to 8bit words. This function is intended for standard MOSI/MISO (cDQ 1) operations, but it can be used for other modes as long only write (rgTX/cTX) or read (rgRX/cRX) is specified.

The number of clock signals generated is the maximum of cTX and cRX.

**Parameters:**

- hdwf – Interface handle.
- cDQ – specify the DQ lines to use to transfer data:

| cDQ |           |                                                                                        |
|-----|-----------|----------------------------------------------------------------------------------------|
| 0   | SISO      | use only DQ0 for read and write                                                        |
| 1   | MOSI/MISO | use DQ0 for write and DQ1 to read data                                                 |
| 2   | DUAL      | use DQ0 for even and DQ1 for odd bits                                                  |
| 3   | QUAD      | use DQ0 for 0,4,8..., DQ1 for 1,5,9..., DQ2 for 2,6,10..., DQ3 for 3,7,11... data bits |

- cBitPerWord – specify the number of bits to transfer for each word.
- rgTX – array of 8bit values (byte words) to write.
- cTX – number of words to write.
- rgRX – buffer for read words.
- cRX – number of words to read.

**FDwfDigitalSpiWriteOne**(HDWF hdwf, int cDQ, int cBits, unsigned int vTX)

**Description:** Performs SPI transmit of up to 32 bits. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cDQ – Specify the DQ lines to use to transfer data.
- cBits – Specify the number of bits to transmit.
- vTX – Specify the data to transmit.

**FDwfDigitalSpiReadOne**(HDWF hdwf, int cDQ, int cBits, unsigned int \*pRX)

**Description:** Performs SPI reception of up to 32 bits. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cDQ – Specify the DQ lines to use to transfer data.
- cBits – Specify the number of bits to receive.
- pRX – Pointer to variable to return the received bits.

```
FDwfDigitalSpiWrite(HDWF hdwf, int cDQ, int cBitPerWord,  
unsigned char *rgTX, int cTX)
```

**Description:** Performs SPI transmission of up to 8-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cDQ – Specify the DQ lines to use to transfer data.
- cBitPerWord – Specify the number of bits to transfer for each word.
- rgTX – Array of 8-bit values (words) to transmit.
- cTX – Number of words to transmit.

```
FDwfDigitalSpiRead(HDWF hdwf, int cDQ, int cBitPerWord,  
unsigned char *rgRX, int cRX)
```

**Description:** Performs SPI reception of up to 8-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cDQ – Specify the DQ lines to use to transfer data.
- cBitPerWord – Specify the number of bits to transfer for each word.
- rgRx – Buffer for receive words.
- cRx – Number of words to received.

```
FDwfDigitalSpiWriteRead16(HDWF hdwf, int cDQ, int cBitPerWord,
unsigned short *rgTX, int cTX, unsigned short *rgRX, int cRX)
```

**Description:** Performs SPI transfer of up to 16-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cDQ – Specify the DQ lines to use to transfer data.
- cBitPerWord – Specify the number of bits to transfer for each word.
- rgTX – Array of 16bit values (words) to transmit.
- cTX – Number of words to transmit.
- rgRX – Buffer for read words.
- cRX – Number of words to read.

```
FDwfDigitalSpiWriteRead32(HDWF hdwf, int cDQ, int cBitPerWord,
unsigned int *rgTX, int cTX, unsigned int *rgRX, int cRX)
```

**Description:** Performs SPI transfer of up to 32-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cDQ – Specify the DQ lines to use to transfer data.
- cBitPerWord – Specify the number of bits to transfer for each word.
- rgTX – Array of 32bit values (words) to write.
- cTX – Number of words to write.
- rgRX – Buffer for read words.
- cRX – Number of words to read.

```
FDwfDigitalSpiRead16(HDWF hdwf, int cDQ, int cBitPerWord,
unsigned short *rgRX, int cRX)
```

**Description:** Performs SPI read of up to 16-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cDQ – specify the DQ lines to use to transfer data.
- cBitPerWord – specify the number of bits to transfer for each word.
- rgRX – buffer for read words.
- cRX – number of words to read.

```
FDwfDigitalSpiRead32(HDWF hdwf, int cDQ, int cBitPerWord,
unsigned int *rgRX, int cRX)
```

**Description:** Performs SPI read of up to 32-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cDQ – specify the DQ lines to use to transfer data.
- cBitPerWord – specify the number of bits to transfer for each word.
- rgRX – buffer for read words.
- cRX – number of words to read.

```
FDwfDigitalSpiWrite16(HDWF hdwf, int cDQ, int cBitPerWord,  
unsigned short *rgTX, int cTX)
```

**Description:** Performs SPI read of up to 16-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cDQ – specify the DQ lines to use to transfer data.
- cBitPerWord – specify the number of bits to transfer for each word.
- rgTX – array of 16-bit values (words) to write.
- cTX – number of words to write.

```
FDwfDigitalSpiWrite32(HDWF hdwf, int cDQ, int cBitPerWord,  
unsigned int *rgTX, int cTX)
```

**Description:** Performs SPI read of up to 32-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cDQ – specify the DQ lines to use to transfer data.
- cBitPerWord – specify the number of bits to transfer for each word.
- rgTX – array of 32-bit values (int) to write.
- cTX – number of bytes to write.

```
FDwfDigitalSpiCmdWriteRead (HDWF hdwf,
int cBitCmd, unsigned char cmd, int cDummy,
int cDQ, int cBitPerWord,
unsigned char *rgTX, int cTX, unsigned char *rgRX, int cRX)
```

**Description:** Performs SPI transfer of up to 8bit words. This function is intended for standard MOSI/MISO (cDQ 1) operations, but it can be used for other modes as long only write (rgTX/cTX) or read (rgRX/cRX) is specified. The number of clock signals generated is the maximum of cTX and cRX.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – Specify the DQ lines to use to transfer data
- cBitPerWord – specify the number of bits to transfer for each word.
- rgTX – array of 8bit values (byte words) to write.
- cTX – number of words to write.
- rgRX – buffer for read words.
- cRX – number of words to read.

```
FDwfDigitalSpiCmdWriteOne (HDWF hdwf,
int cBitCmd, unsigned char cmd, int cDummy,
int cDQ, int cBits, unsigned int vTX)
```

**Description:** Performs SPI transmit of up to 32 bits. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – Specify the DQ lines to use to transfer data.
- cBits – Specify the number of bits to transmit.
- vTX – Specify the data to transmit.

```
FDwfDigitalSpiCmdReadOne (HDWF hdwf,
int cBitCmd, unsigned char cmd, int cDummy,
int cDQ, int cBits, unsigned int *pRX)
```

**Description:** Performs SPI reception of up to 32 bits. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – Specify the DQ lines to use to transfer data.
- cBits – Specify the number of bits to receive.
- pRX – Pointer to variable to return the received bits.

```
FDwfDigitalSpiCmdWrite(HDWF hdwf,
int cBitCmd, unsigned char cmd, int cDummy,
int cDQ, int cBitPerWord,
unsigned char *rgTX, int cTX)
```

**Description:** Performs SPI transmission of up to 8-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – Specify the DQ lines to use to transfer data.
- cBitPerWord – Specify the number of bits to transfer for each word.
- rgTX – Array of 8-bit values (words) to transmit.
- cTX – Number of words to transmit.

```
FDwfDigitalSpiCmdRead(HDWF hdwf,
int cBitCmd, unsigned char cmd, int cDummy,
int cDQ, int cBitPerWord, unsigned char *rgRX, int cRX)
```

**Description:** Performs SPI reception of up to 8-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – Specify the DQ lines to use to transfer data.
- cBitPerWord – Specify the number of bits to transfer for each word.
- rgRx – Buffer for receive words.
- cRx – Number of words to received.

```
FDwfDigitalSpiCmdWriteRead16(HDWF hdwf,
int cBitCmd, unsigned char cmd, int cDummy,
int cDQ, int cBitPerWord,
unsigned short *rgTX, int cTX, unsigned short *rgRX, int cRX)
```

**Description:** Performs SPI transfer of up to 16-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – Specify the DQ lines to use to transfer data.
- cBitPerWord – Specify the number of bits to transfer for each word.
- rgTX – Array of 16bit values (words) to transmit.
- cTX – Number of words to transmit.
- rgRx – Buffer for read words.
- cRx – Number of words to read.



```
FDwfDigitalSpiCmdWriteRead32 (HDWF hdwf,
int cBitCmd, unsigned char cmd, int cDummy,
int cDQ, int cBitPerWord,
unsigned int *rgTX, int cTX, unsigned int *rgRX, int cRX)
```

**Description:** Performs SPI transfer of up to 32-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – Specify the DQ lines to use to transfer data.
- cBitPerWord – Specify the number of bits to transfer for each word.
- rgTX – Array of 32bit values (words) to write.
- cTX – Number of words to write.
- rgRx – Buffer for read words.
- cRx – Number of words to read.

```
FDwfDigitalSpiCmdRead16 (HDWF hdwf,
int cBitCmd, unsigned char cmd, int cDummy,
int cDQ, int cBitPerWord, unsigned short *rgRX, int cRX)
```

**Description:** Performs SPI read of up to 16-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – specify the DQ lines to use to transfer data.
- cBitPerWord – specify the number of bits to transfer for each word.
- rgRx – buffer for read words.
- cRx – number of words to read.

```
FDwfDigitalSpiCmdRead32 (HDWF hdwf,
int cBitCmd, unsigned char cmd, int cDummy,
int cDQ, int cBitPerWord, unsigned int *rgRX, int cRX)
```

**Description:** Performs SPI read of up to 32-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – specify the DQ lines to use to transfer data.
- cBitPerWord – specify the number of bits to transfer for each word.
- rgRx – buffer for read words.
- cRx – number of words to read.

```
FDwfDigitalSpiCmdWrite16(HDWF hdwf,  
int cBitCmd, unsigned char cmd, int cDummy,  
int cDQ, int cBitPerWord, unsigned short *rgTX, int cTX)
```

**Description:** Performs SPI read of up to 16-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – specify the DQ lines to use to transfer data.
- cBitPerWord – specify the number of bits to transfer for each word.
- rgTX – array of 16-bit values (words) to write.
- cTX – number of words to write.

```
FDwfDigitalSpiCmdWrite32(HDWF hdwf,  
int cBitCmd, unsigned char cmd, int cDummy,  
int cDQ, int cBitPerWord, unsigned int *rgTX, int cTX)
```

**Description:** Performs SPI read of up to 32-bit words. See FDwfDigitalSpiWriteRead for more information.

**Parameters:**

- hdwf – Interface handle.
- cBitCmd – specify the number of command bits.
- cmd – Specify command value
- cDummy – Specify the number of dummy bits before data transfer
- cDQ – specify the DQ lines to use to transfer data.
- cBitPerWord – specify the number of bits to transfer for each word.
- rgTX – array of 32-bit values (int) to write.
- cTX – number of bytes to write.

## 13.3 I2C

See Digital\_I2c.py examples.

**FDwfDigitalI2cReset** (HDWF hdwf)

**Description:** Resets the I2C configuration to default value.

**Parameters:**

- hdwf – Interface handle.

**FDwfDigitalI2cClear** (HDWF hdwf, int \*pfFree)

**Description:** Verifies and tries to solve eventual bus lockup. The argument returns true, non-zero value if the bus is free.

**Parameters:**

- hdwf – Interface handle.

- pfFree – pointer to return the

**FDwfDigitalI2cRateSet** (HDWF hdwf, double hz)

**Description:** Sets the data rate.

**Parameters:**

- hdwf – Interface handle.

- hz – bit rate to set (default 100kHz).

**FDwfDigitalI2cTimeoutSet** (HDWF hdwf, double sec)

**Description:** Sets the data rate.

**Parameters:**

- hdwf – Interface handle.

- sec – timeout in seconds (default 1sec).

**FDwfDigitalI2cReadNakSet** (HDWF hdwf, int fNakLastReadByte)

**Description:** Specifies if the last read byte should be acknowledged or not. The I2C specifications require NAK, this parameter set to true.

**Parameters:**

- hdwf – Interface handle.

- fNakLastReadByte – value to set (default 1, true).

**FDwfDigitalI2cStretchSet** (HDWF hdwf, int fEnable)

**Description:** Enables or disables clock stretching.

**Parameters:**

- hdwf – Interface handle.

- fEnable – Enable (default) or disable the clock stretching.

```
FDwfDigitalI2cSclSet(HDWF hdwf, int idxChannel)
```

**Description:** Specifies the DIO channel to use for I2C clock.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – DIO channel to use for SCL.

```
FDwfDigitalI2cSdaSet(HDWF hdwf, int idxChannel)
```

**Description:** Specifies the DIO channel to use for I2C data.

**Parameters:**

- hdwf – Interface handle.
- idxChannel – DIO channel to use for SDA.

```
FDwfDigitalI2cWriteRead(HDWF hdwf, unsigned char adr8bits,  
unsigned char *rgbTx, int cTx, unsigned char *rgbRx, int cRx, int *pNak)
```

**Description:** Performs I2C write, repeated start and read. In case zero bytes are specified for read (cRx) only write and for zero write (cTx) only read is performed. The read/write bit in the address is controlled by the function. The returned NAK index returns one based index of the first negative acknowledged transfer byte, zero when all the bytes were acknowledged. When the first address is acknowledged it returns 1. Returns negative value for other communication failures like timeout.

**Parameters:**

- hdwf – Interface handle.
- adr8bits – specify the address in 8-bit format, bAAAAAAAX.
- rgbTX – array of bytes to write.
- cTX – number of bytes to write.
- rgbRx – buffer for read bytes.
- cRx – number of bytes to read.
- pNak – pointer to variable to return eventual NAK index.

```
FDwfDigitalI2cRead(HDWF hdwf, unsigned char adr8bits,  
unsigned char *rgbRx, int cRx, int *pNak)
```

**Description:** Performs I2C read. See DwfDigitalI2cWriteRead function for more information.

**Parameters:**

- hdwf – Interface handle.
- adr8bits – specify the address.
- rgbRx – buffer for read bytes.
- cRx – number of bytes to read.
- pNak – pointer to variable to return eventual NAK index.

```
FDwfDigitalI2cWrite(HDWF hdwf, unsigned char adr8bits,
unsigned char *rgbTx, int cTx, int *pNak)
```

**Description:** Performs I2C write. See DwfDigitalI2cWriteRead function for more information.

**Parameters:**

- hdwf – Interface handle.
- adr8bits – specify the address.
- rgbTX – array of bytes to write.
- cTX – number of bytes to write.
- pNak – pointer to variable to return eventual NAK index.

```
FDwfDigitalI2cWriteOne(HDWF hdwf, unsigned char adr8bits,
unsigned char bTx, int *pNak)
```

**Description:** Performs I2C write of one byte. See DwfDigitalI2cWriteRead function for more information.

**Parameters:**

- hdwf – Interface handle.
- adr8bits – specify the address.
- bTX – bytes to write.
- pNak – pointer to variable to return eventual NAK index.

```
FDwfDigitalI2cSpyStart(HDWF hdwf)
```

**Description:** Start I2C spy.

**Parameters:**

- hdwf – Interface handle.

```
FDwfDigitalI2cSpyStatus(HDWF hdwf,
int *fStart, int *fStop, unsigned char *rgData, int *cData, int *iNak)
```

**Description:** Decoded I2C communication. Active values are returned when when Stop is met, before ReStart or the requested number of data is decoded.

See example: SDK/ samples/ py/ Digital\_I2c\_Spy.py

**Parameters:**

- hdwf – Interface handle.
- fStart – Return 1 if start and 2 if restart was detected, otherwise 0.
- fStop – Return 1 if stop was detected, otherwise 0.
- rgData – Buffer for data. First value is address when fStart is not zero.
- cData – Specify the maximum number of data and returns the decoded words.
- iNak – Returns NAK index + 1. Negative values indicated error.

## 13.4 CAN

See Digital\_Can.py example.

**FDwfDigitalCanReset**(HDWF hdwf)

**Description:** Resets the CAN configuration to default value. Use FDwfDigitalOutReset to reset the output.

**Parameters:**

- hdwf – Interface handle.

**FDwfDigitalCanRateSet**(HDWF hdwf, double hz)

**Description:** Sets the data rate.

**Parameters:**

- hdwf – Interface handle.

- hz – data rate to set.

**FDwfDigitalCanPolaritySet**(HDWF hdwf, double fInvert)

**Description:** Sets the signal polarity.

**Parameters:**

- hdwf – Interface handle.

- fInvert – 0 normal, 1 invert.

**FDwfDigitalCanTxSet**(HDWF hdwf, int idxChannel)

**Description:** Specifies the DIO channel to use for transmission.

**Parameters:**

- hdwf – Interface handle.

- idxChannel – DIO channel to use for TX.

**FDwfDigitalCanRxSet**(HDWF hdwf, int idxChannel)

**Description:** Specifies the DIO channel to use for reception.

**Parameters:**

- hdwf – Interface handle.

- idxChannel – DIO channel to use for RX.

```
FDwfDigitalCanTx(HDWF hdwf, int vID, int fExtended, int fRemote,  
int cDLC, unsigned char *rgTX)
```

**Description:** Performs a CAN transmission. Specifying -1 for vID it initializes the TX channel.

**Parameters:**

- hdwf – Interface handle.
- vID – identifier.
- fExtended – 0 base frame, 1 extended frame; 11- or 29-bit identifier.
- fRemote – 0 data frame, 1 remote request.
- rgTX - array of data bytes to send.

```
FDwfDigitalCanRx(HDWF hdwf, int *pvID, int *pfExtended, int *pfRemote,  
int *pcDLC, unsigned char *rgRX, int cRX, int *pvStatus)
```

**Description:** Returns the received frames since the last call. With cRX zero initializes the reception.

**Parameters:**

- hdwf – Interface handle.
- pvID – pointer to return the identifier.
- pfExtended - pointer to return the extended bit value.
- pfRemote – pointer to return the remote bit value.
- pcDLC – pointer to return the data length code.
- rgRX – pointer to array to return the data bytes.
- cRX - the maximum number of data bytes to receive, rgRX buffer size
- pvStatus – pointer to return the function status:
  - 0 - nothing received.
  - 1 – frame received without error.
  - 2 – frame received with bit stuffing error.
  - 3 – frame received with CRC error.

## 13.5 SWD

See Digital\_Swd.py example.

**FDwfDigitalSwdReset** (HDWF hdwf)

**Description:** Resets the SWD configuration to default value. Use FDwfDigitalOutReset to reset the outputs and FDwfDigitalSwdClear for SWD reset command!

**Parameters:**

- hdwf – Interface handle.

**FDwfDigitalSwdRateSet** (HDWF hdwf, double hz)

**Description:** Sets the data rate.

**Parameters:**

- hdwf – Interface handle.

- hz – data rate to set.

**FDwfDigitalSwdCkSet** (HDWF hdwf, double idxChannel)

**Description:** Specifies the channel to use for clock signal, default DIO-0.

**Parameters:**

- hdwf – Interface handle.

- idxChannel – DIO channel to use for clock.

**FDwfDigitalSwdIoSet** (HDWF hdwf, int idxChannel)

**Description:** Specifies the channel to use for data signal, default DIO-1.

**Parameters:**

- hdwf – Interface handle.

- idxChannel – DIO channel to use for data.

**FDwfDigitalSwdTurnSet** (HDWF hdwf, int cTurn)

**Description:** Specifies the turnaround length, default 1, valid values 1,2,3,4

**Parameters:**

- hdwf – Interface handle.

- cTurn – turnaround cycles.

**FDwfDigitalSwdTrailSet** (HDWF hdwf, int cTrail)

**Description:** Specifies the trail length for data transfers, default 0

**Parameters:**

- hdwf – Interface handle.

- cTrail – trail cycles after data transfer.



---

**FDwfDigitalSwdParkSet**(HDWF hdwf, int fDrive)

---

**Description:** Specifies to drive or not the park bit.**Parameters:**

- hdwf – Interface handle.
- fDrive – 1 to drive and 0 for no drive.

---

**FDwfDigitalSwdNakSet**(HDWF hdwf, int fContinue)

---

**Description:** Specifies to continue sequence when not acknowledged.**Parameters:**

- hdwf – Interface handle.
- fDrive – 1 to continue and 0 to stop on no acknowledgement.

---

**FDwfDigitalSwdIoIdleSet**(HDWF hdwf, int fHigh)

---

**Description:** Specifies the data IO line for idle.**Parameters:**

- hdwf – Interface handle.
- fHigh – 0 low, 1 high, 2 three state.

---

**FDwfDigitalSwdClear** (HDWF hdwf, `int` cReset, `int` cTrail)

---

**Description:** Performs a SWD reset.**Parameters:**

- hdwf – Interface handle.
- cReset – reset length.
- cTrail – trail length

---

**FDwfDigitalSwdWrite** (HDWF hdwf, `int` APnDP, `int` A32, `int` \*pAck, `unsigned char` Write)

---

**Description:** Performs a SWD write.**Parameters:**

- hdwf – Interface handle.
- APnDP – 1 for AccessPort and 0 for DataPort.
- A32 – address bits 3:2
- pAck – returns acknowledgement bits: 1 OK, 2 WAIT, 4 FAILURE
- Write – data to write.

---

**FDwfDigitalSwdRead** (HDWF hdwf, `int` APnDP, `int` A32, `int` \*pAck, `unsigned char` \*pRead, `int` \*pCrc)

---

**Description:** Performs a SWD read.**Parameters:**

- hdwf – Interface handle.
- APnDP – 1 for AccessPort and 0 for DataPort.
- A32 – address bits 3:2
- pAck – returns acknowledgement bits: 1 OK, 2 WAIT, 4 FAILURE
- pRead – read data.
- pCrc – 0 for parity ok and 1 for error.

## 14 Devices

### 14.1 Electronics Explorer

The Electronics Explorer has nine AnalogIO channels. The master enable validates the power supplies and reference voltages.

| Channel | Node | Name    | Description                                                                              |
|---------|------|---------|------------------------------------------------------------------------------------------|
| 0       |      | Vcc     | Fixed supply                                                                             |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                                              |
|         | 1    | Voltage | Select voltage 3.3V or 5V<br>Status returns the voltage reading                          |
|         | 2    | Current | Status returns the current reading                                                       |
| 1       |      | VP+     | Positive supply                                                                          |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                                              |
|         | 1    | Voltage | Specify voltage level, 0 to 9V<br>Status returns the voltage reading                     |
|         | 2    | Current | Specify the current limitation between 0 and 1.5A<br>Status returns the current reading  |
| 2       |      | VP-     | Negative supply                                                                          |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                                              |
|         | 1    | Voltage | Specify voltage level, 0 to -9V<br>Status returns the voltage reading                    |
|         | 2    | Current | Specify the current limitation between 0 and -1.5A<br>Status returns the current reading |
| 3       |      | Ref1    | Voltage Reference 1                                                                      |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                                              |
|         | 1    | Voltage | Specify voltage level between -10V and -10V                                              |
| 4       |      | Ref2    | Voltage Reference 2                                                                      |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                                              |
|         | 1    | Voltage | Specify voltage level between -10V and -10V                                              |
| 5       |      | Vmtr1   | Voltmeter 1                                                                              |
|         | 0    | Voltage | Status returns the voltage reading, -15V to 15V                                          |
| 6       |      | Vmtr2   | Voltmeter 2                                                                              |
|         | 0    | Voltage | Status returns the voltage reading, -15V to 15V                                          |
| 7       |      | Vmtr3   | Voltmeter 3                                                                              |
|         | 0    | Voltage | Status returns the voltage reading, -15V to 15V                                          |
| 8       |      | Vmtr4   | Voltmeter 4                                                                              |
|         | 0    | Voltage | Status returns the voltage reading, -15V to 15V                                          |

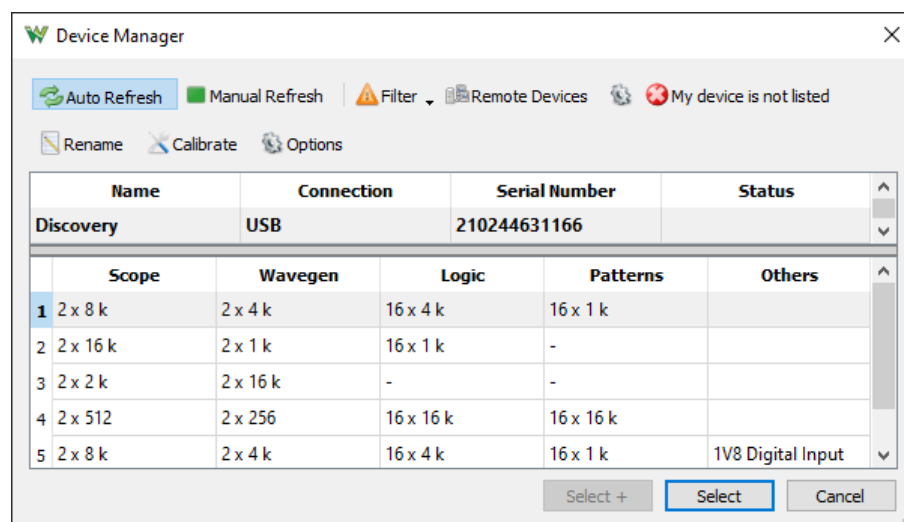
## 14.2 Analog Discovery

The Analog Discovery has four AnalogIO channels. The master enable validates the power supplies.

| Channel | Node | Name    | Description                                                |
|---------|------|---------|------------------------------------------------------------|
| 0       |      | V+      | Positive supply                                            |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                |
| 1       |      | V-      | Negative supply                                            |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                |
| 2       |      | SYS     | USB power monitor                                          |
|         | 0    | Voltage | Status returns the line voltage                            |
|         | 1    | Current | Status returns the line current                            |
|         | 2    | Temp    | Status returns the device temperature                      |
| 3       |      | V+-     | Power supply monitor                                       |
|         | 0    | Voltage | Status returns the voltage input for the supply regulators |
|         | 1    | Current | Status returns the current taken by the supply regulators  |

There are five configurations available for the device. See `FDwfDeviceConfigOpen`.

```
c:\Program Files (x86)\Diligent\WaveFormsSDK\samples\py>python Device_Enumeration.py
DWF Version: b'3.18.19'
Number of Devices: 1
-----
Device 0 :
  Name: b'Analog Discovery' b'SN:210244631166'
  ID: 2 rev: C 0x3
  Configurations:
  0. AnalogIn: 2 x 8192 AnalogOut: 2 x 4096 DigitalIn: 16 x 4096 DigitalOut: 16 x 1024
  1. AnalogIn: 2 x 16384 AnalogOut: 2 x 1024 DigitalIn: 16 x 1024 DigitalOut: 0 x 0
  2. AnalogIn: 2 x 2048 AnalogOut: 2 x 16384 DigitalIn: 0 x 0 DigitalOut: 0 x 0
  3. AnalogIn: 2 x 512 AnalogOut: 2 x 256 DigitalIn: 16 x 16384 DigitalOut: 16 x 16384
  4. AnalogIn: 2 x 8192 AnalogOut: 2 x 4096 DigitalIn: 16 x 4096 DigitalOut: 16 x 1024 1V8_Digital_Input
```



## 14.3 Analog Discovery 2

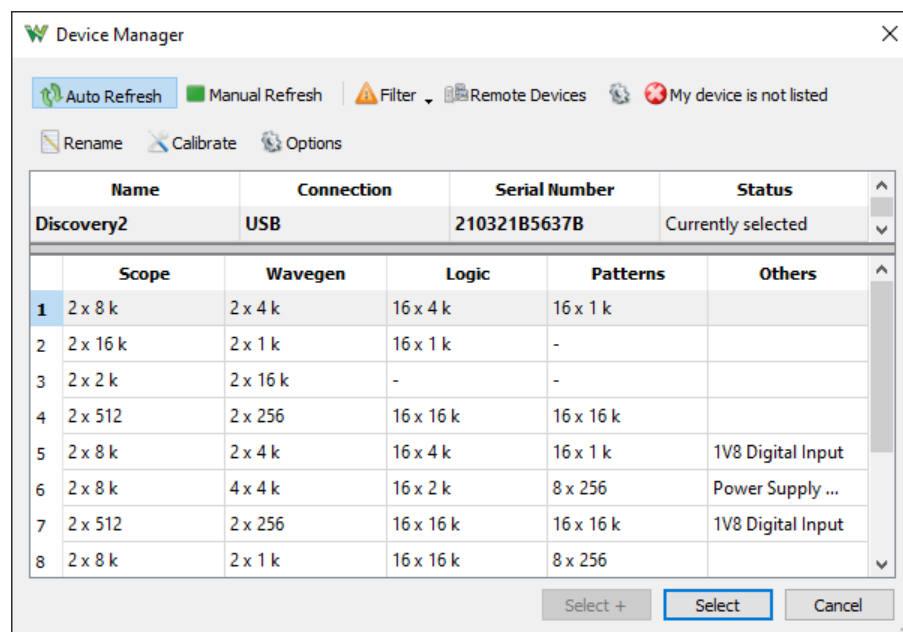
The Analog Discovery 2 has four AnalogIO channels. The master enable validates the power supplies.

| Channel | Node | Name    | Description                                 |
|---------|------|---------|---------------------------------------------|
| 0       |      | V+      | Positive supply                             |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0 |
|         | 1    | Voltage | Specify voltage level, 0.5 to 5V            |
| 1       |      | V-      | Negative supply                             |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0 |
|         | 1    | Voltage | Specify voltage level, -0.5 to -5V          |
| 2       |      | USB     | USB power monitor                           |
|         | 0    | Voltage | Status returns the line voltage             |
|         | 1    | Current | Status returns the line current             |
|         | 2    | Temp    | Status returns the device temperature       |
| 3       |      | AUX     | AUX power monitor                           |
|         | 0    | Voltage | Status returns the line voltage             |
|         | 1    | Current | Status returns the line current             |

There are eight configurations available for the device. See `FDwfDeviceConfigOpen`.

```
c:\Program Files (x86)\Diligent\WaveFormsSDK\samples\py>python Device_Enumeration.py
DWF Version: b'3.18.19'
Number of Devices: 1
```

```
-----
Device 0 :
  Name: b'Analog Discovery 2' b'SN:210321B5637B'
  ID: 3 rev: G 0x7
  Configurations:
    0. AnalogIn: 2 x 8192   AnalogOut: 2 x 4096   DigitalIn: 16 x 4096   DigitalOut: 16 x 1024
    1. AnalogIn: 2 x 16384  AnalogOut: 2 x 1024   DigitalIn: 16 x 1024   DigitalOut: 0 x 0
    2. AnalogIn: 2 x 2048   AnalogOut: 2 x 16384  DigitalIn: 0 x 0       DigitalOut: 0 x 0
    3. AnalogIn: 2 x 512    AnalogOut: 2 x 256    DigitalIn: 16 x 16384  DigitalOut: 16 x 16384
    4. AnalogIn: 2 x 8192   AnalogOut: 2 x 4096   DigitalIn: 16 x 4096   DigitalOut: 16 x 1024   1V8_Digital_Input
    5. AnalogIn: 2 x 8192   AnalogOut: 4 x 4096   DigitalIn: 16 x 2048   DigitalOut: 8 x 256     Power_Supply_Waveform_Generator
    6. AnalogIn: 2 x 512    AnalogOut: 2 x 256    DigitalIn: 16 x 16384  DigitalOut: 16 x 16384  1V8_Digital_Input
    7. AnalogIn: 2 x 8192   AnalogOut: 2 x 1024   DigitalIn: 16 x 16384  DigitalOut: 8 x 256
```



## 14.4 Digital Discovery

For the DigitalOut and IO functions, and AnalogIO DIOPP/PE the indexing 15:0 refers to DIO39:24. The Digital Discovery has three AnalogIO channels. The master enable activates the VIO output.

| Channel | Node | Name      | Description                                                                                                                                                                        |
|---------|------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0       | 0    | Voltage   | Configures the digital voltage between 1.2V and 3.3V                                                                                                                               |
|         | 1    | DINPP     | Configures the weak pull for DIN lines with values:<br>0 down, 0.5 middle, 0.75 up, 1 up<br>See <a href="#">Digital Discovery RM Input Dividers</a>                                |
|         | 2    | DIOPE     | Configure pull enable (1) for DIO 39-24 as bit field set, like:<br>0x00F1 enables for DIO 31,30,29,28 and 24<br>See <a href="#">Digital Discovery RM I/O Level Translators</a>     |
|         | 3    | DIOPP     | Configure pull up/down (1/0) for DIO 39-24 as bit field set                                                                                                                        |
|         | 4    | Drive     | Configure drive strength for DIO lines:<br>0 (auto), 2mA, 4mA, 6mA, 8mA, 12mA, 16mA                                                                                                |
|         | 5    | Slew      | Configure slew rate for DIO lines:<br>quietio, slow, fast<br>See <a href="#">Xilinx DS162</a>                                                                                      |
| 1       | 6    | Frequency | Configure, fine adjust the system frequency between:<br>50-100/125MHz (50e6-100e6/125e6)<br>Zero value does not change the last setting.<br>Negative resets to the default 100MHz. |
|         |      | VIO       | VIO power monitor<br>See <a href="#">Digital Discovery RM Power Supplies</a>                                                                                                       |
|         | 0    | Voltage   | Status returns the VIO voltage reading                                                                                                                                             |
|         | 1    | Current   | Status returns the VIO current reading                                                                                                                                             |
|         | 2    | USB       | USB power monitor                                                                                                                                                                  |
|         | 0    | Voltage   | Status returns the USB voltage reading                                                                                                                                             |
| 2       | 1    | Current   | Status returns the USB current reading                                                                                                                                             |

Based on the FDwfDigitalInInputOrderSet and FDwfDigitalInSampleFormatSet the captured samples and the configured FDwfDigitalInTriggerSet bits are as follows:

| Order | Format | Samples                  | Bits        |
|-------|--------|--------------------------|-------------|
| DIN   | 8      | DIN [7-0]                | bit 7 to 0  |
| DIN   | 16     | DIN [15-0]               | bit 15 to 0 |
| DIN   | 32     | DIO [31-24] - DIN [23-0] | bit 31 to 0 |
| DIO   | 8      | DIO [31-24]              | bit 7 to 0  |
| DIO   | 16     | DIO [39-24]              | bit 15 to 0 |
| DIO   | 32     | DIN [15-0] - DIO [39-24] | bit 31 to 0 |

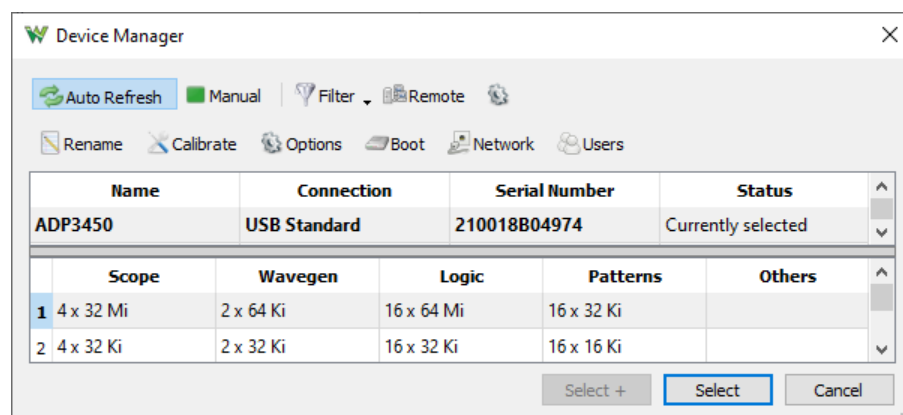
## 14.5 Analog Discovery Pro 3000 Series

The ADP3450 and ADP3250 have four AnalogIO channels. The master enable activates the VIO output.

| Channel | Node | Name        | Description                                                                                             |
|---------|------|-------------|---------------------------------------------------------------------------------------------------------|
| 0       | 0    | Voltage     | Configures the digital voltage between 1.2V and 3.3V                                                    |
|         | 1    | DIOPE       | Configure pull enable (1) for DIO 15-0 as bit field set, like: 0xF001 enables for DIO 15,14,13,12 and 0 |
|         | 2    | DIOPP       | Configure pull up/down (1/0) for DIO 15-0 as bit field set                                              |
| 1       |      | Zynq        | System monitor of the device Zynq IC                                                                    |
|         | 0    | Temperature | Temperature in degree Celsius                                                                           |
|         | 1    | VccInt      | 1V rail                                                                                                 |
|         | 2    | VccAux      | 1.8V rail                                                                                               |
|         | 3    | VccBRam     | 1V rail                                                                                                 |
|         | 4    | VccPlnt     | 1V rail                                                                                                 |
|         | 5    | VccPAux     | 1.8V rail                                                                                               |
|         | 6    | VccDDR      | 1.35V rail                                                                                              |
| 2       |      | ZynqMin     | Minimum values since power on or reset/reboot                                                           |
|         | 0-6  |             |                                                                                                         |
| 3       |      | ZynqMax     | Maximum values since power on or reset/reboot                                                           |
|         | 0-6  |             |                                                                                                         |

There are two configurations available for the device. See `FDwfDeviceConfigOpen`.

```
c:\Program Files (x86)\Diligent\WaveFormsSDK\samples\py>python Device_Enumeration.py
DWF Version: b'3.18.19'
Number of Devices: 1
-----
Device 0 :
  Name: b'Analog Discovery Pro 3450' b'SN:210018B04974'
  ID: 6 rev: D 0x204
  Configurations:
    0. AnalogIn: 4 x 32768 AnalogOut: 2 x 32768 DigitalIn: 16 x 32768 DigitalOut: 16 x 16384
    1. AnalogIn: 4 x 65536 AnalogOut: 2 x 4096 DigitalIn: 16 x 8192 DigitalOut: 16 x 1024
```



The `(FDwfEnumDeviceType/pDeviceRevision >> 8) & 3` identifies boot/connection mode:

- 0 – Linux
- 1 - Standard Recovery
- 2 - Standard
- 3 - AXI (embedded Linux interface)

## 14.6 Analog Discovery Pro 5250

The ADP5250 has six AnalogIO channels. The master enable validates the three power supplies.

The following are not supported:

- AnalogIn: acquisition modes: shift, screen, record; noise buffer, average filter, trigger holdoff, trigger types: pulse, transition, window
- AnalogOut: play mode, device noise, AM/FM, wait, run, repeat
- DigitalIn: acquisition modes: shift, screen, record, sync; noise buffer, timeout trigger, protocol trigger,
- DigitalOut and Protocols (only I2C and SPI master)

The I2C signals are fixed SCL DIO-6, SDA DIO-7; it supports 100kHz, 200kHz and 1MHz; it does not support writing 0 bytes.

The SPI signals are CLK DIO-0, MOSI DIO-1, MISO DIO-2, CS DIO-3; supported word length is 8 bits; it does not support delay adjustment.

| Channel | Node | Name    | Description                                                                             |
|---------|------|---------|-----------------------------------------------------------------------------------------|
| 0       |      | p6V     | +6V supply channel                                                                      |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                                             |
|         | 1    | Voltage | Specify voltage level, 0 to 6V<br>Status returns the voltage reading                    |
|         | 2    | Current | Specify the current limitation between 0 and 1A<br>Status returns the current reading   |
| 1       |      | p25V    | +25V supply channel                                                                     |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                                             |
|         | 1    | Voltage | Specify voltage level, 0 to 25V<br>Status returns the voltage reading                   |
|         | 2    | Current | Specify the current limitation between 0 and 0.5A<br>Status returns the current reading |
| 2       |      | n25V    | -25V supply channel                                                                     |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                                             |
|         | 1    | Voltage | Specify voltage level, 0 to -25V<br>Status returns the voltage reading                  |
|         | 2    | Current | Specify the current limitation between 0 and 0.5A<br>Status returns the current reading |
| 3       |      | DMM     | Digital Multimeter                                                                      |
|         | 0    | Enable  | Enables or disables the DMM, 1/0                                                        |
|         | 1    | Mode    | Select measurement, DwfDmm                                                              |
|         | 2    | Range   | Specify the input range, 0 stands for auto range<br>Status returns the current range    |
|         | 3    | Meas    | Status returns the measurement value                                                    |
|         | 4    | Raw     | Not used                                                                                |
|         | 5    | Input   | Input impedance for DCVoltage 10G option, 1e7 or 1e10                                   |
| 4       |      | System  | System Monitor                                                                          |
|         | 0    | Adj     | Adjustment temeperature                                                                 |
|         | 1    | DMM     | DMM temperature                                                                         |
|         | 2    | Core    | Core temperature                                                                        |
|         | 3    | Fan     | Fan speed in RPM                                                                        |
| 5       |      |         |                                                                                         |
|         | 0    | Digital | Adjust DIN threshold voltage between 0 and 2V                                           |



## 14.7 Eclipse Z7 Zmods

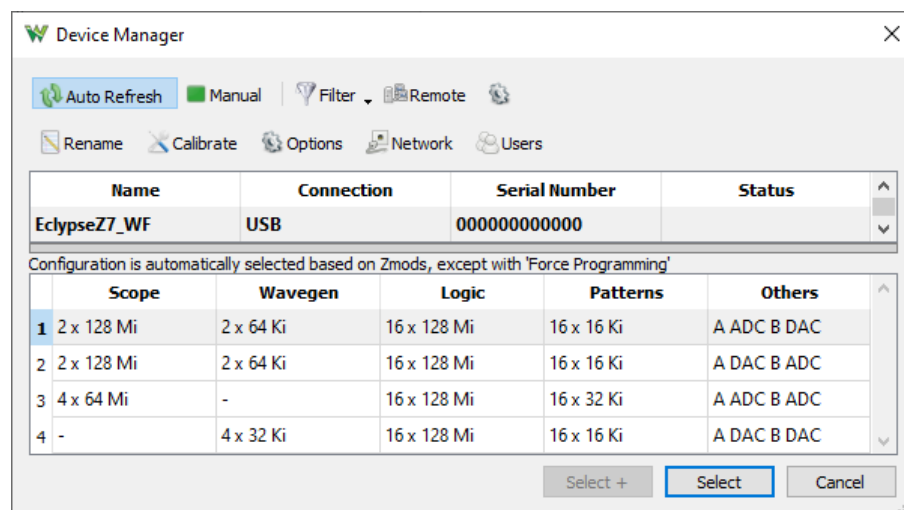
The Eclipse Z7 Zmod support provides three AnalogIO channels.

| Channel | Node | Name        | Description                                   |
|---------|------|-------------|-----------------------------------------------|
| 0       |      | Zynq        | System monitor of the device Zynq IC          |
|         | 0    | Temperature | Temperature in degree Celsius                 |
|         | 1    | VccInt      | 1V rail                                       |
|         | 2    | VccAux      | 1.8V rail                                     |
|         | 3    | VccBRam     | 1V rail                                       |
|         | 4    | VccPInt     | 1V rail                                       |
|         | 5    | VccPAux     | 1.8V rail                                     |
|         | 6    | VccDDR      | 1.35V rail                                    |
| 1       |      | ZynqMin     | Minimum values since power on or reset/reboot |
|         | 0-6  |             |                                               |
| 2       |      | ZynqMax     | Maximum values since power on or reset/reboot |
|         | 0-6  |             |                                               |

There are four configurations provided to support combinations of Zmod Scope/Digitizer (ADC) or AWG (DAC) loaded in ZMOD A and B. The suitable configuration is automatically selected based on the loaded Zmods.

Note that the software does not support combining Scope and Digitizer Zmods.

```
c:\Program Files (x86)\Diligent\WaveFormsSDK\samples\py>python Device_Enumeration.py
DWF Version: b'3.18.45'
Number of Devices: 1
-----
Device 0 :
  Name: SN:000000000000
  ID: 7 rev: A 0x201
  Configurations:
  0. AnalogIn: 2 x 32768  AnalogOut: 2 x 32768  DigitalIn: 16 x 32768  DigitalOut: 16 x 32768  A_ADC_B_DAC
  1. AnalogIn: 2 x 32768  AnalogOut: 2 x 32768  DigitalIn: 16 x 32768  DigitalOut: 16 x 32768  A_DAC_B_ADC
  2. AnalogIn: 4 x 32768  AnalogOut: 0 x 0      DigitalIn: 16 x 32768  DigitalOut: 16 x 32768  A_ADC_B_ADC
  3. AnalogIn: 0 x 0      AnalogOut: 4 x 32768  DigitalIn: 16 x 16384  DigitalOut: 16 x 16384  A_DAC_B_DAC
```



## 14.8 Discovery Power Supply 3340

The DPS3340 has four AnalogIO channels. The master enable validates the power supplies.

The analog-in functions can be used for voltage/current readback monitoring-logging with oscilloscope-like functions, triggered capture and stream-recording. This can work in parallel with analog-io readback, \*Status functions.

The analog-out functions can be used for slow voltage or current arbitrary waveform generator or play-streaming. The analog-io channel settings have priority over analog-out. To use the analog-out functionality on a channel this should not be enabled from the analog-io function.

| Channel | Node | Name       | Description                                                                                                                                                                                         |
|---------|------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0       |      | Output 1   | +5V power supply                                                                                                                                                                                    |
|         | 0    | Enable     | Enables or disables the supply channel, 1/0                                                                                                                                                         |
|         | 1    | Voltage    | Specify voltage level, 1V to 5V<br>Status returns the voltage reading                                                                                                                               |
|         | 2    | Current    | Specify the current limitation between 10mA and 3A<br>Status returns the current reading                                                                                                            |
|         | 3    | Protection | Specify the protection mode<br>0 – current limitation<br>1 – channel fuse, disable channel when current setting is reached<br>2 – master fuse, disable all channels when current setting is reached |
|         | 4    | Fuse       | Specify fuse timeout 0 to 60sec, in 1ms steps                                                                                                                                                       |
|         | 5    | Prime      | Specify fuse timeout for enable or voltage or current change                                                                                                                                        |
| 1       |      | Output 2   | -15V power supply                                                                                                                                                                                   |
|         | 0    | Enable     | Enables or disables the supply channel, 1/0                                                                                                                                                         |
|         | 1    | Voltage    | Specify voltage level, -1V to -15V<br>Status returns the voltage reading                                                                                                                            |
|         | 2    | Current    | Specify the current limitation between 10mA and 3A<br>Status returns the current reading                                                                                                            |
|         | 3    | Protection | Specify the protection mode                                                                                                                                                                         |
|         | 4    | Fuse       | Specify fuse timeout 0 to 60sec, in 1ms steps                                                                                                                                                       |
|         | 5    | Prime      | Specify fuse timeout for enable or voltage or current change                                                                                                                                        |
| 2       |      | Output 3   | +15V power supply                                                                                                                                                                                   |
|         | 0    | Enable     | Enables or disables the supply channel, 1/0                                                                                                                                                         |
|         | 1    | Voltage    | Specify voltage level, +1V to +15V<br>Status returns the voltage reading                                                                                                                            |
|         | 2    | Current    | Specify the current limitation between 10mA and 3A<br>Status returns the current reading                                                                                                            |
|         | 3    | Protection | Specify the protection mode                                                                                                                                                                         |
|         | 4    | Fuse       | Specify fuse timeout 0 to 60sec, in 1ms steps                                                                                                                                                       |
|         | 5    | Prime      | Specify fuse timeout for enable or voltage or current change                                                                                                                                        |
| 3       |      | Others     |                                                                                                                                                                                                     |
|         | 1    | Average    | Averaging for voltage/current readback, AnalogIO-status functions<br>1ms to 4Msec, in 1ms steps                                                                                                     |
|         | 2    | VDC19      | 19VDC voltage monitoring                                                                                                                                                                            |

## 14.9 Analog Discovery 3

The Analog Discovery 3 has four AnalogIO channels. The master enable validates the power supplies.

| Channel | Node | Name    | Description                                                                                                                                                                     |
|---------|------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0       |      | V+      | Positive supply                                                                                                                                                                 |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                                                                                                                                     |
|         | 1    | Voltage | Specify voltage level, 0.5 to 5V and status reading range $\pm 15V$                                                                                                             |
| 1       |      | V-      | Negative supply                                                                                                                                                                 |
|         | 0    | Enable  | Enables or disables the supply channel, 1/0                                                                                                                                     |
|         | 1    | Voltage | Specify voltage level, -0.5 to -5V and reading range $\pm 15V$                                                                                                                  |
| 2       |      | USB     | USB power monitor                                                                                                                                                               |
|         | 0    | Temp    | Status returns the device PCB temperature in $^{\circ}C$                                                                                                                        |
|         | 1    | Die     | Status returns the device FPGA temperature in $^{\circ}C$                                                                                                                       |
|         | 2    | VUSB    | Status returns the USB voltage in Volts                                                                                                                                         |
|         | 3    | IUSB    | Status returns the USB current in Amperes                                                                                                                                       |
|         | 4    | VAUX    | Status returns the auxiliary supply voltage in Volts                                                                                                                            |
|         | 5    | IAUX    | Status returns the auxiliary supply current in Amperes                                                                                                                          |
|         | 6    | VCC1    | Status returns the USB CC1 voltage                                                                                                                                              |
| 3       |      | VCC2    | Status returns the USB CC2 voltage                                                                                                                                              |
|         | 0    | Limit   | Specify the user supplies current limit 0.6W to 15W. Zero is the default auto based on USB CC and AUX supply presence. Status returns the current value set or auto determined. |

There are six configurations available for the device. See `FDwfDeviceConfigOpen`.

```
c:\Program Files (x86)\Diligent\WaveFormsSDK\samples\py>python Device_Enumeration.py
```

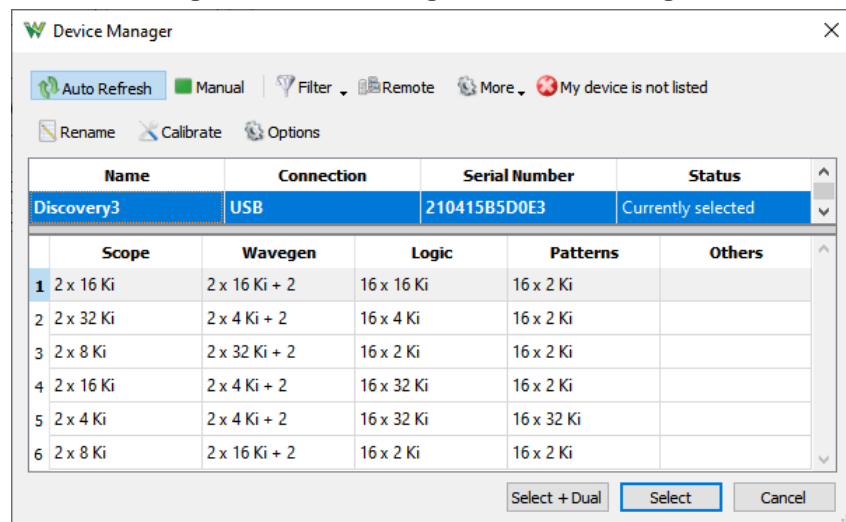
Device 0 :

Name: Analog Discovery 3 SN:210415B5D0E3

ID: 10 rev: C 0x3

Configurations:

|    |                     |                      |                       |                        |
|----|---------------------|----------------------|-----------------------|------------------------|
| 0. | AnalogIn: 2 x 16384 | AnalogOut: 4 x 16384 | DigitalIn: 16 x 16384 | DigitalOut: 16 x 2048  |
| 1. | AnalogIn: 2 x 32768 | AnalogOut: 4 x 4096  | DigitalIn: 16 x 4096  | DigitalOut: 16 x 2048  |
| 2. | AnalogIn: 2 x 8192  | AnalogOut: 4 x 32768 | DigitalIn: 16 x 2048  | DigitalOut: 16 x 2048  |
| 3. | AnalogIn: 2 x 16384 | AnalogOut: 4 x 4096  | DigitalIn: 16 x 32768 | DigitalOut: 16 x 2048  |
| 4. | AnalogIn: 2 x 4096  | AnalogOut: 4 x 4096  | DigitalIn: 16 x 32768 | DigitalOut: 16 x 32768 |
| 5. | AnalogIn: 2 x 8192  | AnalogOut: 4 x 16384 | DigitalIn: 16 x 2048  | DigitalOut: 16 x 2048  |



## 15 Deprecated functions

The following functions are replaced by `FDwfAnalogOutNode` \*providing access to the Amplitude and Frequency Modulation of Analog Out channels.

```
FDwfAnalogOutPlayStatus(HDWF hdwf, int idxChannel,
int *cdDataFree, int *cdDataLost, int *cdDataCorrupted)
```

**Description:** Retrieves information about the play process. The data lost occurs when the device generator is faster than the sample send process from the PC. In this case, the device buffer gets emptied and generated samples are repeated. Corrupt samples are a warning that the buffer might have been emptied while samples were sent to the device. In this case, try optimizing the loop for faster execution; or reduce the frequency or run time to be less or equal to the device buffer size (run time  $\leq$  buffer size/frequency).

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- cdDataFree – Pointer to variable to return the available free buffer space, the number of new samples that can be sent.
- cdDataLost – Pointer to variable to return the number of lost samples.
- cdDataCorrupted – Pointer to variable to return the number of samples that could be corrupted.

```
FDwfAnalogOutPlayData(HDWF hdwf, int idxChannel, double *rgdData, int cdData)
```

**Description:** Sends new data samples for play mode. Before starting the Analog Out instrument, prefill the device buffer with the first set of samples using the `AnalogOutDataSet` function. In the loop of sending the following samples, first call `AnalogOutStatus` to read the information from the device, then `AnalogOutPlayStatus` to find out how many new samples can be sent, then send the samples with `AnalogOutPlayData`.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- rgdData – Pointer to samples array to be sent to the device.
- cdData – Number of samples to send.

```
FDwfAnalogOutEnableSet(HDWF hdwf, int idxChannel, int fEnable)
```

**Description:** Enables or disables the channel specified by `idxChannel`. With channel index -1, each Analog Out channel enable will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- fEnable – TRUE to enable, FALSE to disable.

**FDwfAnalogOutEnableGet**(HDWF hdwf, int idxChannel, int \*pfEnable)

**Description:** Verifies if a specific channel is enabled or disabled.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pfEnable – Pointer to variable to receive enabled state.

**FDwfAnalogOutFunctionInfo**(HDWF hdwf, int idxChannel, int \*pfsfunc)

**Description:** Returns the supported generator function options. They are returned (by reference) as a bit field. This bit field can be parsed using the IsBitSet Macro. Individual bits are defined using the FUNC constants in dwf.h. These are:

| FUNC Constants    |    | FUNC Constant Capabilities                                                      |
|-------------------|----|---------------------------------------------------------------------------------|
| funcDC            | 0  | Generate DC value set as offset.                                                |
| funcSine          | 1  | Generate sine waveform.                                                         |
| funcSquare        | 2  | Generate square waveform.                                                       |
| funcTriangle      | 3  | Generate triangle waveform.                                                     |
| funcRampUp        | 4  | Generate a waveform with a ramp-up voltage at the beginning.                    |
| funcRampDown      | 5  | Generate a waveform with a ramp-down voltage at the end.                        |
| funcNoise         | 6  | Generate noise waveform from random samples.                                    |
| funcPulse         | 6  | Generate pulse waveform.                                                        |
| funcTrapezium     | 6  | Generate trapezium.                                                             |
| funcSinePower     | 6  | Generate sine power waveform.                                                   |
| funcCustomPattern | 28 | Generate waveform from custom repeated data with fixed sample rate.             |
| funcPlayPattern   | 29 | Generate waveform from custom data in stream play style with fixed sample rate. |
| funcCustom        | 30 | Generate waveform from custom repeated data.                                    |
| funcPlay          | 31 | Generate waveform from custom data in stream play style.                        |

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pfsfunc – Variable to receive the supported generator function options.

---

**FDwfAnalogOutFunctionSet**(HDWF hdwf, int idxChannel, FUNC func)

---

**Description:** Sets the generator output function for the specified instrument channel. With channel index -1, each enabled Analog Out channel function will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- func – Generator function option to set.

---

**FDwfAnalogOutFunctionGet**(HDWF hdwf, int idxChannel, FUNC \*pfunc)

---

**Description:** Retrieves the current generator function option for the specified instrument channel.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- ptrigsrc – Pointer to variable to receive the generator function option.

---

**FDwfAnalogOutFrequencyInfo**(  
HDWF hdwf, int idxChannel, double \*phzMin, double \*phzMax)

---

**Description:** Returns the supported frequency range for the instrument. The maximum value shows the DAC frequency. The frequency of the generated waveform: repetition frequency for standard types and custom data; DAC update for noise type; sample rate for play type.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- phzMin – Variable to receive the supported minimum frequency.
- phzMax – Variable to receive the supported maximum frequency.

---

**FDwfAnalogOutFrequencySet**(HDWF hdwf, int idxChannel, double hzFrequency)

---

**Description:** Sets the frequency. With channel index -1, each enabled Analog Out channel frequency will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- hzFrequency – Frequency value to set expressed in Hz.

---

**FDwfAnalogOutFrequencyGet**(HDWF hdwf, int idxChannel, double \*phzFrequency)

---

**Description:** Gets the currently set frequency for the specified channel on the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- hzFrequency – Pointer to variable to receive frequency value in Hz.

```
FDwfAnalogOutAmplitudeInfo(  
HDWF hdwf, int idxChannel, double *pvMin, double *pvMax)
```

**Description:** Retrieves the amplitude range for the specified channel on the instrument. The amplitude is expressed in Volts units for carrier and in percentage units (modulation index) for AM/FM.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pvMin – Minimum amplitude level or modulation index.
- pvMax – Maximal amplitude level or modulation index.

```
FDwfAnalogOutAmplitudeSet(HDWF hdwf, int idxChannel, double vAmplitude)
```

**Description:** Sets the amplitude or modulation index for the specified channel on the instrument. With channel index -1, each enabled Analog Out channel amplitude will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- vAmplitude – Amplitude of channel in Volts or modulation index in percentage.

```
FDwfAnalogOutAmplitudeGet(HDWF hdwf, int idxChannel, double *pvAmplitude)
```

**Description:** Gets the currently set amplitude or modulation index for the specified channel on the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pvAmplitude – Pointer to variable to receive amplitude value in Volts or modulation index in percentage.

```
FDwfAnalogOutOffsetInfo(HDWF hdwf, int idxChannel, double *pvMin, double  
*pvMax)
```

**Description:** Retrieves available the offset range in units of volts.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pvMin – Minimum offset voltage or modulation offset percentage.
- pvMax – Maximum offset voltage or modulation offset percentage.

---

```
FDwfAnalogOutOffsetSet(HDWF hdwf, int idxChannel, double vOffset)
```

---

**Description:** Sets the offset value for the specified channel on the instrument. With channel index -1, each enabled Analog Out channel offset will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- vOffset – Value to set voltage offset in Volts or modulation offset percentage.

---

```
FDwfAnalogOutOffsetGet(HDWF hdwf, int idxChannel, double *pvOffset)
```

---

**Description:** Gets the current offset value for the specified channel on the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pvOffset – Pointer to variable to receive offset value in Volts or modulation offset percentage.

The function above is used to:

---

```
FDwfAnalogOutSymmetryInfo(  
HDWF hdwf, int idxChannel, double *ppercentageMin, double *ppercentageMax)
```

---

**Description:** Obtains the symmetry (or duty cycle) range (0..100). This symmetry is supported for standard signal types. It the pulse duration divided by the pulse period.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- ppercentageMin – Minimum value of Symmetry percentage.
- ppercentageMax – Maximum value of Symmetry percentage.

---

```
FDwfAnalogOutSymmetrySet(  
HDWF hdwf, int idxChannel, double percentageSymmetry)
```

---

**Description:** Sets the symmetry (or duty cycle) for the specified channel on the instrument. With channel index -1, each enabled Analog Out channel symmetry will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- percentageSymmetry –Value of percentage of Symmetry (duty cycle).



```
FDwfAnalogOutSymmetryGet (  
HDWF hdwf, int idxChannel, double *ppercentageSymmetry)
```

**Description:** Gets the currently set symmetry (or duty cycle) for the specified channel of the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- ppercentageSymmetry — Pointer to variable to receive value of symmetry (duty cycle).

```
FDwfAnalogOutPhaseInfo (  
HDWF hdwf, int idxChannel, double *pdegreeMin, double *pdegreeMax)
```

**Description:** Retrieves the phase range (in degrees 0...360) for the specified channel of the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pdegreeMin – Minimum value of Phase (in degrees).
- pdegreeMax – Maximum value of Phase (in degrees).

```
FDwfAnalogOutPhaseSet (  
HDWF hdwf, int idxChannel, double degreePhase)
```

**Description:** Sets the phase for the specified channel on the instrument. With channel index -1, each enabled Analog Out channel phase will be configured to use the same, new option.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- degreePhase – Value of Phase in degrees.

```
FDwfAnalogOutPhaseGet (HDWF hdwf, int idxChannel, double *pdegreePhase)
```

**Description:** Gets the current phase for the specified channel on the instrument.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pdegreePhase – Pointer to variable to receive Phase value (in degrees).

```
FDwfAnalogOutDataInfo(  
HDWF hdwf, int idxChannel, int *pnSamplesMin, double *pnSamplesMax)
```

**Description:** Retrieves the minimum and maximum number of samples allowed for custom data generation.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- pnSamplesMin - Minimum number of samples available for custom data.
- pnSamplesMax – Maximum number of samples available for custom data.

```
FDwfAnalogOutDataSet(HDWF hdwf, int idxChannel, double *rgdData, int cdData)
```

**Description:** Sets the custom data or to prefill the buffer with play samples. The samples are double precision floating point values (rgdData) normalized to  $\pm 1$ .

With the custom function option, the data samples (cdData) will be interpolated to the device buffer size. The output value will be Offset + Sample\*Amplitude, for instance:

- 0 value sample will output: Offset.
- +1 value sample will output: Offset + Amplitude.
- -1 value sample will output: Offset – Amplitude.

**Parameters:**

- hdwf – Open interface handle on a device.
- idxChannel – Channel index.
- rgdData – Buffer of samples to set.
- cdData – Number of samples to set in rgdData.